网络编程迷踪录

李延瑞

2025 年 4 月 15 日

网络编程迷踪录

李延瑞（lyr.m2@live.cn）

Copyright © 2025 李延瑞

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation.

序

我从未想到过，终有一天我会写一份文档，介绍如何用 C 编写网络程序，原因是这么多年了，我从未学会网络编程。也许自从国内计算机专家将 socket 这个英文单词翻译「套接字」的时候，似乎已经注定了我很难学会它。

原本我计划是在 60 岁，假如那时我还活着，智力还算正常，再学习 C 语言版本的网络套接字编程。我甚至计划着，到那个时候，我依然学不会。

今年过年那段时间，得了甲流，没能回老家，滞留淄博。无力出去挣钱，百无聊赖，又温习了一遍作为小众语言的 Awk。不怎么用 Windows 的人应该都知道，Awk 语言主要是用于写那种一两行的脚本程序，以杀人越货的姿态处理日常文本。可我这次无意中发现，作为 Awk 语言诸多实现中最为通用的版本 GNU Awk，居然有一份文档，名为「TCP/IP Internetworking with gawk」。

你敢相信么？Awk 语言能够编写网络程序，这种事几乎相当于有生之年我能学会套接字编程，或者几乎相当于你愿意借给我￥10 万元。

Awk 都有如此追求，我为何没有呢？于是，我学习了 Awk 的网络编程，甚至为它写了一份笔记[1]。刚开始，我以为学会了 Awk 网络编程，从此以后再也不需要考虑学习网络编程的事。不过，我的乐观没能坚持太久，几天后我发现了，Awk 语言编写的网络服务端无法支持多个客户端的并发访问，充其量只能作为中小学生科普网络编程的教具。只是此时，Awk 用尽了它的花言巧语，已将我引诱到了迷雾重重的悬崖边上。我一脚踏空，跌了下去。

下坠者似流星划过空气。我竭力抓住了套接字地址，中止了坠落，而后竭力攀援而上。一路摸索着 `socket`, `connect`, `bind`, `listen`, `accept`, `recv`, `send`, `EAGAIN`, `fd_set`, `select`, `fork`, `wait`, `signal`, `pthread`, 达夫设备, 协程……它们或凸出，或凹陷，或荆棘，或灌木……向上，向上……终于回到崖顶。

现在，我计划在 60 岁的时候，能像很多人那样，好好度过一个春天。

2025 年 4 月，写于山东郯城乡下老家

[1] https://liyanrui.github.io/2025/awk-notes/awk-networking/index.html

目录

序 i.

第 1 章 套接字地址

1.1 前言 1. 1.2 存在先于本质 1. 1.3 套接字 1. 1.4 是否有些恐慌？ 2. 1.5 插曲：双重指针 4. 1.6 迷雾重重 5. 1.7 更安全，更稳健 7. 1.8 有些计算机，不单纯 10. 1.9 反函数 10. 1.10 一网打尽 11. 1.11 总结 12.

第 2 章 我在这里!

2.1 前言 13. 2.2 套接字 13. 2.3 连接 14. 2.4 发送数据 14. 2.5 多个套接字地址？ 15. 2.6 完整的程序 16. 2.7 总结 17.

第 3 章 我是三体人

3.1 前言 18. 3.2 我的套接字地址 18. 3.3 我的套接字 18. 3.4 监听状态 19. 3.5 客户端套接字 20. 3.6 谁寄来的信？ 21. 3.7 读信和回信 22. 3.8 完整的程序 22. 3.9 总结 25.

第 4 章 像上帝创造生物……

4.1 前言 26. 4.2 sim 项目 26. 4.3 示意标记 26. 4.4 字符串类的声明 27. 4.5 字符串类的实现 28. 4.6 防碰头设计 32. 4.7 测试 32. 4.8 总结 34.

第 5 章 封装

5.1 前言 35. 5.2 客户端 35. 5.3 服务端 37. 5.4 数据发送与接收 40. 5.5 安全检测 42. 5.6 重写 ywj 43. 5.7 重写 threebody 44. 5.8 总结 45.

第 6 章 两朵乌云

6.1 前言 46. 6.2 接受两个连接 46. 6.3 阻塞 49. 6.4 不阻塞会如何？ 50. 6.5 可以挽救的错误 52. 6.6 CPU 很忙 55. 6.7 总结 55.

第 7 章 I/O 多路复用

7.1 前言 56. 7.2 重温文件 56. 7.3 文件描述符集合 59. 7.4 阻塞式轮询 60. 7.5 文件可读监视器 61. 7.6 select 可堪大用？ 64. 7.7 总结 64.

第 8 章 长缨在手

8.1 前言 65. 8.2 客户端集 65. 8.3 接收数据 67. 8.4 发送数据 68. 8.5 大胆假设 68. 8.6 小心求证 69. 8.7 你上当了 73. 8.8 结语 74.

第 9 章　缚住苍龙

9.1 前言 75. 9.2 单向链表 75. 9.3 修改 SimServer 类 78. 9.4 修改 sim_server_run_once 80. 9.5 修改 sim_server_receive 82. 9.6 修改 sim_server_send 83. 9.7 threebody 并发 83. 9.8 总结 84.

第 10 章　世界是你的，也是我的

10.1 前言 85. 10.2 电子穿过双缝 85. 10.3 麦田守望者 87. 10.4 守望的艺术 90. 10.5 非阻塞等待 91. 10.6 信号处理 93. 10.7 更安全的信号处理 94. 10.8 坍缩 95. 10.9 总结 98.

第 11 章　弦动

11.1 前言 99. 11.2 叮咚 99. 11.3 历史 100. 11.4 开始混乱 102. 11.5 服务端并发 104. 11.6 互斥 106. 11.7 内耗 107. 11.8 超距作用 109. 11.9 总结 112.

第 12 章　天地一指

12.1 前言 113. 12.2 新的维度 113. 12.3 梳子 116. 12.4 牛顿 117. 12.5 返回值 120. 12.6 拉链 120. 12.7 可重入性 121. 12.8 总结 123.

第 13 章　返朴

13.1 前言 124. 13.2 简化 124. 13.3 无效的客户端 125. 13.4 公之于众 128. 13.5 新的 threebody 128. 13.6 宏 130. 13.7 协程 131. 13.8 叮咚或咚叮 132. 13.9 总结 135.

跋 136.

索引 137.

第 1 章　套接字地址

1.1 前言

可能没有任何一本网络编程方面的书籍会告诉你，网络编程真正重要的问题，从来都不是两台计算机上的进程如何传送信息，而是套接字地址如何表示。

倘若你对上述说法觉得奇怪甚至荒谬，那么请思考这样一个问题，一个普通的计算机用户，他要上网，在十多年前，他可能还需要知道 IP 地址、网关、DNS 等概念以及如何为操作系统设置它们，而在现在，他只需知道可用的 WIFI 的名字。

我们既然要编写网络程序，是以网络体系已经存在且能正常运行为前提的，故而无需关心其原理和内幕，亦即你无需知道 OSI 模型是什么，Internet 四层架构是什么，架构的每一层有哪些协议，在计算机操作系统中如何实现这些协议，以及 TCP 协议需要几次握手和几次挥手……你只需要知道什么是 IP 地址，什么是域名，便已足够。不过，要阅读本文及其后续一系列文章，你需要熟悉 C 语言，无需精通。

如果你在找与网络通信相关的工作，面试官可能会考问上述这些我觉得不重要的东西。这种情况下，你的确需要了解它们。不过，当你学会网络编程之后再去学习网络原理，也许更知其然。

1.2 存在先于本质

你刚开始学习 C 语言，有人告诉你需要先了解计算机组成原理，汇编语言编程，操作系统原理，编译原理，然后你才能写一个 Hello world 程序：

```c
/* hello-world.c */
#include <stdio.h>
int main(void)
{
    printf("Hello world!\n");
    return 0;
}
```

然后用 gcc 编译它：

```
$ gcc hello-world.c -o hello-world
```

这个人也许是善意的，真心想给你有用的建议，可是他很容易让你根本没有机会写出上述的 Hello world 程序。我看过的网络编程书籍，好像每本书的作者都是这个人。

1.3 套接字

网络编程，无论理论家们是如何阐述它的，但在现实中，它就是面向套接字的编程。

何为套接字？socket 的这个单词的本意是窝、臼、槽、插座、插口等，至于它的「套接字」这个翻译，我不喜欢，但是在已形成世俗的力量面前，我也没有力量推翻它。在网络编程中，套接字表示通信的端点，它拥有一个地址，由计算机的 IP 地址和端口号组合而成的地址，例如

```
192.168.0.10:8080
```

冒号之前的部分是 IP 地址，例如 192.168.0.10，之后的部分是端口号，例如 8080。计算机操作系统基于套接字，可为某个进程与另一台计算机上的某个进程建立数据传输通道。

何谓进程？进程就是活着的程序，亦即计算机中正在运行的程序。当问题复杂到很难通过一个程序予以解决，我们便不得不考虑通过两个甚至更多个程序协同解决。这些参与解决同一个复杂问题的多个程序，它们未必是在同一台计算机上运行，它们需要基于某种通信网络实现通信，基于信息交流实现协同工作。

我们需要建立一种直觉，即所谓网络通信，指的是某台计算机或两台不同的计算机上的两个进程之间的通信。这两个进程，皆需具备发送和接受信息的能力，否则无法完成能够实现二者协同工作的通信。这种通信方式，与我们往日书信交流有相似之处。我们每个人，都具备写信和收信的能力，不是吗？

在计算机网络中，常用的一种通信形式是，很多台计算机中的进程，主动向某台计算机中的进程发送信息，并接受后者回复的信息，我们称前者为客户端（client），后者为服务端（server）。在现实生活中，服务端像是一个明星级的人物，有许多喜欢他的人会给他写信，至于他是否能逐一回信，我就不清楚了。计算机网络中的服务端会尽最大努力做到逐一回信，甚至不惜为此事创建一个团队以提高回信速度。像这样的网络程序，通常称为 C/S 架构，亦即客户端/服务端架构。

进程之间基于套接字的通信，使得进程可以是异构的。例如，一个进程，可以是 C 语言写的，而另一个可以是 Java 语言写的，只要它们都遵守基于套接字通信这个约定，便能够实现通信。当存在不计其数的进程进行通信，便形成了互联网。

1.4 是否有些恐慌？

现今的操作系统，无论是 Linux 还是任何一种还活着的 Unix，以及我很少使用的 Windows，它们都以套接字为核心提供了一组 C 函数，通常将其统称为套接字 API（Application Program Interface，应用程序接口）。基于套接字 API，我们能能够自己的程序从操作系统中获得与其他进程通信的功能，亦即这些功能不必我们亲自实现，只需我们学会使用。

学习基于套接字 API 的网络编程，第一个需要学会的函数是 getaddrinfo，它可以将文字形式套接字地址转化为一个数字，我可以不厌其烦地称其为数字形式的套接字地址。

getaddrinfo 的形式如下：

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);
```

现在，你可能已经开始有些恐慌，如同此刻的我。请自信一些，上述代码给出了 getaddrinfo 的声明，以及使用它时所需的头文件。getaddrinfo 接受 4 个参数，若它运行成功，返回 0，否则返回一个非 0 的错误码。node 参数表示 IP 地址，service 表示端口，hints 可让我们对 getaddrinfo 的运行施加一些干扰，res 为 getaddrinfo 对文字形式的套接字地址的转化结果。

何谓 IP 地址？每一台连接在网络上的计算机都会有的一个地址，形式上通常是以英文句号隔开的 4 个数字，且每个数字的最大值是 255。例如 192.168.1.10 是我的计算机的 IP 地址。这种形式的地址，称为 IP v4 地址，即第 4 代 IP 地址。如果你不知道自己的 IP 地址，可以询问你的网络管理员，或者你的朋友，该如何使用像 ifconfig 这样的命令查出你计算机上的网卡目前正在使用的 IP 地址：

```
$ ifconfig
... ... ...
wlp3s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.10  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 2409:8a3c:1502:4bc0:b06d:459:cd45:f30f  ... ... ...
        ... ... ...
```

现代还有一种地址，叫 IP v6，即第 6 代 IP 地址，例如上述我的 ifconfig 命令输出的 inet6 后面的一串数字：

```
2409:8a3c:1502:4bc0:b06d:459:cd45:f30f
```

便是 IP v6 地址。现在使用 IP v6 地址的网络不太多，至于它的未来如何，对于学习网络编程而言，暂且无需关心。

何谓端口？一个普通的数字，通常可以随意指定，但尽量不要小于 1024，因为低于 1024 的端口有一些是被操作系统或其他明星级的网络服务占用的。

现在，我们已经可以很轻松地向 getaddrinfo 提供文字形式的套接字地址了。例如

```
int a = getaddrinfo("192.168.1.10", "8080", 第 3 个参数, 第 4 个参数);
```

接下来看 getaddrinfo 的第 3 个参数 hints，它是一个指针，指向类型为结构体 addrinfo 的对象。结构体 addrinfo 的定义如下：

```
struct addrinfo {
    int              ai_flags;
    int              ai_family;
    int              ai_socktype;
    int              ai_protocol;
    socklen_t        ai_addrlen;
    struct sockaddr *ai_addr;
    char            *ai_canonname;
    struct addrinfo *ai_next;
};
```

不要急于恐慌，通常情况下，我们真正需要关心的只有 ai_family 和 ai_socktype 这两个成员（或字段），其他成员皆可清零，例如

```
struct addrinfo hints;
memset(&hints, 0, sizeof(struct addrinfo));
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
```

memset 可将指定的内存空间清零，即以 0 值填充该空间。ai_family 用于设定 IP 地址是第 4 代（AF_INET）还是第 6 代（AF_INET6）。至于 ai_socktype，暂且理解为用于设定在网络通信时数据传输过程的可靠程度，SOCK_STREAM 表示相当可靠。还有一种不太可靠的设定，即 SOCK_DATAGRAM。这两种可靠程度分别对应着两种信息传输协议，SOCK_STREAM 对应 TCP 协议 IPPROTO_TCP，而 SOCK_DATAGRAM 对应 UDP 协议 IPPROTO_UDP，如同我们曾经用邮局寄信，前者相当于挂号信形式，后者相当于平信形式，平信较挂号信，容易出现信件丢失的情况。信息传输协议是选择 TCP 还是 UPD，可以通过 ai_protocol 设定，但是 ai_socktype 已经决定了所用的协议，故而无需设定。

hints 参数用于通告 getaddrinfo，192.168.1.10 是 IP v4 地址，数据传输是要用要用靠谱的 SOCK_STREAM 对应的传输协议，即 TCP，而不是不靠谱的传输协议 UDP。不过，UDP 也并非无用，其优势在于传输速度更快，适用于传输一些不太重要的数据，例如网络上聊天方面的信息。不过在学习网络编程的过程中，可以先忽略 UDP，我们不会遇到必须使用它的情况。待日后用到，现学即可，它比 TCP 容易多了。

现在，我们对 getaddrinfo 已经熟悉到了以下程度：

```
struct addrinfo hints;
memset(&hints, 0, sizeof(struct addrinfo));
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
int a = getaddrinfo("192.168.1.10", "8080", &hints, 第 4 个参数);
```

第 4 个参数 res 是一个双重指针：

```
struct addrinfo **restrict res
```

res 指向的对象是一个指针，后者指向与上述 hints 同类型的结构体对象。之所以是双重指针，是因为 getaddrinfo 要将 IP 地址和端口转换的结果存于一个链表，需要用一个指针记录该链表的首结点位置（内存地址），res 则指向这个指针。

1.5 插曲：双重指针

可能你对 C 语言的熟悉程度不足以理解上述关于双指针的讨论，不必为此不安，毕竟并没有任何人要求必须对 C 语言了如指掌方可学习网络编程。

假设在一个函数内部动态分配了一块内存，用于存储某个对象，该如何将该对象作为函数的返回值呢？例如，

```
int *foo(void) {
    int *a = malloc(sizeof(int));
    *a = 3;
    return a;
}
```

为了让 foo 函数更安全一些，要求它的返回值只能是正确或错误码，像 getaddrinfo 函数那样，返回 0 表示成功，返回非 0，表示失败，此时 foo 该如何将指针 a 的值返回呢？答案是让 foo 参数是一个双重指针：

```
int foo(int **p) {
    int *a = malloc(sizeof(int));
    if (a) {
        *a = 3;
        *p = a;
        return 0;
    } else return -1;
}
```

请记住这个原则：一个函数内，若需要通过参数将内部的某个变量的值传给函数外部，则参数的类型必须是该变量类型的指针。在上述的 foo 函数中，内部的变量 a，若将其值通过参数 p 传给函数外部，由于 a 的类型是 int *，那么 p 的类型便是 a 类型的指针，亦即 int **。

希望上述简单的 C 代码示例能够让你明白为什么 getaddrinfo 的第 4 个参数是双重指针，并且你也学会了如何在函数的参数中通过灵活地使用指针取出函数内部的数据。

1.6 迷雾重重

现在我们已经能够让 getaddrinfo 运行起来了，即

```
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(struct addrinfo));
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
int a = getaddrinfo("192.168.1.10", "8080", &hints, &res);
```

可是 getaddrinfo 将文字形式的 192.168.1.10:8080 转化的数字藏于 res 所指向的结构体中的何处呢？答案是，在结构体 addrinfo 的 ai_addr 成员所指向的结构体对象之中。ai_addr 的类型是 struct sockaddr *，而结构体 sockaddr 的定义是

```
struct sockaddr {
    sa_family_t     sa_family;      /* Address family */
    char            sa_data[];      /* Socket address */
};
```

即使不知道 sa_family_t 是无符号的整型类型，应该也能看出，sockaddr 并无套接字地址信息，原因是 sockaddr 只是一个通用的类型，它几乎无意义，但其指针形式能指向真正有用的结构体 sockaddr_in：

```
struct sockaddr_in {
    sa_family_t     sin_family;     /* AF_INET */
    in_port_t       sin_port;       /* Port number */
    struct in_addr  sin_addr;       /* IP address */
};
```

sockaddr_in 描述的是文字形式的套接字地址转化为数字的结果，sin_port 是无符号的 16 位整型类型，表示端口，IP 地址在 sin_addr 中。sin 不是正弦的意思，其中的 s 表示 socket，in 表示 Internet……真是鬼画符一样的缩写。sin_addr 类型又是一个结构体，但这个结构体跟没有差不多：

```
struct in_addr {
    in_addr_t s_addr;
};
```

s_addr 即文本形式的 IP 地址转换而成的数字，一个 32 位的无符号整型数。

sockaddr_in 存储的套接字地址中的 IP 地址是 IP v4。套接字 API 为 IP v6 准备的套接字地址数据结构是 sockaddr_in6 结构体类型。

struct sockaddr * 类型的指针可以安全地指向任何一个 struct sockaddr_in 类型的对象（IP v4 地址），也可以指向任何一个 struct sockaddr_in6 类型的对象（IP v6 地址）。之所以需要这么多结构体，只有一个原因，套接字 API 为了兼容 IP v4 和 IP v6 地址在设计上尽了自己最大的努力。

也许你早已看出，实际上套接字 API 无需如此努力，使用 void * 类型的指针便可指向任何类型的对象。你的看法是正确的，问题在于套接字 API 诞生时，C 语言还没有 void * 指针。后来，为了与历史兼容，它只能保持旧时方式。

现在我们基本上已经知道如何查看 getaddrinfo 生成的数字形式的 IP 地址和端口了，即

```
struct sockaddr_in *my_addr = (struct sockaddr_in *)(res->ai_addr);
printf("%u:%hu\n", my_addr->sin_addr.s_addr, my_addr->sin_port);
```

试着将程序写完整：

```
/* 源文件 foo.c */
#include <stdio.h>
#include <string.h> /* memset 需要它 */
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```c
int main(void) {
    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    int a = getaddrinfo("192.168.1.10", "8080", &hints, &res);
    struct sockaddr_in *my_addr = (void *)res->ai_addr;
    printf("%u:%hu\n", my_addr->sin_addr.s_addr, my_addr->sin_port);
    return 0;
}
```

使用 gcc 编译 foo.c，并运行所得程序：

```
$ gcc foo.c -o foo
$ ./foo
167880896:36895
```

IP 地址 192.168.1.10 被转换为 167880896，端口 8080 被转换为 36895。或许你会觉得，一路辗转，所得不过如此，前途依然渺茫。且莫着急，毕竟我们已经得到了类似邮政编码的东西了，距离将信件发送出去，已近在咫尺。

1.7 更安全，更稳健

关于 getaddrinfo，还有一些事情需要探索，例如直到现在，我们似乎未曾真正关心过它的返回值，亦即上述代码中的 int a。前面说过，a 的值若为 0，则表示 getaddrinfo 运行正确，否则运行失败。如何处理失败的情况呢？基本的做法是，打印错误信息，让程序退出：

```c
if (a != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
    exit(-1); /* 让程序以非 0 值退出 */
}
```

套接字 API 有 gai_strerror 函数，它可将 getaddrinfo 返回的错误码「翻译」为人类可理解的文本信息。

还有一个问题，getaddrinfo 用于保存数字化的套接字地址信息，即上述代码 res 所指向的结构体对象，其类型为 struct addrinfo，它是在 getaddrinfo 内部创建的，亦即它占用了一段动态分配的内存空间，在不再使用它时，需要进行释放，以防程序出现内存泄漏。本文的开始伴随 getaddrinfo 的声明一同出现的另一个函数 freeaddrinfo，便是用于释放动态分配的类型为 struct addrinfo 的对象。

更为安全的 foo.c 应该像下面这样写：

```c
/* 源文件 foo.c */
#include <stdio.h>
```

```
#include <string.h>  /* memset 需要它 */
#include <stdlib.h>  /* exit 需要它 */
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```
int main(void) {
    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    int a = getaddrinfo("192.168.1.10", "8080", &hints, &res);
    if (a != 0) {
        fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
        exit(-1); /* 让程序以非 0 值退出 */
    }
```

```
    struct sockaddr_in *my_addr = (void *)res->ai_addr;
    printf("%u:%hu\n", my_addr->sin_addr.s_addr, my_addr->sin_port);
    freeaddrinfo(res);

    return 0;
}
```

还有一个问题，前文提到过，getaddrinfo 将得到的数字化的 IP 地址和端口存于一个链表中，上述代码中的 res 指向的是该链表的首结点。也许对此你会觉得奇怪，一个文字形式的套接字地址，怎么可能会被转换成多个数字化的套接字地址呢？答案是，getaddrinfo 所做的工作远比我们想象得更多，它不仅可将文字形式的套接字地址转换为数字形式，也可以将域名或主机名与端口号构成的地址转化为数字形式的套接字地址。

域名或主机名，可以指代一台拥有多个 IP 地址的计算机。这并不奇怪，毕竟一台计算机上可以装配多张网卡，而每张网卡都可以配置一个 IP 地址。为稳妥起见，需要在遍历 getaddrinfo 所生成的地址列表，以获得所有可能的数字化的套接字地址：

```
for (struct addrinfo *it = res; it; it = it->ai_next) {
    struct sockaddr_in *my_addr = (void *)it->ai_addr;
    printf("%u:%hu\n", my_addr->sin_addr.s_addr, my_addr->sin_port);
}
```

现在，对上述的 foo.c 略作修改，除了增加套接字地址列表的遍历访问过程，将 getaddrinfo 所需的文字形式的 IP 地址和端口由硬性设定改为命令行参数传入：

```c
/* 源文件 foo.c */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```c
int main(int argc, char **argv) {
    struct addrinfo hints, *res;
    if (argc != 3) {
        fprintf(stderr, "usage: %s <host> <port>\n", argv[0]);
        exit(-1);
    }
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
```

```c
    int a = getaddrinfo(argv[1], argv[2], &hints, &res);
    if (a != 0) {
        fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
        exit(-1); /* 让程序以非 0 值退出 */
    }
    for (struct addrinfo *it = res; it; it = it->ai_next) {
        struct sockaddr_in *my_addr = (void *)it->ai_addr;
        printf("%u:%hu\n", my_addr->sin_addr.s_addr, my_addr->sin_port);
    }
    freeaddrinfo(res);
    return 0;
}
```

重新编译 foo.c，然后按以下方式执行 foo 程序：

```
$ ./foo 192.168.1.10 8080
167880896:36895
$ ./foo localhost 8080
16777343:36895
$ ./foo 127.0.0.1 8080
16777343:36895
$ ./foo 0.0.0.0 8080
0:36895
```

localhost 和 127.0.0.1 的数字转换结果相同，都是 16777343。localhost 表示本地计算机，即运行 foo 程序的这台计算机，而 127.0.0.1 表示网络回环地址，任何一台计算机，即使它没有网卡，也拥有这个地址。可将 localhost 和 127.0.0.1 理解成「自我」。一个人，可以不与整个社会发生任何关系，但是只要他还活着，必定会有「自我」。

1.8 有些计算机，不单纯

试着用 foo 程序访问域名 www.baidu.com 的 IP 地址：

```
$ ./foo www.baidu.com 80
776379431:20480
4014382119:20480
```

可以发现，www.baidu.com 指向的计算机，它拥有两个 IP 地址。事实上，许多知名的网页服务器（端口默认为 80），都有多个 IP。这便是 getaddrinfo 生成的是一个地址列表，而不是单个地址的原因。也许你会觉得有些不可思议，getaddrinfo 是如何获得域名为 www.baidu.com 的计算机的 IP 地址的呢？答案是，getaddrinfo 会调用操作系统提供的域名查询服务，在 DNS 服务器上查找 www.baidu.com 的 IP 地址集。

用主机名或域名指代一台网络上的计算机，是因为文字形式的 IP 地址并不便于人类记忆。普通人，勉强能记住几个 IP v4 地址，但是要记住一个 IP v6 可能就很难了。主机名和域名的主要区别是，前者无人管理，只能在局域网中使用，而后者会有网络上专门的计算机系统予以管理并提供查询服务，故而可用于指代某台远程的计算机。

上述网络基础知识，现在可以知道，也可以不知道，getaddrinfo 已经将所有细节封装了。总之，我们已经可以轻易做到将文字形式的套接字地址转化为数字形式了，接下来便是如何利用后者，将信息发送出去。现在，倘若你使用的是 Linux 系统，不妨使用以下命令，再回顾一下 getaddrinfo 的一切：

```
$ man 3 getaddrinfo
```

1.9 反函数

下面额外介绍一个与 getaddrinfo 相反的函数 getnameinfo 的用法。getnameinfo 可将结构体 addrinfo 中存储的数字化的套接字地址转换为文本形式。对于我们而言，文本形式的套接字地址是已知的，故而 getnameinfo 似乎没什么用处。不过，对于上述 foo 程序输出的 www.baidu.com 的 IP 地址，也许你更希望它是文本形式，所以 getnameinfo 还是有用的。

getnameinfo 的声明如下：

```
#include <sys/socket.h>
#include <netdb.h>
int getnameinfo(const struct sockaddr *addr, socklen_t addrlen,
                char *host, socklen_t hostlen,
                char *serv, socklen_t servlen, int flags);
```

你可能又开始恐慌了……我也是。getnameinfo 接受 socketaddr 结构体，然后将 IP 地址转换为文本存储于长度为 hostlen 的缓冲区 host，并将端口转换为文本存于长度为 servlen 的缓冲区 serv，该数组的长度是 servlen。最后一个参数 flags 用于组合一些标识，让 getnameinfo 输出我们所需要的结果，例如如果希望输出的是文本形式的 IP 地址和端口，而不是主机名或域名以及服务名（服务名是为端口取的便于人类记住的名字，例如端口 80，对应的服务名是 http），只需将 flags 的值设定为

```
NI_NUMERICHOST | NI_NUMERICSERV
```

下面是 getnameinfo 的用法示例：

```
/* 输出 IP 地址和端口 */
char host[NI_MAXHOST], port[NI_MAXSERV];
for (struct addrinfo *it = res; it; it = it->ai_next) {
```

```
    getnameinfo(it->ai_addr, it->ai_addlen,
                host, sizeof(host),
                port, sizeof(port),
                NI_NUMERICHOST | NI_NUMERICSERV);
```

```
    printf("%s:%s\n", host, port);
}
```

可使用这段代码替换前文最后一个版本的 foo.c 中遍历 getaddrinfo 生成的套接字地址列表的那部分代码。

若你想更加了解 getnameinfo，可以

```
$ man 3 getnameinfo
```

1.10 一网打尽

上文不断修改的 foo.c，所实现的皆为文字形式的 IP v4 地址及端口构成的套接字地址的数字化转换，虽然提及了 IP v6 地址，但是并未给出相关代码将其转换为数字。实际上，非常简单，只需将上文最后实现的 foo.c 中一处代码

```
hints.ai_family = AF_INET;
```

修改为

```
hints.ai_family = AF_INET6;
```

便可实现将文字形式的 IP v6 地址和端口构成的套接字地址转换为数字化套接字地址。

事实上，getaddrinfo 走得更远，胸怀更为宽广，它能够同时支持 IP v4 和 IP v6，只需将 hints.ai_family 的值设成 AF_UNSPEC。完成此修改，重新编译 foo.c，执行新生成的 foo 程序，获取 www.baidu.com 对应的套接字地址：

```
$ ./foo www.baidu.com 80
2409:8c00:6c21:118b:0:ff:b0e8:f003:80
2409:8c00:6c21:11eb:0:ff:b0bf:59ca:80
39.156.70.239:80
39.156.70.46:80
```

1.11 总结

不知不觉，我们已经基本学会了 4 个套接字 API 函数了，并不难，我甚至有些过度讲解了。你应该比我聪明，原本便可借助 man 命令便可获得这些函数最为权威的讲解，然后迅速学会它们，免受我的啰嗦。不过，man 命令最大的问题是，它无法告诉你，学习网络编程居然可以直接从学习 getaddrinfo 开始。

第 2 章　我在这里!

2.1 前言

叶文洁将地球在宇宙中的坐标暴露给三体人时，她并不知道这个信息要多久能抵达，而且她对三体人几乎一无所知，她只是知道那个遥远的星系有着一个比地球更强大的文明。

我们试着模仿叶文洁，写一个向外太空暴露地球坐标的网络程序。等到有一天我们对人类彻底失去希望的时候，就运行这个程序。

假设网络上有一台计算机，它的域名是 www.threebody.com。这台计算机中运行的某个网络服务端所用的端口是 8080，于是该服务端的套接字地址便是 www.threebody.com:8080。我们已经学会了 getaddrinfo 函数，很容易将这个文字化套接字地址转化为数字化的套接字地址：

```
struct addrinfo hints, *res;
memset(&hints, 0, sizeof(struct addrinfo));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
int a = getaddrinfo("www.threebody.com", "8080", &hints, &res);
if (a != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
    exit(-1);
}
```

对于叶文洁而言，她所用的向服务端发送消息的进程即客户端。客户端并不需要知道域名为 www.threebody.com 的计算机位于何处，它只需向地址为 www.threebody.com:8080 的套接字发送信息即可。

再次强调，服务端和客户端，都是进程。进程，就是计算机中正在运行的程序。

2.2 套接字

在 Unix 或 Linux 系统或者任一合乎 POSIX 标准的操作系统中，套接字实际上是文件描述符，指代专用于网络通信的文件。为了行文简便，我们只需认为套接字是一种特殊的文件。以后我们会经常说，从套接字读取数据，向套接字写入数据。你只需将这些描述理解为，从套接字指代的文件读取数据，向套接字指代的文件写入数据。

创建套接字的函数，是 socket，其声明如下：

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

在 getaddrinfo 生成的结构体 addrinfo 列表中，每一个结点都含有 socket 函数所需要的参数值。以 addrinfo 列表首结点为例，socket 函数用法如下：

```
int server = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
```

若 socket 函数运行成功，则返回文件描述符，否则返回 -1。不过，socket 函数生成的套接字，尚无地址，不能用于通信。

2.3 连接

connect 函数可将服务端的套接字地址赋予套接字，其形式如下：

```
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

毫无意外，connect 所需要的参数值，我们也已经有了：

```
int a = connect(server, res->ai_addr, res->ai_addrlen);
```

connect 运行成功，会返回 0，否则会返回 -1。connect 函数第一个参数是用于表示服务端的套接字，后两个参数分别是套接字地址和地址长度（字节数），亦即该函数可将服务端套接字地址赋予一个尚无地址的套接字。同时，connect 会悄悄地构造一个对于我们而言不可见的套接字，将本机的 IP 地址和一个随机的端口号作为套接字地址赋予它。因此，connect 创建的实际上是一对套接字，对于上例而言，这对套接字可用文字形式的地址表示为：

```
(www.threebody.com:8080, localhost:随机端口)
```

前者为服务端套接字地址，后者则是客户端套接字地址——对于本文示例而言，即叶文洁用于向三体人发送信息的进程的套接字地址。这对套接字构成了一个网络连接。基于该连接，客户端便可向服务端发送和接收数据。

2.4 发送数据

send 函数可发送数据，其形式如下：

```
#include <sys/socket.h>
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

send 函数可将长度为 len 的缓冲区 buf 中的数据按照 flags 规定的方式写入套接字 sockfd。大多数时候只需要将 flags 设为 0，表示采用普通的数据写入方式。若 send 函数运行成功，返回值是已写入数据的字节数，否则为 -1 表示数据写入失败。

send 函数的用法如下：

```
char *earth_coordinate = "I am here!";
size_t n = strlen(earth_coordinate);
ssize_t b = send(server, earth_coordinate, n, 0);
if (b < 0) {
    fprintf(stderr, "send error!\n");
    exit(-1);
}
```

2.5 多个套接字地址？

在此需要再次强调，getaddrinfo 函数构造的套接字地址可能是多个，若已经忘记了，需要回顾网络编程的核心技术，即第 1 章。我们需要从这些地址中选出一个可用的。

何谓「可用的套接字地址」？答案是，能够让 connect 函数成功运行的地址。以下代码便是可用地址的一个简单的选择过程：

```
int server = -1;
for (struct addrinfo *it = res; it; it = it->ai_next) {
    server = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    if (server == -1) continue;
    int a = connect(server, it->ai_addr, it->ai_addrlen);
    if (a == -1) {
        close(server); /* 关闭套接字 */
        continue;
    }
    break;
}
if (server == -1) {
    fprintf(stderr, "failed to connect!\n");
    exit(-1);
}
```

注意，在上述代码的 for 循环中，每一次循环，都会构造套接字 server，若 connect 失败，便会 close(server)。之前从未提及 close 函数，是因为它并非套接字 API 中的函数，而是 C 标准库中通过文件描述符关闭文件的函数。之所以要关闭文件，是因为 socket 函数中已经打开了套接字，而套接字指代文件。

若上述代码所得的 server 的值并非 -1，则意味着有了可用的服务端套接字以及客户端套接字，这对套接字构成了一个可用的通信连接，亦即客户端套接字与服务端套接字处于连通状态，然后便可向服务端套接字写入数据：

```
char *earth_coordinate = "I am here!";
size_t n = strlen(earth_coordinate);
ssize_t b = send(server, earth_coordinate, n, 0);
if (b < 0) {
    fprintf(stderr, "send error!\n");
    exit(-1);
}
```

2.6 完整的程序

```c
/* ywj.c */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>  /* close 函数需要它 */
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```c
int main(void) {
    /* 服务端套接字列表 */
    struct addrinfo hints, *addr_list;
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    int a = getaddrinfo("www.threebody.com", "8080", &hints, &addr_list);
    if (a != 0) {
        fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
        exit(-1);
    }
```

```c
    /* 选择可用的连接 */
    int server = -1;
    for (struct addrinfo *it = addr_list; it; it = it->ai_next) {
        server = socket(addr_list->ai_family,
                        addr_list->ai_socktype,
                        addr_list->ai_protocol);
        if (server == -1) continue;
        int a = connect(server, it->ai_addr, it->ai_addrlen);
        if (a == -1) {
            close(server); /* 关闭 socket 函数打开的文件 */
            continue;
        }
        break;
    }
    freeaddrinfo(addr_list);
```

```
    if (server == -1) {
        fprintf(stderr, "failed to connect!\n");
        exit(-1);
    }
```

```
    /* 向服务端发送数据 */
    char *earth_coordinate = "I am here!";
    size_t n = strlen(earth_coordinate);
    ssize_t b = send(server, earth_coordinate, n, 0);
    if (b == -1) {
        fprintf(stderr, "send error!\n");
        exit(-1);
    }
    close(server);
    return 0;
}
```

编译上述源码：

```
$ gcc ywj.c -o ywj
```

现在，我们对这个世界假装着失望了，运行 ywj 程序：

```
$ ./ywj
```

可惜的是，我们企图毁灭地球的信息无法发送出去，大概几分钟后，ywj 程序以

```
send error!
```

而告终。原因是，www.threebody.com 对应的计算机并不存在。幸好它不存在。

2.7 总结

务必清楚，作为客户端，要向服务端发送数据，需要在明面上创建表达服务端的套接字，同时也需要暗自创建表达客户端自身的套接字，如此方能形成可用的通信连接。这个过程完整的套接字 API 管线是：

```
getaddrinfo -> socket -> connect -> send
```

其中，`socket` 和 `connect` 所需要的参数，`getaddrinfo` 皆已备好。现在，你应该大概明白，为何在第 **1** 章中说，网络编程真正重要的问题，从来都不是两台计算机上的进程如何传送信息，而是套接字地址如何表示。

第 3 章　我是三体人

3.1 前言

在第 2 章中，程序 ywj 向 www.threebody.com:8080 发送信息，该过程因目标计算机不存在而以失败告终。

不存在的，可以创建，然后欺骗 ywj，让她以为它存在。若是在 Linux 系统中，可使用超级用户权限编辑 /etc/hosts 文件，为其添加以下内容

```
127.0.0.1        www.threebody.com
```

也许你已识破我的诡计了，我在将这个域名与我的计算机的网络回环地址绑定，然后我只需在我的计算机里编写一个程序，用它接收 ywj 发来的信息并予以回复，便可完成欺骗。这一瞒天过海之计能否奏效，我心里也没底，试试看。

现在，我是计算机里的三体人。我是服务端。我要接收来自 ywj 的数据，也要回复她。

3.2 我的套接字地址

ywj 给我发送信息，她需要知道我的套接字地址——文字形式的，即我所在的计算机的域名（`www.threebody.com`）和端口（`8080`）。这个套接字地址，对我而言，便是 `127.0.0.1:8080`，我也需将其转化为数字形式：

```
struct addrinfo hints, *addr_list;
memset(&hints, 0, sizeof(struct addrinfo));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
int a = getaddrinfo("127.0.0.1", "8080", &hints, &addr_list);
if (a != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
    exit(-1);
}
for (struct addrinfo *it = addr_list; it; it = it->ai_next) {
    /* ... 想必你已很熟悉地址列表遍历过程了... */
}
```

转换的结果，即我的套接字地址。

3.3 我的套接字

我需先为自己创建一个无地址的套接字：

```
int server = -1;
for (struct addrinfo *it = addr_list; it; it = it->ai_next) {
    server = socket(it->ai_family, it->ai_socktype, it->ai_protocol);
    if (server == -1) continue;
    break;
}
```

既然 ywj 认为我是服务端，那我就用 server 命名我的套接字，只是在为这个套接字赋予地址时，我不能像 ywj 那样用 connect，因为我是服务端，我无需主动连接任何人。

我需要使用 bind 为我的套接字赋予地址。bind 函数的形式如下：

```
#include <sys/socket.h>
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

getaddrinfo 已经为我们调用 bind 准备好了所有参数。bind函数的用法如下：

```
int server = -1;
for (struct *addrinfo it = addr_list; it; it = it->ai_next) {
    server = socket(it->ai_family, it->ai_socktype, it->ai_protocol);
    if (server == -1) continue;
    int a = bind(server, res->ai_addr, res->ai_addrlen);  /* <== 看此处！ */
    if (a == -1) {
        close(server);
        continue;
    }
    break;
}
 /* getaddrinfo 获得的地址列表完成使命 */
freeaddrinfo(addr_list);
```

```
/* 检测所得套接字是否有效 */
if (server == -1) {
    fprintf(stderr, "failed to bind!\n");
    exit(-1);
}
```

同之前遇到过的那些套接字 API 函数类似，bind 运行成功，返回 0，否则返回 -1。

3.4 监听状态

我若想接收到来自客户端的数据，前提是我的套接字需要处于监听状态，listen 函数可为其设定该状态。

listen 的形式如下：

```
#include <sys/socket.h>
int listen(int sockfd, int backlog);
```

若要理解 listen 函数的作用，首先需要假设你是一个大人物。你可能每天都能收到大量信件，而你根本没时间操心收信这样的琐事。假设你专门成立了一个部门负责接收信件，并且这个部门可能只有 1 个人处理这一事务，我们可称其为收信员。在每一天里，他接收的信件必定有一个数量限制，这个数量限制即 backlog。

收信员将接收的信件整齐地叠放成一摞，这摞信件在数量上最多有 backlog，先收到的信总是放在后收到的信的上面，不妨将这摞信件称为 A 摞。你要读信，每次只需拿 A 摞最上面的那封。同时，若有新的信件到来，收信员会将其塞入 A 摞最底部。

当 A 摞信件数量达到 backlog 时，若还有新的信件陆续到来，收信员就会另起一摞，姑且称之为 B 摞。同 A 摞，B 摞也总是先到的信件在上面，后到的信件在下面。当收信员一旦发现 A 摞信件的数量减少了，即不满 backlog 时，便从 B 摞的上面取走一些，塞到 A 摞下面，以补 A 摞所缺。同样，B 摞也有一个最大信件数量限制，只是我们不太清楚，但收信员知道。当 B 摞的信件数量超出限制时，他便拒绝接收新的信件了。

这个收信员，并非 listen 函数，后者只是用于通知他 A 摞的信件数量限制，他实际上是操作系统中的某个底层网络子系统，他是套接字 API 层面幕后的无名英雄。

与 listen 函数的内涵相比，其调用则非常简单：

```
int b = listen(server, 10); /* 设上述 A 摞新建最大数量为 10 */
```

若 listen 成功运行，上述它的返回值 b 为 0，否则为 -1。

3.5 客户端套接字

经过 listen 函数标记的套接字，需要将它传递于 accept 函数，由后者构造一个可用的通信连接。还记得连接的概念吗？一对套接字。

accept 函数的声明如下：

```
#include <sys/socket.h>
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

它所需要的所有参数，我现在都有：

```
int client = accept(server, NULL, NULL);
```

若 accpet 成功运行，它的返回值是操作系统悄悄地为我构造的一个套接字，它表示客户端。当 ywj 向我请求连接，那么当我接收了她的请求，操作系统便为我构造一个客户端套接字，通过这个套接字，我可以接收 ywj 发来的数据，也可以给她一些回复。若 accept 运行失败，会返回 -1。

注意，accept 的第 2 个和第 3 个参数，用于存储 accept 从来信排队序列中获得的来信地址以及该地址的长度——根据地址长度可判断该地址是 IP v4 还是 IP v6 地址），倘若不关心它，可将这两个参数皆设为 NULL。

ywj 持有两个套接字，一个表示服务端，一个表示她自身。现在，我也有两个套接字，一个表示我自身，另一个表示客户端。如果 ywj 的服务端是我，而我的客户端是 ywj，便意味我与她建立了连接。我们双方各自持有的两个套接字，本质上都是我们机器上的两份文件。ywj 向她的服务端套接字指代的文件写数据，网络系统会将这个数据复制到我的客户端套接字指代的文件里。

网络系统是什么？它是由不计其数的的计算机在遵守某种信息传输协议的前提下以有线或无线的方式连接起来的系统，客户端和服务端不过是这个系统的末端而已。在数字通信技术诞生之前，我们使用的网络系统就是邮政局，由不计其数的邮政工作人员和邮递员在遵守某种商业契约的前提下构成的系统。

3.6 谁寄来的信？

上一节未深究 accept 的第 2 个和第 3 个参数的用途。倘若我想弄清楚我收到的信件从何处寄来，亦即客户端的套接字地址，就需要在这两个参数上做文章。

accetp 函数的第 2 个参数 addr，其类型为 struct sockaddr *，是网络地址指针。第 1 章 1.6 节介绍了 sockaddr 结构体以及 sockaddr_in（IP v4 地址）和 sockaddr_in6（IP v6 地址）结构体，特别是可使用 sockaddr 类型的指针指向后两者的对象，但是在类型兼容方面，sockaddr 无法兼容 sockaddr_in 和 sockaddr_in6，原因是 sockaddr 指针是在 C 语言还没有 void 指针的时代发明出来的替代品。void 指针能够指向任何类型的实例（对象），但是却无法用 void 类型容纳任何数据。

结构体 sockaddr_storage 类型，不仅其指针形式可以指向 sockaddr_in 或 sockaddr_in6 的对象，其数据结构也能兼容它们，故而为了让 accept 函数能够不关心 sockaddr_in 和 sockaddr_in6 的区别，可使用 sockaddr_storage 类型的指针作为 accetp 第 2 个参数的类型。例如

```
struct sockaddr_storage addr;
socklen_t addr_len = sizeof(addr);  /* 务必将其初始化为 addr 的大小 */
int client = accept(server, (struct sockaddr *)&addr, &addr_len);
```

注意，accept 的第 3 个参数 addr_len 必须初始化为第 2 个参数所指向的结构体对象占用的内存大小（字节数）。当 accept 返回后，第 2 个参数所指向的结构体对象实际占用的内存大小会保存在第 3 个参数中。

若只是想查看客户端套接字地址的文字形式，可以直接将 accept 获取的网络地址传递给 getinfoname，由后者转化成文字形式的套接字地址：

```
char host[NI_MAXHOST], port[NI_MAXSERV];
getnameinfo((struct sockaddr *)&addr, addrlen,
            host, sizeof(host),
            port, sizeof(port),
            NI_NUMERICHOST | NI_NUMERICSERV);
```

host 存储客户端的 IP 地址，port 存储其端口。

3.7 读信和回信

使用 recv 函数可以从上一节构造的套接字 client 中读取 ywj 给我发来的数据：

```
const size_t buffer_size = 1024;
char buffer[buffer_size];
ssize_t n = recv(client, buffer, buffer_size - 1, 0);
if (n == -1) {
    fprintf(stderr, "recv error!\n");
    exit(-1);
} else if (n == 0) {
    printf("Connection is closed!\n");
} else {
    buffer[n] = '\0';
    printf("Received %zd bytes: \n%s\n", n, buffer);
}
```

recv 用法与 ywj 使用的 send 相似，只需在内存中准备一个确定长度的空间，用于存放受到的信息。若 recv 的返回值非 -1 且非 0，则意味着已从 client 中获得了 n 个字节的信息，但是若将 buffer 中的内容作为字符串打印出来，需要在其末尾增加字符串结束符 \0，否则 printf 不知道字符串的截止位置，从而造成内存越界访问。

若我也需要向客户端 ywj 发送信息，只需使用 send 函数向 client 写信息即可，例如

```
char *other_buffer = "Hi, I am a threebody human!";
ssize_t m = send(client, other_buffer, strlen(other_buffer), 0);
```

套接字用完后，需要使用 close 函数予以关闭：

```
close(client);
close(server);
```

原因是，在 Unix 或 Linux 系统中，套接字是文件描述符，而文件描述符是系统资源。凡是使用了系统资源，都需要归还，如同使用 malloc 分配一块内存，用完后，需要使用 free 归还给系统。

3.8 完整的程序

将上文所述的各个步骤融合为一个可以与 ywj 作一次应答的程序：

```
/* threebody.c */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```c
int main(void) {
    struct addrinfo hints, *addr_list;
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    int a = getaddrinfo("localhost", "8080", &hints, &addr_list);
    if (a != 0) {
        fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
        exit(-1);
    }
```

```c
    /* 从 server 到 client 构造可用的连接 */
    int server = -1;
    for (struct addrinfo *it = addr_list; it; it = it->ai_next) {
        server = socket(it->ai_family, it->ai_socktype, it->ai_protocol);
        if (server == -1) continue;
        int a = bind(server, it->ai_addr, it->ai_addrlen);
        if (a == -1) {
            close(server);
            continue;
        }
        break;
    }
    freeaddrinfo(addr_list);
```

```c
    if (server == -1) {
        fprintf(stderr, "failed to bind!\n");
        exit(-1);
    }
    if (listen(server, 10) == -1) {
        fprintf(stderr, "failed to listen!\n");
        exit(-1);
    }
```

```c
    struct sockaddr_storage addr;
    socklen_t addr_len = sizeof(addr);
    int client = accept(server, (struct sockaddr *)&addr, &addr_len);
    if (client == -1) {
        fprintf(stderr, "failed to accept!\n");
        exit(-1);
    }
```

```c
    /* 获取客户端套接字地址 */
    char host[NI_MAXHOST], port[NI_MAXSERV];
    getnameinfo((struct sockaddr *)&addr, addr_len,
            host, sizeof(host),
            port, sizeof(port),
            NI_NUMERICHOST | NI_NUMERICSERV);
```

```c
    /* 接收信息 */
    const size_t buffer_size = 1024;
    char buffer[buffer_size];
    ssize_t n = recv(client, buffer, buffer_size - 1, 0);
    if (n == -1) {
        fprintf(stderr, "recv error!\n");
        exit(-1);
    } else if (n == 0) printf("Connection is closed!\n");
    else {
        buffer[n] = '\0';
        printf("Received %zd bytes: \n%s from %s:%s\n", n, buffer, host, port);
    }
```

```c
    /* 发送信息 */
    char *other_buffer = "Hi, I am a threebody human!";
    ssize_t m = send(client, other_buffer, strlen(other_buffer), 0);
    /* 关闭套接字 */
    close(client);
    close(server);
    return 0;
}
```

编译上述源码并执行：

```
$ gcc threebody.c -o threebody
$ ./threebody
```

在 threebody 进程所在的同一台计算机上运行 ywj 程序，threebody 进程会打印以下内容

```
Received 10 bytes:
I am here! from 127.0.0.1:38066
```

随后退出。threebody 打印的信息，正是 ywj 进程发送的，但是 threebody 发送给 ywj 的信息，后者收不到，因为后者尚未实现接收回信的功能。

3.9 总结

在 Unix 或 Linux 系统中，套接字实际上是文件。基于它实现的网络通信，本质是文件的读写操作。ywj 和 threebody 都是在各自的计算机中创建了两份文件，其中一份用于建立连接，另一份用于收发数据。

ywj 使用 `connect` 为服务端套接字赋予地址的过程中，操作系统暗自为她构造的套接字，其用途是向 threebody 发送连接请求，它与 threebody 构造的用于监听的套接字遥相呼应。要实现网络通信，ywj 只需读写她构造的服务端套接字，同时 threebody 也只需读写它使用 `accept` 构造的客户端套接字。

正在运行的网络系统能够帮助 ywj 和 threebody 将这些套接字连接起来，从而将网络通信变成了简单的的文件读写，这就是网络编程的本质。

第 4 章 像上帝创造生物……

4.1 前言

现在，你已经实现了一个简单的 C/S 架构的程序，这个程序分为两个部分，一个是 ywj，它是客户端（Client），另一个是 threebody，它是服务端（Server）。这两个部分通过套接字合为一体。不妨再勇敢一些，因特网（Internet）也不过是使用了不计其数的套接字将各个部分连接起来的一个庞大的慢吞吞的程序罢了，就像很多小块相互咬合形成的巨大的拼图。

到目前为止，一台计算机中任何一个进程所能想象的最大空间是因特网，它是进程的宇宙。这个宇宙并没有所谓的真空，其中的任何一台计算机，不是忙于计算，便是忙于收发数据，即使它们有时处于空闲状态，但这种状态也颇像量子力学所认为的，真空非空，充满了量子涨落。

不过，还是先不要沉湎于这些无尽的遐想，我们需要认真看一看 ywj.c 和 threebody.c 中的 `main` 函数，它们的代码已失去了简明性，像是此刻你桌面上摆放的那些用于连接计算机的电源线以及乱七八糟的信号线和数据线。我们需要尝试用面向对象的办法收纳这些代码。

C 语言在语法上不能像 C++、C#、Java 等语言那样优雅地支持面向对象编程，但是若将面向对象编程视为一种编程范式，C 在一定程度上可以实现其基本思想。我们可以用结构体模拟类，用普通函数模拟对象的方法。

4.2 sim 项目

从此刻开始，我们不仅仅是学习一些套接字 API 函数的用法，而是围绕它们，学习如何使用 C 语言模拟面向对象编程的基本理念，开发一个容易被他人使用的库。这是一个很小的项目。项目的名称或代号是 simple，质朴，简单。

由于 C 语言没有命名空间，程序中定义的所有类型和函数的名字，都在一个共同的空间里，很容易易出现重复，导致程序无法通过编译，或通过编译后，运行时出现异常。一般情况下，在一个项目里定义的类型和函数，通常以项目名作为前缀，可避免多数的命名重复情况。例如以 simple 作为类型和函数名的前缀：

```
typedef struct simple_client SimpleClient;
SimpleClient *simple_client_new(const char *host, const char *port);
```

simple 作为前缀，有些长了。为了让代码不至于太长，我努力将其缩写为 sim，于是上述代码可改写为

```
typedef struct sim_client SimClient;
SimClient *sim_client_new(const char *host, const char *port);
```

4.3 示意标记

下文为了叙事方便，使用了一些示意型的标记，它们以 C 语言注释文本的形式，用于表明向 C 源码文件增加和修改代码片段。在此，我以 foo.c 为例，解释这些标记的含义。

首先，是初始化标记。比如新建一份 C 源码文件 foo.c，并写入一些初始内容，或 foo.c 已存在，但对其完全重写，这类情况对应以下标记：

```
/* foo.c */
```

上述标记之后的内容便为 foo.c 的初始内容或重写的内容。

若在 foo.c 中增加一些代码，则用 C 语言的自增运算符 ++ 表示：

```
/* foo.c ++ */
```

若对 foo.c 中的一些内容进行修改，则用以下标记

```
/* foo.c [改] */
```

至于改动了何处，可根据变动后的内容中的不变之处自行确定。既然是改，定有不变之处。

4.4 字符串类的声明

C 语言标准库对字符串的支持颇为孱弱，以致许多用于处理字符串的函数在设计上似乎以吓退 C 语言初学者为荣，实则不得已而为之。想必有许多 C 语言初学者被 `strlen` 伤害过，其中应该也有很多人被伤害过，却不自知。不过，C 字符串之所以如此，是为了程序性能。倘若我们不在意损失程序些许性能，实现一个更好用一些的 C 字符串模块，不仅便于 sim 项目的开发，也是学习用 C 编写面向对象程序的良好起点。

首先，我们创建 sim-string.h 文件，它包含了一些它需要的头文件：

```
/* sim-str.h */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

我们定义一个类型 `SimString` 表示字符串类，它实际上是一个结构体类型的别名：

```
/* sim-str.h ++ */
typedef struct sim_str SimStr;
```

至于结构体 `sim_str` 的定义，不会在 sim-str.h 中给出，这意味着，作为字符串类的用户，你不必知情。

在面向对象编程中，类是一种数据类型，它只是比 `int`，`char *` 这些普通类型复杂一些，确切地说，类在绝大多数情况下是由一些基本类型组合而成。在编程中，我们需要为类型构造实例，如同用基本类型声明和定义变量，只不过我们通常将类的实例称为「类的对象」。构造对象，往往需要以函数的方式，这类函数便称为对象的构造函数。我们声明 `SimStr` 对象的构造函数为

```
/* sim-str.h ++ */
SimStr *sim_str(const char *raw);
```

sim_str 函数与结构体 sim_str 同名，不违背 C 标准，因为结构体有单独的命名空间，即 struct。这个函数的命名，可能会让很多人内心隐隐不安，他们更习惯 sim_str_new 这样的命名，而我觉得实在没必要多写 4 个字符。sim_str 基于 C 的普通字符串构造 SimStr 对象。

对象的构造函数未必只有一种。SimStr 对象也可以由下面这个函数构造：

```
/* sim-str.h ++ */
/* 将 m 个字节且前 n 个字节存有数据的缓冲区转化为字符串对象 */
SimStr *sim_str_absorb(void *buffer, size_t m, size_t n);
```

与构造函数相反的是析构函数，它用于释放对象，实际上这个函数应该叫解构。倘若你知道何为解构主义，想必会认同我的这个观点。不过，既然业界已约定俗成，我们也用析构函数这个术语。我们的析构函数，从现在开始，约定为构造函数名加上后缀 _free。SimStr 对象的析构函数为

```
void sim_str_free(SimStr *str);
```

如果你对 C 语言内存模型中的堆与栈的概念有基本的认识，即使你尚未看到我如何定义 SimStr 对象的构造与析构函数，应当也能猜得出，SimStr 是堆空间里的对象，事实正是如此。

SimStr 对象还有一些方法，它们实际上也是函数，只是若将 SimStr 对象视为一个生物体，这些函数是该生物体的一些行为。我们目前所需要的 SimStr 对象的方法如下：

```
/* sim-str.h ++ */
/* 获取字符串长度 */
size_t sim_str_size(SimStr *self);
/* 允许外界访问 data 成员 */
const char *sim_str_share(SimStr *self);
/* 将字符串对象转化为普通的 C 字符串 */
const char *sim_str_raw(SimStr *self);
```

根据代码中的注释，你大概清楚上述方法的用途。但是在读这些函数时，以 sim_str_raw 为例，我希望你的读法是这样的，「我（self），作为 SimStr 对象（结构体 sim_str 的实例）的原始形式（raw）」。对象的方法里的第一个参数，便是对象本身。那些在语法层面支持面向对象的语言，上述代码中的 self 往往是不可见的。

4.5 字符串类的实现

在 sim-str.c 中实现 sim-str.h 所声明的一切。首先，载入 sim-str.c 所需的头文件：

```
/* sim-str.c */
#include "sim-str.h"
```

然后定义名义上是 SimStr 的结构体 str_str：

```c
/* sim-str.c ++ */
struct sim_str {
    char *data; /* 支持以'0' 为字符串结束标志，亦可不支持 */
    size_t n;   /* data 中存储的字符串的长度（字节数） */
    size_t m;   /* data 占用内存空间的字节数，m >= n */
    const char *error;
};
```

结构体 sim_str 的前三个成员，我给出的注释已足够清楚了，若不能完全理解，在后续定义构造函数时，必见分晓。

也许你早已猜出我会如何定义这个结构体，但是你应该想不到，我会为 SimSir 对象配备一个字符串，用于记录 SimStr 对象可能的出错信息。此举虽然多耗费了一个指针的空间，但它很有用处。例如，在 SimSir 对象的构造函数中便可用到：

```c
/* sim-str.c ++ */
SimStr *sim_str(const char *raw) {
    SimStr *str = malloc(sizeof(SimStr));
    if (!str) {
        fprintf(stderr, "sim_str: malloc error!\n");
        return NULL;
    }
```

```c
    str->n = 0;
    str->m = 0;
    if (raw) {
        size_t n = strlen(raw);
        str->data = malloc(n * sizeof(char));
        if (!str->data) {
            str->error = "sim_str: failed to malloc 'data' member!";
        } else {
            str->n = n;
            str->m = n;
            str->error = NULL;
            memcpy(str->data, raw, n);
        }
    } else {
        str->data = NULL;
        str->error = NULL;
    }
    return str;
}
```

sim_str 函数考虑了构造 SimStr 对象过程中所有可能的情况。若你觉得我过于自信，请不吝摧毁。尤其是在为 str->data 分配内存时，若分配失败，依然可以返回一个 SimStr 对象，只不过它是带着错误信息的空的字符串对象。

SimStr 的错误信息成员，是上一节我在类的声明时并未想到的，现在需要 SimStr 对象具备一个能够判断自身是否安全的方法。首先，在 sim-str.h 中的头文件区包含 stdbool.h：

```
/* sim-str.h ++ */
#include <stdbool.h>
```

用于检测 SimStr 对象安全性的方法的声明与定义如下：

```
/* sim-str.h ++ */
/* self 安全，返回 true，否则返回 false */
bool sim_str_safe(SimStr *self);
```

```
bool sim_str_safe(SimStr *self) {
    if (self) {
        if (self->error) {
            fprintf(stderr, "%s\n", self->error);
            return false;
        }
    } else {
        fprintf(stderr, "sim_str_safe error: NULL pointer!\n");
        return false;
    }
    return true;
}
```

现在，为 SimStr 对象定义另一个构造函数：

```
/* sim-str.c ++ */
/* buffer 为 m 字节，前 n 个字节存有数据 */
SimStr *sim_str_absorb(void *buffer, size_t m, size_t n) {
    if (!buffer || n == 0) return NULL;
    SimStr *str = malloc(sizeof(SimStr));
    if (!str) {
        fprintf(stderr, "sim_str: malloc error!\n");
        return NULL;
    }
```

```
    str->n = n;
    str->m = m;
    str->data = buffer;
    str->error = NULL;
    return str;
}
```

buffer 被 sim_str_absorb 吸入后，创建的 SimStr 对象便得到了 buffer 所指空间的所有权。此时，倘若 buffer 还被其他对象或变量拥有，若程序通过它们释放了 buffer 指向的空间，对这个 SimStr 对象而言，极为致命。所以，用这个构造函数时，需要保证 SimStr 对象能够唯一拥有 buffer 所指空间的所有权。

SimStr 对象的析构函数定义如下：

```
/* sim-str.c ++ */
void sim_str_free(SimStr *str) {
    if (str) {
        if (str->data) free(str->data);
        free(str);
    } else {
        fprintf(stderr, "sim_str_free error: NULL pointer!");
    }
}
```

SimStr 对象的三个方法，实现起来颇为简单，只要你敢于放弃安全性：

```
/* sim-str.c ++ */
size_t sim_str_size(SimStr *self) {
    return self->n;
}
```

```
/* sim-str.c ++ */
const char *sim_str_share(SimStr *self) {
    return self->data;
}
```

```
/* sim-str.c ++ */
const char *sim_str_raw(SimStr *self) {
    if (self->n == self->m) {
        /* 对 self->data 扩容 1 个字节 */
        /* 用于存储 C 字符串结束符 */
        void *p = realloc(self->data, self->m + 1);
```

```
        if (!p) return NULL;
        else {
            self->m += 1;
            self->data = p;
        }
    }
    /* 添加字符串结束符 */
    *(self->data + self->n) = '\0';
    return self->data;
}
```

我觉得安全性不应该完全由开发基础库的人负责。类似于菜刀的安全性若完全由开发基础库的人负责，那它可能什么东西都切不了，最钝的菜刀是最安全的。若需要担保上述两个方法的安全性，可在使用它们之前用 `sim_str_safe` 检测 `SimStr` 对象的安全性。

4.6 防碰头设计

复杂程序，往往由许多模块构成，其中可能会出现一些模块依赖其他模块的情况，会导致某个模块的.h 文件可能会被其他模块多次 `#include`。倘若.h 文件里只有类型和函数的声明，它被多次 `#include`，只是增加了 C 编译器的工作量，但是若.h 含有变量或函数的定义，这种情况下会因变量或函数重复定义而导致编译出错。

使用条件宏，可避免.h 文件被多次重复载入。例如，在 sim-str.h 开始处，添加

```
#ifndef SIM_STR_H
#define SIM_STR_H
```

然后在 sim-str.h 的结尾处，添加

```
#endif
```

这三个宏组合起来，表达的含义是，当头文件 sim-str.h 第一次被 `#include` 时，宏 `SIM_STR_H` 尚未定义，于是 sim-str.h 的内容便被正常载入。当 sim-str.h 再次被 `#include`，此时编译器可确定宏 `SIM_STR_H` 已定义，便拒绝再次载入 sim-str.h 的内容。

4.7 测试

程序 sim-str-test.c 可验证 `SimStr` 类是否可用，更是在向你演示这个类的用法。

```
/* sim-str-test.c */
#include <sim-str.h>
int main(void) {
    const char *raw = "Hello world!";
    SimStr *foo = sim_str(raw);
```

```
    if (sim_str_safe(foo)) {
        printf("foo length: %lu\n", sim_str_size(foo));
        printf("foo content: %s\n", sim_str_raw(foo));
    }
    sim_str_free(foo);
```

```
    size_t n = strlen(raw);
    size_t m = 2 * n;
    void *buffer = malloc(m * sizeof(char));
    memcpy(buffer, raw, n);
    SimStr *bar = sim_str_absorb(buffer, m, n);
    if (sim_str_safe(bar)) {
        fwrite(sim_str_share(bar),
               sizeof(char),
               sim_str_size(bar),
               stdout);
        printf("\n");
        printf("%s\n", sim_str_raw(bar));
    }
```

```
    sim_str_free(bar);
    return 0;
}
```

用 gcc 编译 sim-str-test.c，并使用 -g 选项让编译所得程序 test 具有调试信息：

```
$ gcc -I. sim-str.c sim-str-test.c -g -o test
```

注意，在 sim-str-test.c 中，我们已经是像使用库的头文件那样使用 sim-str.h 了，足以彰显 sim 是一个库的志向。只是此刻 sim 还不是库，sim-str.h、sim-str.c 以及 sim-str-test.c 都在同一目录下。

通常，用尖括号包含的头文件，C 编译器会首先在系统的头文件目录（例如 /usr/include）中搜索头文件，不会在其他目录包括当前目录中搜索，而引号包含的头文件，像 gcc 这样的 C 编译器只会在当前目录中搜索，不会考虑其他路径。故而，在上述 gcc 命令中，我使用了 -I. 选项（其中的. 表示当前目录）让 gcc 在当前目录中搜索 sim-str.h。

运行 test：

```
$ ./test
foo length: 12
foo content: Hello world!
Hello world!
```

在 Linux 里，可使用 valgrind 工具检测 test 程序的内存安全性：

```
$ valgrind --leak-check=yes ./test
==35099== Memcheck, a memory error detector
... ... ... ... ...
foo length: 12
foo content: Hello world!
Hello world!
==35099==
==35099== HEAP SUMMARY:
==35099==     in use at exit: 0 bytes in 0 blocks
==35099==   total heap usage: 7 allocs, 7 frees, 1,138 bytes allocated
==35099==
==35099== All heap blocks were freed -- no leaks are possible
... ... ... ... ...
```

4.8 总结

也许你并不想逐一复制本文的所有代码片段，拼凑成完整的 sim-str.h 和 sim-str.c 文件，我允许你通过下面的连接不劳而获：

```
https://liyanrui.github.io/2025/c-network-programming/coop/sim-str.h
https://liyanrui.github.io/2025/c-network-programming/coop/sim-str.c
https://liyanrui.github.io/2025/c-network-programming/coop/sim-str-test.c
```

虽然我不喜欢 C++，Java 以及 C# 之类的面向对象编程语言，但我不得不承认，面向对象编程范式，着实是了不起的发明。现在，应该考虑如何用这种方式封装那些用起来颇为繁琐的套接字函数，以及如何在网络编程中使用本文实现的字符串类。

第 5 章 封装

5.1 前言

sim 项目的字符串类获得了业界好评，ywj 和 threebody 受到了鼓舞，希望加入 sim 的阵营。

5.2 客户端

新建一份文件 sim-network.h，除了防碰头设计，还包含了 sim-str.h：

```
/* sim-network.h */
#ifndef SIM_NETWORK_H
#define SIM_NETWORK_H
#include <sim-str.h>

#endif
```

以下示意标记

```
/* sim-network.h ++ */
... ... 代码片段... ...
```

我希望你能将其理解为，代码片段是添加在 `#define SIM_NETWORK_H` 和 `#endif` 之间的区域。

创建 sim-network.c 文件，先让它包含 sim-network.h：

```
/* sim-network.c */
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include "sim-network.h"
```

客户端程序 ywj 较为简单，我们不妨先从客户端类 `SimClient` 的定义开始：

```
/* sim-network.h ++ */
typedef struct sim_client SimClient;
```

```
/* sim-network.c ++ */
struct sim_client {
    int server;   /* 服务端套接字 */
    const char *error;
};
```

与 SimStr 相似，SimClient 对象也有 error 成员，用于记录它的生命周期内可能的出错信息。我会努力让 sim 项目中所有的对象，都拥有 error 成员。

SimClient 对象的构造函数的声明如下：

```
/* sim-network.h ++ */
SimClient *sim_client(const char *host, const char *port);
```

在实现 sim_client 函数之前，需要先将套接字地址由文字化转化为数字化的过程封装为一个仅在 sim-network.c 中使用的函数。不过，在定义这个函数之前，我们先定义一个函数指针：

```
/* sim-network.c ++ */
typedef int (*AddrSelector)(int sockfd,
                            const struct sockaddr *addr, socklen_t addrlen);
```

函数指针 AddSelector 指向的函数，用于 getaddrinfo 返回的地址列表中挑选可用地址。倘若你还没有对套接字 API 函数中的 connect 和 bind 的用法过于淡忘，想必应该明白了，它们就是这样的函数。基于这个函数指针，便可定义一个函数，从文字化套接字地址转化的数字化套接字地址列表中选择第一个可用的地址，如下：

```
/* sim-network.c ++ */
static int first_valid_address(const char *host,
                               const char *port, AddrSelector selector) {
    struct addrinfo hints, *addr_list;
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    if (getaddrinfo(host, port, &hints, &addr_list) != 0) {
        return -1;
    }
```

```
    int fd = -1;
    for (struct addrinfo *it = addr_list; it; it = it->ai_next) {
        fd = socket(it->ai_family, it->ai_socktype, it->ai_protocol);
        if (fd == -1) continue;
        if (selector(fd, it->ai_addr, it->ai_addrlen) == -1) {
            close(fd);
            fd = -1;
            continue;
        }
        break;
    }
```

```
    freeaddrinfo(addr_list);
    return fd;
}
```

在 first_valid_address 的辅助下，可以很简单的实现 SimClient 对象的构造函数：

```
/* sim-network.c ++ */
SimClient *sim_client(const char *host, const char *port) {
    int fd = first_valid_address(host, port, connect);
    if (fd == -1) return NULL;

    SimClient *client = malloc(sizeof(SimClient));
    if (!client) {
        fprintf(stderr, "sim_client error: failed to malloc!\n");
        return NULL;
    }
    client->server = fd;
    client->error = NULL;
    return client;
}
```

如果你对 C 语言的函数指针的用法不熟悉，那么上述代码可以作为一个很实用的示例。

SimClient 对象的析构函数的声明和定义如下：

```
/* sim-network.h ++ */
void sim_client_free(SimClient *client);
```

```
/* sim-network.c ++ */
void sim_client_free(SimClient *client) {
    if (client) {
        close(client->server);
        free(client);
    } else fprintf(stderr, "sim_client_free error: NULL pointer!\n");
}
```

上述代码，若有不解之处，请及时回顾第 **2** 章。

5.3 服务端

服务端类的声明和定义如下：

```
/* sim-network.h ++ */
typedef struct sim_server SimServer;
```

```c
/* sim-network.c ++ */
struct sim_server {
    int listener;  /* 用于监听的套接字 */
    int client;    /* 客户端套接字 */
    const char *error;
};
```

SimServer 对象的构造函数的声明和定义如下：

```c
/* sim-network.h ++ */
SimServer *sim_server(const char *host, const char *port);
```

```c
/* sim-network.c ++ */
SimServer *sim_server(const char *host, const char *port) {
    int fd = first_valid_address(host, port, bind);
    if (fd == -1) return NULL;
    if (listen(fd, 10) == -1) return NULL;
    SimServer *server = malloc(sizeof(SimServer));
    if (!server) {
        fprintf(stderr, "sim_server error: failed to malloc!\n");
        return NULL;
    }
    server->listener = fd;
    server->client = -1;
    server->error = NULL;
    return server;
}
```

SimServer 对象的析构函数声明如下：

```c
/* sim-network.h ++ */
void sim_server_free(SimServer *server);
```

sim_server_free 只需关闭用于 SimServer 对象的用于监听的套接字：

```c
/* sim-network.c ++ */
void sim_server_free(SimServer *server) {
    if (server) {
        close(server->listener);
        free(server);
```

```
    } else {
        fprintf(stderr, "sim_server_free error: NULL pointer!\n");
    }
}
```

SimServer 对象需要一个 run 方法，表达服务端开始运转，接受客户端的连接，该方法声明和定义如下：

```
/* sim-network.h ++ */
void sim_server_run_once(SimServer *self);
```

```
/* sim-network.c ++ */
void sim_server_run_once(SimServer *self) {
    int fd = accept(self->listener, NULL, NULL);
    if (fd == -1) {
        self->error = "sim_server_run_once error: failed to accept!";
    } else {
        /* 恢复 self 无错状态 */
        if (self->error) self->error = NULL;
    }
    self->client = fd;
}
```

至于 SimServer 对象的 client 套接字，每次当 SimServer 对象处理完一个客户端连接后，可使用 close 方法关闭，该方法的声明与定义如下：

```
/* sim-network.h ++ */
void sim_server_close(SimServer *server);
```

```
/* sim-network.c ++ */
void sim_server_close(SimServer *self) {
    if(self) {
        close(self->client);
        self->client = -1;
    } else {
        fprintf(stderr, "sim_server_close error: NULL pointer!\n");
    }
}
```

上述代码，若有不解之处，请及时回第 **3** 章。

5.4 数据发送与接收

SimClient 对象和 SimServer 对象之间的通信方法本质相同，区别只是使用的套接字不同，SimClient 用它的服务端套接字，而 SimServer 用它的客户端套接字。我们可以先实现两个通用的函数，对所有套接字通信一视同仁，然后基于它们为 SimClient 对象和 SimServer 对象定义数据发送和接收方法。

首先，实现向一个套接字写入数据。在第 **2** 章中，我们使用套接字 API 函数 send 向套接字写入数据。若 send 运行成功，其返回值是已写入数据的字节数。可能当时你并未深想这句话意味着什么。事实上，send 有时未必能将数据一次性完全写入套接字，会有一些剩余。网络状况不稳定或信息接收方处理速度较慢，都有可能导致这种情况出现。

解决上述问题的方法很简单，send 函数能够将剩余数据继续写入套接字，只需重复这个过程，直至数据没有剩余。该过程的实现如下：

```
/* sim-network.c ++ */
/* 将字符串对象中的数据发送到套接字 x */
static int send_robustly(int x, SimStr *str) {
    const char *buffer = sim_str_share(str);
    size_t n = sim_str_size(str); /* buffer 字节数 */
    size_t m = 0; /* 发送了多少字节 */
    size_t remaining = n; /* 还有多少字节未发送 */
    while (m < n) {
        ssize_t t = send(x, buffer + m, remaining, 0);
        if (t == -1) break;
        m += t;
        remaining -= t;
    }
    return (m < n) ? -1 : 0;
}
```

同理，套接字 API 函数 recv 函数也存在类似 send 这样的问题。从套接字读取数据时，有可能缓冲区太小，无法一次性读取所有数据，必须多次读取，直至无剩余数据为止。该过程的实现如下：

```
/* 从套接字 x 接收数据，返回值为接受到的数据 */
SimStr *recv_robustly(int x) {
    size_t m = 1024;
    char *data = malloc(m * sizeof(char));
    size_t n = 0; /* 已接收的字节数 */
```

```
    while (1) {
        if (n == m) { /* 扩容 */
            m *= 2;
            data = realloc(data, m);
        }
        size_t remaining = m - n; /* 剩余空间长度 */
        ssize_t h = recv(x, data + n, remaining, 0);
        if (h == -1) {
            free(data);
            return NULL;
        } else if (h == 0) break;
        else {
            n += h;
            if (h < remaining) break;
        }
    }
    if (n == 0) {
        free(data);
        return NULL;
    } else return sim_str_absorb(data, m, n);
}
```

注意，recv_robustly 在接收数据的过程中，发现缓冲区 data 已满，便将其扩容一倍，使得缓冲区空间总是够用。当发现接收数据的字节数 h 小于 data 剩余空间长度 rest 时，便意味着数据接收完毕。

基于上述的 send 和 recv 稳健版，可以为 Client 和 Server 对象定义数据发送和接收方法：

```
/* sim-network.h ++ */
void sim_client_send(SimClient *self, SimStr *str);
SimStr *sim_client_receive(SimClient *self);
```

```
/* sim-network.c ++ */
#define SEND_COMMON(member, error_info)  do { \
    if (send_robustly(self->member, str) == -1) { \
        self->error = error_info; \
    } else { \
      /* 恢复self 正常 */ \
        if (self->error) self->error = NULL; \
    } \
} while (0)
```

```c
void sim_client_send(SimClient *self, SimStr *str) {
    SEND_COMMON(server, "sim_client_send error!");
}
void sim_server_send(SimServer *self, SimStr *str) {
    SEND_COMMON(client, "sim_server_send error!");
}
```

```c
/* sim-network.h ++ */
void sim_server_send(SimClient *self, SimStr *str);
SimStr *sim_server_receive(SimClient *self);
```

```c
/* sim-network.c ++ */
#define RECEIVE_COMMON(member, error_info) do { \
    SimStr *str = recv_robustly(self->member); \
    if (!str) { \
        self->error = error_info; \
    } else { \
        /* 恢复 self 正常 */ \
        if (self->error) self->error = NULL; \
    } \
    return str; \
} while (0)
```

```c
SimStr *sim_client_receive(SimClient *self) {
    RECEIVE_COMMON(server, "sim_client_receive error!");
}

SimStr *sim_server_receive(SimServer *self) {
    RECEIVE_COMMON(client, "sim_server_receive error!");
}
```

上述代码，为了消除重复代码，我用了宏，但愿不会让你不适。应该有更好的避免代码重复的方法，待日后再予深究。

5.5 安全检测

上述 SimClient 和 SimServer 对象的一些方法的实现，利用对象的 error 记录了错误信息。我们为这两种对象的安全性提供了检测方法：

```c
/* sim-network.h ++ */
bool sim_client_safe(SimClient *self);
bool sim_server_safe(SimServer *self);
```

```c
/* sim-network.h ++ */
#define SIM_OBJ_SAFE(error_info) do { \
    if (self) { \
        /* 不太致命的错误 */ \
        if (self->error) return false; \
    } else { \
        /* 致命错误 */ \
        fprintf(stderr, error_info); \
        return false; \
    } \
    return true; \
} while (0)
```

```c
bool sim_client_safe(SimClient *self) {
    SIM_OBJ_SAFE("sim_client_safe error: NULL pointer!\n");
}

bool sim_server_safe(SimServer *self) {
    SIM_OBJ_SAFE("sim_server_safe error: NULL pointer!\n");
}
```

上述的 `SIM_SAFE` 宏，对 `sim_str_safe` 也适用，以后我们应该专门建立一个 sim-macro.h 文件，集中存放一些通用的宏。

5.6 重写 ywj

ywj 说，我期待这一天已经很久了！

```c
/* ywj.c */
#include "sim-network.h"
int main(void) {
    SimClient *ywj = sim_client("www.threebody.com", "8080");
    if (!ywj) {
        fprintf(stderr, "sim_client failed!\n");
        exit(-1);
    }
```

```
    /* 发送数据 */
    SimStr *msg_to = sim_str("ywj: I am here!");
    sim_client_send(ywj, msg_to);
    sim_str_free(msg_to);
    /* 从 www.threebody.com:8080 接收信息 */
    SimStr *msg_from = sim_client_receive(ywj);
    if (sim_str_safe(msg_from)) {
        printf("%s\n", sim_str_raw(msg_from));
    }
    sim_str_free(msg_from);
    /* 析构，退出 */
    sim_client_free(ywj);
    return 0;
}
```

用 gcc 编译上述 ywj.c，需要联编 sim-str.c 和 sim-network.c：

```
$ gcc -I. sim-str.c sim-network.c ywj.c -o ywj
```

5.7 重写 threebody

threebody 说，终于轮到我了！

```
/* threebody.c */
#include "sim-network.h"
int main(void) {
    SimServer *threebody = sim_server("localhost", "8080");
    if (!threebody) {
        fprintf(stderr, "sim_server failed!\n");
        exit(-1);
    }
    sim_server_run_once(threebody);
```

```
    /* 从客户端读取信息 */
    SimStr *msg_from = sim_server_receive(threebody);
    if (sim_str_safe(msg_from)) {
        printf("%s\n", sim_str_raw(msg_from));
    }
    sim_str_free(msg_from);
```

```
    /* 向客户端发送信息 */
    SimStr *msg_to = sim_str("threebody: Hi!");
    sim_server_send(threebody, msg_to);
    sim_str_free(msg_to);
    sim_server_close(threebody);
    /* 析构，退出 */
    sim_server_free(threebody);
    return 0;
}
```

编译：

```
$ gcc -I. sim-str.c sim-network.c threebody.c -o threebody
```

5.8 总结

如果你依然不想逐一复制本文所有代码片段，拼凑成完整的.h 和.c 文件，你依然可以不劳而获……

```
https://liyanrui.github.io/2025/c-network-programming/coop/sim-str.h
https://liyanrui.github.io/2025/c-network-programming/coop/sim-str.c
https://liyanrui.github.io/2025/c-network-programming/coop/sim-network.h
https://liyanrui.github.io/2025/c-network-programming/coop/sim-network.c
https://liyanrui.github.io/2025/c-network-programming/coop/ywj.c
https://liyanrui.github.io/2025/c-network-programming/coop/threebody.c
```

我们的 sim 项目，现已初具规模，它有两个模块了。此刻，你应该也基本熟悉如何用 C 编写简单的面向对象范式的程序了。面向对象编程已在业界独领风骚三十余年，自然是博大精深的，不过，恐龙也是博大精深的，统治地球上亿年。现有的面向编程技巧对于我们而言，似乎已经足够了。持而盈之，不若其已。

第 6 章　两朵乌云

6.1 前言

这份不守江湖规矩的网络编程指南，实际上已经几乎完结了。一台计算机上作为服务端的进程 A，能够与另一台计算机上作为客户端的进程 B 通信，本已是网络编程的全部内容了，更何况我们额外用了面向对象的范式，封装了一些常用的套接字函数。

不过，也许你并未注意到，我们的头上飘荡着一朵乌云。假设还有一台计算机上的进程 C，令它作为客户端，与 B 同时和进程 A 通信，此时进程 A 的程序该如何设计呢？

6.2 接受两个连接

假设 A 为 ywj，B 为 threebody，C 为另一个 ywj。

首先，需要改写 threebody。基于第 **5** 章中实现的 sim 项目，只需将 threebody 程序写为以下形式，便可实现与两个客户端通信：

```c
/* threebody.c */
#include "sim-network.h"
int main(void) {
    SimServer *threebody = sim_server("localhost", "8080");
    if (!threebody) {
        fprintf(stderr, "sim_server failed!\n");
        exit(-1);
    }
```

```c
    /* 接受 2 个连接 */
    for (int i = 0; i < 2; i++) {
        sim_server_run_once(threebody);
        if (sim_server_safe(threebody)) {
            /* 从客户端接收信息 */
            SimStr *msg = sim_server_receive(threebody);
            if (msg) {
                printf("%s\n", sim_str_raw(msg));
                sim_str_free(msg);
            }
```

```c
            /* 向客户端发送信息 */
            msg = sim_str("threebody: Hi");
            sim_server_send(threebody, msg);
```

```
            sim_str_free(msg);
            sim_server_close(threebody);
        }
    }
```

```
    sim_server_free(threebody);
    return 0;
}
```

编译 threebody.c，得到 threebody：

```
$ gcc -I. sim-str.c sim-network.c threeboy.c -o threebody
```

至于另一个 ywj，我们重新写一个程序作为她，如下：

```
/* other-ywj.c */
#include <unistd.h>
#include "sim-network.h"
int main(void) {
    SimClient *ywj = sim_client("www.threebody.com", "8080");
    if (!ywj) {
        fprintf(stderr, "sim_client failed!\n");
        exit(-1);
    }
```

```
    /* 延迟 15 秒，模拟存在许多运算 */
    sleep(15);
    /* 发送数据 */
    SimStr *msg_to = sim_str("other ywj: I am here!");
    sim_client_send(ywj, msg_to);
    sim_str_free(msg_to);
    /* 从 www.threebody.com:8080 接收信息 */
    SimStr *msg_from = sim_client_receive(ywj);
    if (msg_from) {
        printf("%s\n", sim_str_raw(msg_from));
        sim_str_free(msg_from);
    }
    /* 析构，退出 */
    sim_client_free(ywj);
    return 0;
}
```

需要注意的是，other-ywj.c 与第 5 章中的 ywj.c 的最大不同在于，前者在建立网络连接之后，在与服务端通信之前，主动延时约 15 秒，象征这个 ywj 比较犹豫迟疑……编译 other-ywj.c，得到 other-ywj：

```
$ gcc -I. sim-str.c sim-network.c other-ywj.c -o other-ywj
```

现在，在 Linux 环境里开始我们的实验。首先，打开三个终端窗口，令它们的工作目录皆为 threebody、other-ywj 以及 ywj 所在的目录。ywj 是第 5 章中的那个 ywj。

在第一个终端里运行 threebody：

```
$ ./threebody
```

然后在第二个终端里运行 other-ywj：

```
$ ./other-ywj
```

在运行 other-ywj 后，请随即尽快，假设在 5 秒之内，在第三个终端里运行 ywj：

```
$ ./ywj
```

上述三个程序的运行结果如下图所示：

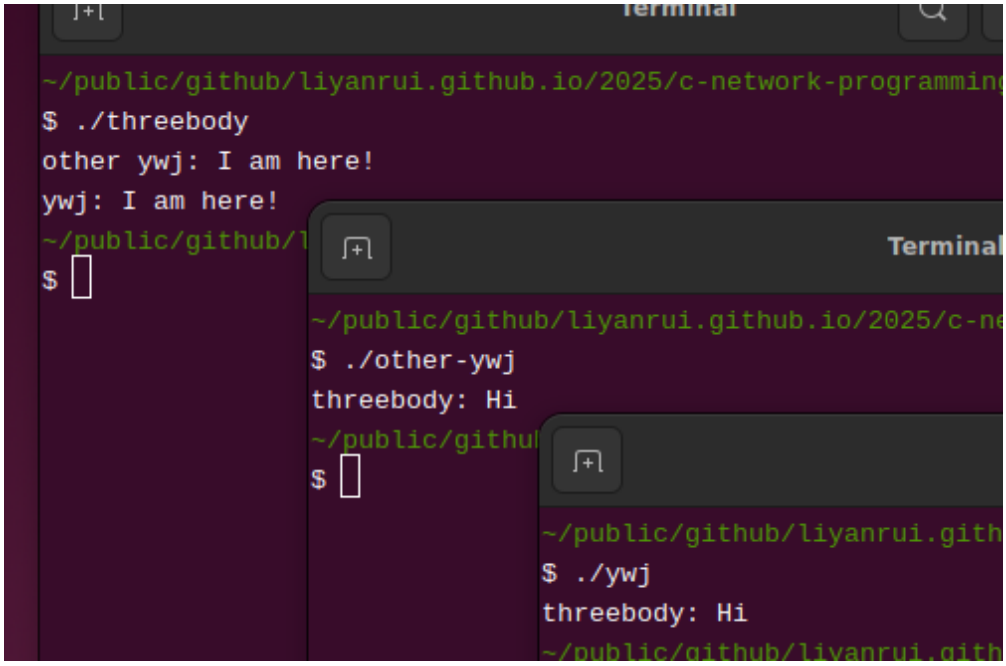

图 6.1　threebody, ywj 和 other-ywj

你会发现，在 other-ywj 运行后，它需要等待大约 15 秒，方能得到 threebody 的回应，这是正常的，因为她被我们设计为一个犹豫迟疑的人。真正让你觉得奇怪的现象应该是，随即运行的 ywj 也需要等待 10 秒左右，而且得到 threebody 的回应也是在 other-ywj 之后。

6.3 阻塞

在上一节的试验中，在运行 other-ywj 之后，随即运行 ywj 为什么会陷入等待呢？

原因是，threebody 还没学会孙悟空的分身术，它只能逐一与客户端建立连接和通信。当即 other-ywj 进程先发起连接，threebody 要与它完成通信后，方能与 ywj 建立连接并通信。这种形式的通信机制，称为阻塞式通信。该通信形式，在 threebody 看来，是非常自然的——有很多人给我发信息，我自然是要逐一进行处理的，但在最后与 threebody 连接并通信的 ywj 看来，非常不合理——我几乎是与其他进程同时发起了的连接，为什么么它们能很快得到回复，而我却等待很久呢？

真正对我们有用的网络服务端，往往同一时间需要接受成千上万的客户端的连接请求。让 threebody 接受更多个甚至无数个客户端的连接，并不难，只需将上述 threebody.c 中的 `for` 循环结构改为 `while (1)` 这样的无休止循环结构，便可得到永不休息，能处理无数个客户端连接的 threebody 了。基于 other-ywj 和 ywj 的验证，显然 threebody 不可能在同一时间能接受成千上万的连接，它只适用于处理寥寥几个客户端的连接。

```
/* threebody.c [改] */
/* 服务端永不停息 */
while (1) {
    sim_server_run_once(threebody);
    if (sim_server_safe(threebody)) {
        /* 从客户端接收信息 */
        SimStr *msg = sim_server_receive(threebody);
        if (msg) {
            printf("%s\n", sim_str_raw(msg));
            sim_str_free(msg);
        }
        /* 向客户端发送信息 */
        msg = sim_str("threebody: Hi");
        sim_server_send(threebody, msg);
        sim_str_free(msg);
        sim_server_close(threebody);
    }
}
```

在服务端造成通信阻塞的套接字 API 函数是封装在 `sim_server_run_once` 中的 `accept`，该函数在默认情况下是阻塞的，在与当前连接的客户端通信过程结束之前，`accept` 不会与其他客户端建立连接。实际上，用于发送和接收数据的 `send` 和 `recv` 也是阻塞的。

一些非套接字 API 函数也是阻塞的，例如 other-ywj.c 中使用的 `sleep` 函数以及等待用户输入信息的 `scanf` 函数。进程被阻塞时，操作系统会将其由「运行」状态切换为「等待」状态，此时进程不会占用 CPU，直到特定事件发生（例如有客户端发起网络连接），使其能够继续运行。

事实上，并非是某这些函数导致进程阻塞，而是当这些函数所读写的文件，其状态若为阻塞模式时，便会导致这些函数出现阻塞，从而导致进程进入等待状态。在 Unix 或 Linux 中，套接字是文件描述符，亦即套接字也是文件，只是专用于网络通信。使用 socket 并辅以 connect 或 bind 而构造的套接字，默认便是阻塞状态，故而导致访问套接字的 accept、send 和 recv 这些函数出现阻塞，从而导致进程阻塞。

因文件阻塞状态导致进程在对其读写时出现阻塞，称为 I/O 阻塞，而像 sleep 函数以及其他一些时间延迟函数导致的进程阻塞，称为时间阻塞。

6.4 不阻塞会如何？

在大致理解进程的阻塞机制之后，我们需要考虑的问题是，在网络通信中，若将服务端的用于监听和通信的套接字设置为非阻塞状态，会发生什么？

在 Unix 或 Linux 系统中，使用 fcntl 函数可修改文件状态。我不打算认真介绍这个函数的用法，原因是，很多应用软件层面的开发者（我也是其中一员）往往对操作系统层面的文件概念并不熟悉，引入过多细节，可能会破坏学习信心。对于网络编程而言，若将一个 socket 设为非阻塞状态，只需调用下面这个封装好的函数：

```
/* sim-network.c ++ */
#include <fcntl.h>

/* 参数 x 是一个 socket */
static int socket_nonblock(int x) {
    return fcntl(x, F_SETFL, fcntl(x, F_GETFL) | O_NONBLOCK);
}
```

其中，fcntl(x, F_GETFL) | O_NONBLOCK 用于获取套接字 x 的当前状态标志并将其与 O_NONBLOCK 合并，这样便拥有了非阻塞状态标志，并且包含 x 原有的状态标志，然后通过 fcntl(x, F_SETFL, 新标志) 将新标志赋予 x。

真正令初学者沮丧的是这段代码中的缩写。首先，fcntl，是 file control 的意思，即控制文件状态。F_GETFL 和 F_SETFL 都是操作指令，分别是 FILE GET FLAGS 与 FILE SET FLAGS 的缩写，可驱使 fcntl 获取或设定文件状态标志。文件状态标志是一组简单的二进制位，它们可以通过位运算符 | 进行组合，从而为文件设定多种状态。

fcntl 的返回值依赖于所用的操作指令。F_GETFL 会让 fcntl 返回文件的当前状态标志，而 F_SETFL 会让它返回 0。还有一些其他的情况，现在用不到。如果 fcntl 返回 -1，表示它运行失败了。

像 fcntl 这样的函数，它的参数以及所用的位运算，散发着古奥的气息。原因是这类函数在早期的 Unix 系统诞生时就存在了。函数名与操作指令之所以简写，犹如在纸张尚未发明的年代，古人在竹简上写着简洁但难懂的语句。我们不仅需要理解它们，还需要让它们具备现代气息。

现在改写 SimServer 对象的构造函数 sim_server 和方法 sim_server_run_once，为监听套接字和通信套接字增加 O_NONBLOCK 标志：

```c
/* threebody.c [改] */
SimServer *sim_server(const char *host, const char *port) {
    int fd = first_valid_address(host, port, bind);
    if (fd == -1) return NULL;
    if (listen(fd, 10) == -1) return NULL;

    SimServer *server = malloc(sizeof(SimServer));
    if (!server) {
        fprintf(stderr, "sim_server error: failed to malloc!\n");
        return NULL;
    }
    socket_nonblock(fd); /* 将 fd 设为非阻塞状态 */
    server->listener = fd;
    server->client = -1;
    server->error = NULL;
    return server;
}
```

```c
/* threebody.c [改] */
void sim_server_run_once(SimServer *self) {
    int fd = accept(self->listener, NULL, NULL);
    if (fd == -1) {
        self->error = "sim_server_run_once error: failed to accept!";
    } else {
        socket_nonblock(fd); /* 将 fd 设为非阻塞状态 */
        /* 恢复 self 无错状态 */
        if (self->error) self->error = NULL;
    }
    self->client = fd;
}
```

重新编译 threebody.c，运行所得程序 threebody：

```
$ gcc -I. sim-str.c sim-network.c threebody.c -o threebody
$ ./threebody
```

然后像上文的试验那样，在两个终端里分别且近乎同时运行 other-ywj 和 ywj，会发生什么奇迹……或灾难呢？

奇迹是有的，ywj 可以立刻得到 threebody 的回应，other-ywj 会在 15 秒后得到 threebody 的回应，这个结果正是我们所期望的，果断的 ywj 立刻得到回应，迟疑的 ywj 得到迟一些的回应。灾难也是有的，结果表明 threebody 大概率收不到来自 ywj 和 other-ywj 的信息。

threebody 无法收到来自客户端的信息，原因是 sim_server_receive 并非我们所认为的那样稳健，sim_client_receive 亦是如此。ywj 和 other-ywj 都收到了 threebody 的回复，那是因为它们的通信套接字并未设成非阻塞状态。事实上，若套接字为非阻塞状态，sim_server_send 和 sim_client_send 也可能会出现类似的问题。

现在的 threebody 是无休止运行的，完成上述试验后，在其运行的终端里，请务必使用 Ctrl + C 终止它！请务必使用 Ctrl + C 终止它！请务必使用 Ctrl + C 终止它！重要的事，要说三遍的。Ctrl + C，即摁下 Ctrl 键不放，再摁 C 键。

6.5 可以挽救的错误

在第 5 章中，SimClient 和 SimServer 对象的 receive 和 send 方法，分别基于 recv_robustly 和 send_robustly 函数实现，我们需要对这两个函数再作一些修改，进一步提高它们的稳健性。

在 recv_robustly 中，当套接字函数 recv 访问的套接字 x 为非阻塞状态时，recv 会直接返回 -1，而非阻塞式读取 x 中的内容，而 recv_robutly 对这种情况的处理方式是

```
ssize_t h = recv(x, data + n, remaining, 0);
 if (h == -1) {
     free(data);
     return NULL;
 }
```

我需要告诉你一个秘密。这个秘密，我已隐瞒了很久，现在不得不说出来。Unix 或 Linux 的大多数系统调用和套接字函数，出错时，会返回 -1，同时会将一个错误码存入全局变量 errno。recv 和 send 在出错时，返回 -1，有时并非意味着错误无法挽回。

recv 遇到以下情况：

- 当套接字处于非阻塞状态，若接收缓冲区为空时；
- 某些数据传输协议（如 UDP）没有缓冲区，在非阻塞模式下未收到数据时
- 对端已关闭连接或停止发送数据时

都会返回 -1，并将 errno 的值赋为 EAGAIN。上一节的 threebody 大概率收不到 other-ywj 和 ywj 发送的数据，便属于上述第 1 种情况。

send 在套接字为非阻塞状态，且系统为数据发送准备的缓冲区内容已满时，会立即返回 -1，并将 errno 的值赋为 EAGAIN。你还需要知道，accept 函数，在用于监听的套接字为非阻塞状态且操作系统的监听队列中没有已建立的连接时，也会返回 -1，并将 errno 的值赋为 EAGAIN。

还有一个事实，accept、send 和 recv 这些套接字函数在工作时，可能会因操作系统发出的中断信号而导致出错，返回 -1，此时 errno 的值为 ENITR。可能你还不清楚何谓操作系统发出的中断信号，你可简单理解为操作系统必须处理一些紧急事务，无暇关心网络通信之事。遇到这种情况，我们也需要重新启动 accept、send 和 recv，让它们尽可能地运行成功。

基于上述秘密，对 sim_server_run_once、send_robustly 和 receive_robustly 略作修改：

```c
/* sim-network.c ++ */
#include <errno.h>
```

```c
/* sim-network.c ++ */
void sim_server_run_once(SimServer *self) {
    int fd;
    while (1) {
        fd = accept(self->listener, NULL, NULL);
         if (fd == -1) {
            if (errno == EAGAIN || errno == EINTR) continue;
            else {
                self->error = "sim_server_run_once error!";
                break;
            }
        } else {
            /* 将 fd 设为非阻塞状态 */
            socket_nonblock(fd);
            /* 恢复 self 无错状态 */
            if (self->error) self->error = NULL;
            break;
        }
    }
    self->client = fd;
}
```

```c
/* sim-network.c [改] */
static int send_robustly(int x, SimStr *str) {
    const char *buffer = sim_str_share(str);
    size_t n = sim_str_size(str); /* buffer 字节数 */
    size_t m = 0; /* 发送了多少字节 */
    size_t remaining = n; /* 还有多少字节未发送 */
    while (m < n) {
        ssize_t t = send(x, buffer + m, remaining, 0);
        if (t == -1) {
            if (errno == EAGAIN || errno == EINTR) continue;
            else break;
        }
        m += t;
        remaining -= t;
    }
```

```
    return (m < n) ? -1 : 0;
}
```

```
/* sim-network.c [改] */
SimStr *recv_robustly(int x) {
    size_t m = 1024;
    char *data = malloc(m * sizeof(char));
    size_t n = 0; /* 已接收的字节数 */
    while (1) {
        if (n == m) { /* 扩容 */
            m *= 2;
            data = realloc(data, m);
            if (!data) {
                free(data);
                return NULL;
            }
        }
        size_t remaining = m - n; /* 剩余空间长度 */
        ssize_t h = recv(x, data + n, remaining, 0);
        if (h == -1) {
            if (errno == EAGAIN || errno == EINTR) continue;
            else {
                free(data);
                return NULL;
            }
        } else if (h == 0) break;
        else {
            n += h;
            if (h < remaining) break;
        }
    }
    if (n == 0) {
        free(data);
        return NULL;
    } else return sim_str_absorb(data, m, n);
}
```

唉……但愿永远不再处理这些乱七八糟的安全问题。一旦追求人身安全，人必须先让自己的思维进入十八层地狱游走一番！

在 sim-network.c 中完成上述修改后，重新编译 threebody.c，然后重新做一遍上一节的试验。然后你会发现，奇迹出现了，但是灾难也出现了。奇迹是，threebody 能收到 other-ywj 和 ywj 发送的数据。灾难是，ywj 又回到不得不等待 other-ywj 先收到 threebody 回复的情况了。

事实上，上述对 `recv_robustly` 和 `send_robustly` 的修改，导致二者变成了时间阻塞，即遇到 `EAGAIN` 错误，就重新接收或发送数据，直至成功为止。

现在的 threebody 是无休止运行的，完成上述试验后，在其运行的终端里，请务必使用 Ctrl + C 终止它！请务必使用 Ctrl + C 终止它！请务必使用 Ctrl + C 终止它！重要的事，要说三遍的。

6.6 CPU 很忙

将套接字设为非阻塞状态后，为什么每次试验 threebody 后，我会告诫你，务必用 Ctrl + C 终止它呢？

现在，假设你在某个终端里运行了 threebody，可以不必运行 ywj 和 other-ywj。然后再打开一个终端，执行以下命令

```
$ top -p $(pgrep threebody)
```

`top -p` 命令可以查看指定 ID 的进程的资源占用情况。`$(pgrep threebody)` 是子 Shell 命令，即在当前终端里运行的 Shell 里又开启了一个 Shell，让后者运行 `pgrep threebody` 命令，并将结果返回给上层 Shell。`pgrep` 的用途根据进程对应的程序名查询该进程的 ID。如果你不知道 Shell 是什么……就是终端里接受你的输入的命令并执行的这个东西。终端只是个躯体，Shell 是它的灵魂。

执行上述命令后，你会看到类似以下信息：

```
  PID USER      ...  %CPU  %MEM  ... COMMAND
11821 garfileo  ...  99.7   0.1  ... threebody
```

重点在 CPU 那列，threebody 的运行，占用了 99.7% 的 CPU 时间！假如是冬天，你没有用 Ctrl + C 强行关闭 threebody，也许你可以不用交暖气费或开空调制暖了。

6.7 总结

我们的头上依然还飘荡着那朵乌云，可是现在似乎又多了一朵。不过我已隐约看到可以将其驱散的希望了。请怀念这一刻吧，这大概是我们最后的田园时光了。作为服务端的进程，很快要进化得令我们这些旧时代的人觉得它面目全非，难以理解，难以调试。

现在，到了你不劳而获的时间！

```
https://liyanrui.github.io/2025/c-network-programming/blocking/sim-str.h
https://liyanrui.github.io/2025/c-network-programming/blocking/sim-str.c
https://liyanrui.github.io/2025/c-network-programming/blocking/sim-network.h
https://liyanrui.github.io/2025/c-network-programming/blocking/sim-network.c
https://liyanrui.github.io/2025/c-network-programming/blocking/ywj.c
https://liyanrui.github.io/2025/c-network-programming/blocking/other-ywj.c
https://liyanrui.github.io/2025/c-network-programming/blocking/threebody.c
```

第 7 章 I/O 多路复用

7.1 前言

网络编程的那两朵乌云，驱散之道不在网络层面，而在网络中的每台计算机的操作系统层面。操作系统提供了一些监视文件内容是否发生变化的功能。

假设存在两个文件 a.txt 和 b.txt，基于操作系统的 `select` 机制能够编写一个可以监视 a.txt 和 b.txt 的内容是否发生变化的程序。这种程序所实现的功能即 I（输入）/O（输出）多路复用，即操作系统提供的 `select` 函数，能够监视多个 I/O 设备。a.txt 和 b.txt 与 I/O 设备有什么关系呢？在 Unix 或 Linux 系统中，I/O 设备皆表示为文件，我只是这两个文件表示 I/O 设备。

7.2 重温文件

熟悉 C 语言的人，肯定知道，打开一份文件，需要使用 C 标准库提供的 `fopen` 函数，但是现在我们需要掌握更底层的 `open`，它也能打开文件，而且它是 POSIX 系统（Unix 或 Linux）暴露给用户的接口。像 `open` 这样的函数亦称系统调用，而 `fopen` 则称为库函数，后者可基于前者实现，且功能更为丰富。实际上，套接字 API 函数也是系统调用。

以下代码可打开 a.txt 文件：

```
int fd_a = open("a.txt", O_RDONLY | O_NONBLOCK);
if (fd_a == -1) {
    fprintf(stderr, "failed to open a.txt");
    exit(-1);
}
```

上述代码以只读（`O_RDONLY`）和非阻塞（`O_NONBLOCK`）模式打开 a.txt 文件。若打开文件失败，`open` 返回 `-1`，否则返回所打开的文件的描述符。对于普通文件——通常是位于硬盘上的那些，`O_NONBLOCK` 是没必要的，因为对普通文件都是非阻塞的，亦即随时可以读取其内容，亦可随时向其写入数据。

我们是用 a.txt 指代一个文件，而对于操作系统而言，它会用 `fd_a` 这样的文件描述符指代一个文件。还记得，套接字也是文件描述符吗？人类总是习惯用文字指代某物，而操作系统则习惯使用数字指代某物。两者同出，异名同谓。玄之又玄，众妙之门。

`open` 与 `fopen` 的行为有一些相似性。例如以下代码使用 `fopen` 打开文件：

```
FILE *fp_a = fopen("a.txt", "r");
if (!fp_a) {
    fprintf(stderr, "failed to open a.txt");
    exit(-1);
}
```

open 返回的是文件描述符，而 fopen 返回的是文件指针。顺便多说一点，若使用 fopen 打开文件，但是有时又需要文件描述符，可使用 fileno 获取，例如：

```
int fd_a = fileno(fp_a);
```

打开的文件，可使用 read 和 write 读写，只是需要我们在内存中备好一个缓冲区，然后使用 close 予以关闭。以下示例 foo.c 是向文件写入数据：

```
/* foo.c */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>    /* 声明了 open 和 close 函数 */
#include <unistd.h>   /* 声明了 read 和 write 函数 */
```

```
int main(void) {
    /* 以只写（write only）模式打开文件 */
    int fd_a = open("a.txt", O_WRONLY);
    if (fd_a == -1) {
        fprintf(stderr, "failed to open file!\n");
        exit(-1);
    }
    /* 备好缓冲区 */
    char buffer[] = "你好，文件！";
    /* 向文件写入数据 */
    ssize_t n = write(fd_a, buffer, sizeof(buffer));
    if (n == -1) {
        fprintf(stderr, "failed to write file!\n");
        exit(-1);
    }
    close(fd_a); /* 关闭文件 */
    return 0;
}
```

write 函数若未出错，返回值是向文件写入的字节数，若出错，则返回 -1。close 用于关闭文件，实质上是向操作系统归还文件描述符。对于操作系统而言，与内存相似，文件描述符也是一种资源，借用了，需要归还。不过，即使不主动归还，在程序终止运行后，操作系统也会自动回收，不过为了严谨，建议主动归还。

编译 foo.c：

```
$ gcc foo.c -o foo
```

要运行 foo 程序，需要先建立文件 a.txt，在 Linux 系统中使用 touch 命令可创建一份空文件：

```
$ touch a.txt
```

然后运行 foo：

```
$ ./foo
```

要查看 foo 是否向 a.txt 写入了数据，不必用文件编辑器，可使用 cat 命令（more 或 less 命令亦可）：

```
$ cat a.txt
你好，文件！
```

文件内容的读取，略为繁琐，因为文件内容可能会超过我们为它准备的缓冲区，需要多次读取。看以下示例 bar.c：

```
/* bar.c */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
```

```
int main(void) {
    /* 以只读（write only）模式打开文件 */
    int fd_a = open("a.txt", O_RDONLY);
    if (fd_a == -1) {
        fprintf(stderr, "failed to open file!\n");
        exit(-1);
    }
```

```
    /* 备好缓冲区 */
    char buffer[1];
    /* 从文件读入数据 */
    while (1) {
        ssize_t n = read(fd_a, buffer, sizeof(buffer));
        if (n == 0) break;
        else if (n == -1) {
            fprintf(stderr, "failed to read file!\n");
            exit(-1);
        } else {
            /* 将读取的数据显示在标准输出设备 */
            write(STDOUT_FILENO, buffer, n);
        }
    }
```

```
    close(fd_a); /* 关闭文件 */
    return 0;
}
```

为了展示文件的数据量超出缓冲区的情况，我故意将缓冲区的长度设成 1 个字节。read 函数读取到的数据，若在屏幕上显示，可使用 write 函数向标准输出设备——STDOUT_FILENO 写入数据。通常情况下，计算机屏幕是标准输出设备。STDOUT_FILENO 实际上是 1。类似地，STDIN_FILENO 是标准输入设备，值为 0，默认是键盘。再者，标准错误设备 STDERR_FILENO，值为 2，默认是屏幕，用于输出错误信息。

在库函数层面，像 fgets 和 fprintf 这样的函数，所用的标准输入、标准输出和标准错误设备分别是 stdin，stdout 和 stderr。我也一直习惯用 fprintf 向 stderr 输出错误信息。

现在，你应该不再害怕 Unix 或 Linux 系统层面的文件读写操作了。

7.3 文件描述符集合

为了监视打开的多个文件，亦即多路 I/O，需要为 select 函数（它也是系统调用）准备一个集合，用于存储文件描述符。

POSIX 系统调用层面为文件描述符集合定义了数据结构 fd_set，并围绕该结构实现了一组集合n预算宏，诸如 FD_ZERO、FD_SET、FD_CLR 以及 FD_ISSET。使用这些宏操作文件描述符集合，可以让我们无需关心 fd_set 的定义。可能没有什么事情比无需关心底层如何实现更令人觉得快乐了。

以下代码，将打开的 a.txt 和 b.txt 文件对应的描述符 fd_a 和 fd_b 添加到文件描述符集合：

```
fd_set fds;
FD_ZERO(&fds);
FD_SET(fd_a, &fds);
FD_SET(fd_b, &fds);
```

FD_ZERO 将文件描述符集合清空，亦即将其所占空间的数据皆初始化为 0。FD_SET 将文件描述符加入集合。FD_CLR 可从文件描述符集合中移除给定的文件描述符，例如：

```
FC_CLR(fd_b, &fds);
```

FD_ISSET 用于判断给定的文件描述符是否在集合中，例如

```
if (FD_ISSET(fd_a, &fds)) {
    printf("fd_a is in fds.\n");
} else {
    printf("fd_a is not in fds.\n");
}
```

7.4 阻塞式轮询

select 的作用很简单，它能够轮循一组文件——以文件描述符集合表示，从中获得符合需求的子集。不过，理解 select 函数的作用，犹如从牛顿力学跃迁到广义相对论，因为它能够让你看到，cpu 的速度是不变的，但时间却可以膨胀或收缩，同时空间在收缩或膨胀，这就是 I/O 多路复用的魅力所在。

select 的声明为

```
#include <sys/select.h>
int select(int nfds, fd_set * readfds,
           fd_set * writefds,
           fd_set * exceptfds,
           struct timeval *timeout);
```

select 接收 5 个参数，第一个参数表示文件描述符的最大值，第二、三、四个参数，都是文件描述符集合，第五个参数表示超时时间值。对于本文要解决的问题，我们只需关心前两个参数，后三个参数皆设为 NULL 即可，而且函数声明里的 _Nullable 也明确标定了这些参数是可以为 NULL 的。

select 的第一个参数也许是最不好懂的，它的作用是让 select 有机会终止轮询文件描述符集合。以下构造文件描述符集合的过程顺便记录了集合中的文件描述符的最大值：

```
fd_set fds;
int fd_max = (fd_a > fd_b) ? fd_a : fd_b;
FD_ZERO(&fds);
FD_SET(fd_a, &fds);
FD_SET(fd_b, &fds);
```

我们可以写出以下代码，遍历 fds：

```
for (int i = 0; i < fd_max + 1; i++) {
    if (FD_ISSET(i, &fds)) {
        /* i 是文件描述符，可基于它读写文件 */
    }
}
```

select 函数也像上述循环语句这般遍历文件描述符集合，故而它的第一个参数是集合中文件描述符最大值加上 1。倘若我们只让 select 轮循文件是否可读的文件描述符集合，在上述构建文件描述符集合 fds 之后，可像下面这样调用它：

```
int ret = select(fd_max + 1, &fds, NULL, NULL, NULL);
```

若 select 成功，返回值是文件描述符集合中有多少个文件描述符指代的文件是可读的（属于第 2 个参数），可写的（属于第 3 个参数），出现异常的（属于第 4 个参数），上例只关心可读的。若 select 返回 -1，表示轮询文件过程出错。

注意，重点来了。上述 select 函数调用语句里，fds 即是输入，也是输出。select 会将 fds 中不可读的文件描述符剔除，剩下的皆为可读的。

注意，又一个重点来了。select 函数，若其第五个亦即最后一个参数若为 NULL，则意味着它是阻塞的。在上例中，select 函数一直等到文件描述符集合内出现可读的描述符方能结束。不过，select 阻塞行为可通过最后一个参数予以控制，能够让它像 sleep 函数那样阻塞一个时段，甚至也能让它变成非阻塞的——让超时时间为 0。例如

```
/* 阻塞模式（timeout = NULL） */
select(nfds, &fds, NULL, NULL, NULL);

/* 非阻塞模式（timeout = 0）*/
struct timeval timeout = {0, 0};
select(nfds, &fds, NULL, NULL, &timeout);

/* 超时阻塞模式（timeout = 5秒）*/
struct timeval timeout = {5, 0};
select(nfds, &fds, NULL, NULL, &timeout);
```

7.5 文件可读监视器

现在，可以着手写一个简单的文件可读监视器程序了。该程序能够监控与其位于相同目录下的 a.txt 和 b.txt 文件是否可读，若可读，则读取其中全部内容并显示。下面的 files-monitor.c 实现了该程序：

```
/* files-monitor.c */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/select.h>
#include <unistd.h>
```

```
int main(void) {
    /* 打开 a.txt 和 b.txt */
    int fd_a = open("a.txt", O_RDONLY);
    if (fd_a == -1) {
        fprintf(stderr, "failed to open a.txt");
        exit(-1);
    }
```

```c
int fd_b = open("b.txt", O_RDONLY);
if (fd_b == -1) {
    fprintf(stderr, "failed to open b.txt");
    exit(-1);
}
```

```c
/* 构建文件描述符集合 */
int fd_max = (fd_a > fd_b) ? fd_a : fd_b;
fd_set fds;
FD_ZERO(&fds);
FD_SET(fd_a, &fds);
FD_SET(fd_b, &fds);
```

```c
/* 阻塞式轮询文件描述符集 */
fd_set fds_copy = fds;
char buffer[1024];
while (1) {
    fds = fds_copy;
    int a = select(fd_max + 1, &fds, NULL, NULL, NULL);
    if (a == -1) {
        fprintf(stderr, "select error!\n");
        exit(-1);
    }
    /* 读取文件内容 */
    for (int i = 0; i < fd_max + 1; i++) {
        if (FD_ISSET(i, &fds)) { /* 找到一个可读文件 */
            ssize_t n;
            while ((n = read(i, buffer, sizeof(buffer))) > 0) {
                write(STDOUT_FILENO, buffer, n);
            }
            if (n == -1) {
                fprintf(stderr, "read failed!\n");
                exit(-1);
            }
        }
    }
}
```

```
    close(fd_b);
    close(fd_a);
    return 0;
}
```

编译 files-monitor.c：

```
$ gcc files-monitor.c -o files-monitor
```

下面是验证 files-monitor 能否正常工作的过程：

```
$ touch a.txt b.txt
$ ./files-monitor
```

打开另一个终端（命令行窗口），以 files-monitor 所在目录为工作目录，执行以下命令：

```
$ echo "你好, a.txt! " >> a.txt
$ echo "你好, b.txt! " >> b.txt
```

echo 命令所用的 >> 符号，是 Shell 的输出重定向符号，它可以将前面的命令的输出，添加到后面指定的文件里，且不会覆盖文件的原有内容，只是添加到文件原有内容之后。要使用输出重定向覆盖文件内容，可使用 > 符号。若不明白我在说什么，那你需要花一点时间，熟悉一下 Shell 的基本用法。有很多种 Shell，我用的是 Bash。不懂 Shell，在 Unix 或 Linux 的世界里，像是不会用腿走路。

files-monitor 应该会输出

```
你好, a.txt!
你好, b.txt!
```

然后它会继续运行，等待 a.txt 和 b.txt 里有新的内容出现，读取后将其输出到屏幕。

假设 files-monitor 持续运行，下面设法查看该进程的资源占用情况。首先获取 files-monitor 进程 ID：

```
$ pgrep -l files-monitor
57897 files-monitor
```

在我的机器上，目前正在运行的 files-monitor 的 ID 是 57897。使用 top 命令便可查看该进程对 CPU 和内存的占用情况：

```
$ top -p 57897
```

结果如图 **7.1** 所示。

```
top - 00:10:18 up 16:36,  1 user,  load average: 2.11, 2.05, 2.07
Tasks:   1 total,   1 running,   0 sleeping,   0 stopped,   0 zombie
%Cpu(s): 38.0 us, 13.7 sy,  0.0 ni, 47.1 id,  1.2 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7859.1 total,   1016.0 free,   3847.1 used,   3853.3 buff/cache
MiB Swap:   2048.0 total,   2042.0 free,      6.0 used.   4012.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  57897 garfileo  20   0    2548   1024   1024 R  98.3   0.0  60:19.66 files-monitor
```

图 7.1　files-monitor 资源占用

可以看到，files-monitor 正在疯狂地霸占着 CPU，在 98.3% 的时间里，它独占 CPU。

7.6 select 可堪大用？

在第 6 章，我们讨论了服务端在面临许多客户端同时访问时的窘迫，并试图从解除套接字阻塞的途径提高服务端处理并发访问的能力，结果我们失败了。为了稳健地接收和发送数据，似乎服务端要么会陷入 I/O 阻塞，要么会陷入时间阻塞。当服务端陷入时间阻塞的时候，我们看到了第 2 朵乌云，即服务端疯狂地吞噬着 CPU。

我原本以为 `select`函数能够驱散这朵乌云，然而上一节的 files-monitor 试验，却浮现了同样的乌云。

也许你早已有些疑惑了。这篇文章只是致力写了一个监视文件的程序，与网络编程有什么关系呢？有关系。因为套接字也是文件描述符啊，亦即它指代的对象也是文件，至少在 Unix 或 Linux 中如此。不过，套接字指代的文件，并非上文中 a.txt 和 b.txt 这样的普通文件，它是操作系统在内存中为我们构造的一种像文件的数据结构。

普通文件总是可读的，套接字指代的文件未必如此，不过后者只是我的猜想。下一步工作，我们需要基于 select 为网络程序的服务端构建 I/O 多路复用机制，验证它是否能避免疯狂消耗 CPU。

7.7 总结

乌云犹在，可是你意外学会了不少 Linux 命令。不要忘记，用 `Ctrl` 和 `C` 组合键终止 files-monitor 程序，否则它会继续自我感动而努力运行着。

第 8 章　长缨在手

8.1 前言

我们已经初步掌握了 select 函数的用法，基于它能够监控多个文件，从中选出所有可读、可写以及出现异常的文件，此即 I/O 多路复用，实际上其全称是同步 I/O 多路复用。这个术语中的「同步」，表现在 select 是阻塞的。select 将一组文件提交给操作系统后，需要等待操作系统的「审批」，原因是只有操作系统能够最快知道文件是可读、可写还是存在异常，因为所有文件都是它负责管理。「多路复用」指的是 select 选出的一组文件，每个文件都是一条 I/O 通道，它们在「复用」当前这个进程对它们的调度，以完成数据传输。

在网络编程中，套接字也是文件，亦即它也是 I/O 通道。在编写服务端程序时，我们可以将一组套接字通过 select 提交给操作系统审批，从中选出可读、可写或有异常者，由它们复用服务端进程。这种机制能提高服务端进程的并发能力，亦即能提高服务端进程面对多个客户端同时发起连接这类情况的处理能力吗？

之前，我们实现的 threebody 程序，它只能逐一接受客户端的连接，并与之通信。它无法知道当前接受了一个客户端的连接之后，还有没有其他客户端在排队等待。同步 I/O 多路复用机制，能够让 threebody 知晓当前有多少客户端与它建立了连接，并知晓这些客户端有哪些发来了数据，以及有哪些在等待接收数据。这就是计算机世界里的广义相对论，在一个假想的时空里，有的客户端的时间很慢，有的客户端的时间很快，但是服务端与所有接入的客户端通信消耗的总时间，可能远小于每个客户端与服务端通信时长的总和。

时间，是一种设备。之所以发明它，是为了让所有事情不同时发生。

为了清晰地展现基于同步 I/O 多路复用机制的服务端并发能力的改进策略，我们暂不使用或改造第 **5** 章创造的 SimServer 类，而是继续沿袭第 **3** 章琐碎但直白的套接字函数以及普通的 C 字符串操作。待彻底熟悉这一改进策略后，再令其与 SimServer 类发生化学反应。

8.2 客户端集

为了更快抓住问题的本质，我们需要忽略一些细节，诸如将文字形式的套接字地址转化为数字，用 socket 和 bind 函数构造用于监听客户端连接的套接字等内容。现在，假设已存在非阻塞的监听套接字 listener，在 select 的帮助下，与客户端建立连接的过程可实现为以下代码：

```
/* 最多可同时与 100 个客户端建立连接 */
#define MAX_CLIENTS 100
/* 用于存储与服务端连接的客户端 */
int clients[MAX_CLIENTS] = {0}; /* 数组所有元素初始化为 0 */
fd_set read_fds;
fd_set write_fds;
int fd_max = listener;
```

```c
while (1) {
    FD_ZERO(&read_fds);
    FD_ZERO(&write_fds);
    FD_SET(listener, &read_fds);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        int client = clients[i];
        if (client > 0) {
            FD_SET(client, &read_fds);
            FD_SET(client, &write_fds);
            /* 更新文件描述符最大值 */
            if (client > fd_max) fd_max = client;
        }
    }
    /* 轮询套接字集合 */
    if (select(fd_max + 1, &read_fds, &write_fds, NULL, NULL) == -1) {
        if (errno != EINTR) {
            fprintf(stderr, "select error!\n");
            continue;
        }
    }
    /* 建立新连接 */
    int new_client;
    if (FD_ISSET(listener, &read_fds)) {
        new_client = accept(listener, NULL, NULL);
        if (new_client == -1) {
            fprintf(stderr, "accetp error!\n");
            continue;
        }
        /* 将 new_client 设为非阻塞 */
        socket_nonblock(new_client);
        /* 将 new_client 加入 clients */
        for (int i = 0; i < MAX_CLIENTS; i++) {
            if (clients[i] == 0) {
                clients[i] = new_client;
                printf("连接客户端 %d\n", new_client);
                break;
            }
        }
    }
```

```
    /* 从客户端接收数据 */
    ... ... ... ... ...
    /* 向客户端发送数据 */
    ... ... ... ... ...
}
```

这段代码所实现的主要功能是，在一个循环里，用 clients 数组不断收集新的客户端套接字，所有的技术细节在第 **3** 和 **7**里皆有阐述。clients 最多能容纳 MAX_CLIENTS 个客户端套接字，这意味着基于上述代码实现的服务端，它最多能同时接受 100 个客户端发起的连接。需要注意的一点是，listener 先被加入到 read_fds，然后经过 select 的挑选，若它还在 read_fds，则意味着可以用它与客户端建立连接。

上述代码的末尾，暂缺「从客户端接收数据」和「向客户端发送数据」的实现，下文予以补全。

8.3 接收数据

在上一节的 while (1) {...} 循环中，若 clients 中的任一客户端属于集合 read_fds，则意味着服务端可从该客户端接收数据，该过程如下：

```
/* 从客户端接收数据 */
#define BUF_SIZE 1024
char buffer[BUF_SIZE];
for (int i = 0; i < MAX_CLIENTS; i++) {
    ind client = clients[i];
    if (client == 0) continue;
    if (FD_ISSET(client, &read_fds)) {
        printf("从客户端 %d 接受数据\n", client);
        ssize_t n = recv(client, buffer, BUF_SIZE, 0);
        if (n > 0) {
            if (n < BUF_SIZE) *(buffer + n) = '\0';
            else *(buffer + BUF_SIZE - 1) = '\0';
            printf("%s\n", buffer);
        } else {
            /* 关闭客户端，并从 clients 中将其移除 */
            close(client);
            clients[i] = 0;
            printf("数据接收失败。关闭客户端 %d\n", client);
        }
    }
}
```

上述代码的数据接收部分，为了追求简明，未考虑安全性。若客户端发送的数据，字节数大于 BUF_SIZE - 1，则服务端收到的数据并不完整。此外，也未考虑处理 **6.5** 节所述的可以挽救的错误，若追求服务端的实用性，这些错误皆需认真考虑。

8.4 发送数据

与接收数据过程相似，服务端只需逐一向 clients 中的套接字发送数据：

```
/* 向客户端发送数据 */
char *msg = "threebody: Hi!";
size_t msg_len = strlen(msg);
for (int i = 0; i < MAX_CLIENTS; i++) {
    ind client = clients[i];
    if (client == 0) continue;
    if (FD_ISSET(client, &read_fds)) {
        printf("向客户端 %d 发送数据\n", client);
        ssize_t n = send(client, msg, msg_len, 0);
        if (n == -1) {
            /* 关闭客户端，并从 clients 中将其移除 */
            close(client);
            clients[i] = 0;
            printf("数据发送失败。关闭客户端 %d\n", client);
        }
    }
}
```

同样为了追求代码的简明，使用 send 发送数据的过程中，未处理可以挽救的 send 错误。

8.5 大胆假设

我们在第 **6** 章中写了一个犹豫迟疑的 ywj，即 other-ywj。它在与 threebody 建立连接后，并未急于向 threebody 发送数据，而是用 sleep(15) 在时间上阻塞了 15 秒。如果新的 threebody 是基于上述代码实现的，对于 other-ywj 建立的连接，它会如何处理呢？

一个大胆的答案是，服务端的 select 将 other-ywj 建立的这个连接对应的套接字提交给操作系统审批时，操作系统会拒绝它通过，除非在 15 秒后，other-ywj 开始向服务端发送数据和接收数据。这就是 select 函数之所以能提高服务端并发能力的根本原因。

结合上文代码，other-ywj 发起的连接，在服务端会表现为一个客户端套接字，假设为 x，它会被存入 clients，但是在服务端每一轮的运行中，select 会将 x 从 read_fds 和 write_fds 中剔除，故而 x 没有机会参与后续的数据接收和发送过程。

至于 ywj，我们也要敢于猜测，它与 threebody 建立连接后，便开始了通信。threebody 中的 select 会将 ywj 对应的套接字保留在 read_fds 和 write_fds 中，从而该套接字可直接用于

threebody 与 ywj 的通信。最终结果是，实现 ywj 无需等待 other-ywj 与 threebody 的通信结束后方能与 threebody 通信。

8.6 小心求证

为了验证上一节我们的猜想，需要基于同步 I/O 多路复用机制构建一个完整的 threebody：

```
/* threebody.c */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <fcntl.h>
#include <sys/select.h>
```

```
#define MAX_CLIENTS 100
int clients[MAX_CLIENTS] = {0};
fd_set read_fds;
fd_set write_fds;
int fd_max;
#define BUF_SIZE 1024
char buffer[BUF_SIZE];
```

```
static int socket_nonblock(int x) {
    return fcntl(x, F_SETFL, fcntl(x, F_GETFL) | O_NONBLOCK);
}
```

```
int main(void) {
    /* 将文字形式的套接字地址数字化 */
    struct addrinfo hints, *addr_list;
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    int a = getaddrinfo("localhost", "8080", &hints, &addr_list);
    if (a != 0) {
        fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(a));
        exit(-1);
    }
```

```c
/* 构造监听套接字 listener */
int listener = -1;
for (struct addrinfo *it = addr_list; it; it = it->ai_next) {
    listener = socket(it->ai_family, it->ai_socktype, it->ai_protocol);
    if (listener == -1) continue;
    int a = bind(listener, it->ai_addr, it->ai_addrlen);
    if (a == -1) {
        close(listener);
        continue;
    }
    break;
}
freeaddrinfo(addr_list);
socket_nonblock(listener); /* 将监听套接字设为非阻塞 */
if (listener == -1) {
    fprintf(stderr, "failed to bind!\n");
    exit(-1);
}
if (listen(listener, 10) == -1) {
    fprintf(stderr, "failed to listen!\n");
    exit(-1);
}
```

```c
while (1) {
    FD_ZERO(&read_fds);
    FD_ZERO(&write_fds);
    FD_SET(listener, &read_fds); /* 将监听套接字加入监视集 */
    fd_max = listener;
    for (int i = 0; i < MAX_CLIENTS; i++) {
        int client = clients[i];
        if (client > 0) {
            FD_SET(client, &read_fds);
            FD_SET(client, &write_fds);
            if (client > fd_max) fd_max = client;
        }
    }
    /* 轮询套接字集合 */
    if (select(fd_max + 1, &read_fds, &write_fds, NULL, NULL) == -1) {
        fprintf(stderr, "select error!\n");
    }
```

```c
/* 建立新连接并将其设为非阻塞*/
int new_client;
if (FD_ISSET(listener, &read_fds)) {
    new_client = accept(listener, NULL, NULL);
    if (new_client == -1) {
        fprintf(stderr, "accetp error!\n");
        continue;
    }
    socket_nonblock(new_client);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i] == 0) {
            clients[i] = new_client;
            printf("连接客户端 %d\n", new_client);
            break;
        }
    }
}
```

```c
/* 从客户端接收数据 */
for (int i = 0; i < MAX_CLIENTS; i++) {
    int client = clients[i];
    if (client == 0) continue;
    if (FD_ISSET(client, &read_fds)) {
        printf("从客户端 %d 接受数据\n", client);
        ssize_t n = recv(client, buffer, BUF_SIZE, 0);
        if (n > 0) {
            if (n < BUF_SIZE) *(buffer + n) = '\0';
            else *(buffer + BUF_SIZE - 1) = '\0';
            printf("%s\n", buffer);
        } else {
            /* 关闭客户端，并从 clients 中将其移除 */
            close(client);
            clients[i] = 0;
            printf("数据接收失败。关闭客户端 %d\n", client);
        }
    }
}
```

```
        /* 向客户端发送数据 */
        char *msg = "threebody: Hi!";
        size_t msg_len = strlen(msg);
        for (int i = 0; i < MAX_CLIENTS; i++) {
            int client = clients[i];
            if (client == 0) continue;
            if (FD_ISSET(client, &read_fds)) {
                printf("向客户端 %d 发送数据\n", client);
                ssize_t n = send(client, msg, msg_len, 0);
                if (n == -1) {
                    /* 关闭客户端，并从 clients 中将其移除 */
                    close(client);
                    clients[i] = 0;
                    printf("数据发送失败。关闭客户端 %d\n", client);
                }
            }
        }
    }
```

```
    close(listener);
    return 0;
}
```

编译 threebody.c：

```
$ gcc threebody.c -o threebody
```

运行 threebody：

```
$ ./threebody
```

然后再打开两个终端窗口，分别用于运行第 **6** 章中的 ywj 和 other-ywj。先运行 other-ywj，它与 threebody 取得连接，threebody 会输出：

```
连接客户端 4
```

运行 other-ywj 后，快速在另一个终端窗口运行 ywj，而后 threebody 会输出

```
连接客户端 5
从客户端 5 接受数据
ywj: I am here!
从客户端 5 接受数据
数据接收失败。关闭客户端 5
```

同时，ywj 也得到了 threebody 的回复：

```
threebody: Hi!
```

而 other-ywj 在运行大约 15 秒后，得到了 threebody 的回复：

```
threebody: Hi!
```

threebody 的输出是

```
从客户端 4 接受数据
other ywj: I am here!
向客户端 4 发送数据
从客户端 4 接受数据
数据接收失败。关闭客户端 4
```

以上试验结果完美的验证了上一节我们大胆的猜想都是正确的！现在先不要急于用 Ctrl + C 组合键关闭 threebody，使用以下命令查看 threebody 的 CPU 占用情况：

```
$ top -p $(pgrep threebody)
```

可得到类似以下结果：

```
   PID ...  %CPU  %MEM   ...  COMMAND
 13291 ...   0.0   0.0   ...  threebody
```

threebody 现在也不再是疯狂吞噬 CPU 的怪物了！现在，可以用 Ctrl + C 关闭 threebody 了。

8.7 你上当了

若认真观察 **8.6** 节服务端向客户端发送数据的代码，应该能发现以下语句是有问题的：

```
if (FD_ISSET(client, &read_fds))
```

由于是向套接字 `cient` 发送数据，我们应该判断 `client` 是否出现于 `write_fds`，而非 `read_fds`。倘若你将上述的条件语句修改为

```
if (FD_ISSET(client, &write_fds))
```

那么上一节的试验便只能成功一半，即 ywj 能很快收到 threebody 的回复，但是 other-ywj 在迟疑的 15 秒的时间里，threebody 会不断给她发送数据。15 秒之后，other-ywj 收到的是不计其数条 `threebody: Hi!`。

实际上，`select` 在将 `read_fds` 和 `write_fds` 提交给操作系统审批时，threebody 用于同 other-ywj 通信的套接字，假设它为 `x`，会被 `select` 从 `read_fds` 中剔除，但它仍旧保留在

write_fds。因为在 select 时，若 other-ywj 尚未给 threebody 发送数据，此时 x 无数据，故而 select 认为它是不可读的，但是此时操作系统为 x 准备的用于写数据的缓冲区也是空的，故而 select 认为它是可写的。

为了保证上一节的试验能够完全成功，最稳妥的办法是，将服务端接收数据和发送数据所用的条件语句都改为

```
if (FD_ISSET(client, &read_fds) && FD_ISSET(client, &write_fds))
```

即套接字必须 client 既可读，又可写，方能用于通信。

8.8 结语

盘桓在我们头上的两朵乌云终于散去。阳光普照服务端，客户端在阳光下自由奔跑。唯一打破这美好意向的是 threebody.c 的代码，步履蹒跚，我们还需要再作一些努力，让它能像第 5 章的 threebody.c 那样清爽，富有生气。

第 9 章 缚住苍龙

9.1 前言

我们基于 select 显著提升了服务端的并发性能。在高兴之余，一定会有更擅长网络编程的朋友告诉我们，select 过时了，poll 更好，最好的是 epoll……他们是正确的，但是现在我们需要的，还不是这种正确。select 的缺陷是，通常情况下最多只能支持 1024 个文件描述符，亦即基于它实现的服务端同时最多只能支持 1024 个客户端。然而，我们的服务端能同时支持这么多个客户端，已经非常了不起了，如同你经营一家超市，每个时间段都能有 1024 个顾客在店内游逛。以后，待我们需要更了不起的时候，再研究 poll 甚至 epoll 也不迟，而现在我们要做的是，将 select 机制融入第 5 章的 Sim 项目。

9.2 单向链表

在第 8 章中，我们用了一个固定长度的数组存储了一组客户端套接字。我们需要实现一个较固定长度的数组更为灵巧的数据结构，将其用于存储客户端套接字。该数据结构应当由一些称为结点的数据单元构成，每个结点可存储 1 个套接字。这种数据结构通常有两种实现，一种是动态数组，一种是链表。我更喜欢链表。

以下代码定义了一个非常简单的单向链表类 SimList 并声明了其析构函数和 add 方法：

```
/* sim-list.h */
#ifndef SIM_LIST_H
#define SIM_LIST_H
#include <stdio.h>
```

```
typedef struct sim_list SimList;
struct sim_list {
    void *data;
    struct sim_list *next;
};
void sim_list_free(SimList *list);
```

```
/* 获取链表长度 */
size_t sim_list_size(SimList *list);
/* 向链表添加一条数据 */
SimList *sim_list_add(SimList *self, void *data);
/* 从链表删除一个单元（结点） */
SimList *sim_list_delete(SimList *self, SimList *node);
#endif
```

我没有为 SimList 对象提供构造函数，原因是我已将 SimList 的定义公布于 sim-list.h 里了，这意味着任何人都可以根据它的定义创建 SimList 对象。

以下代码实现了 SimList 对象的析构函数和所有方法：

```
/* sim-list.c */
#include <stdlib.h>
#include "sim-list.h"
```

```
void sim_list_free(SimList *list) {
    if (list) {
        SimList *it = list;
        while (it) {
            SimList *next = it->next;
            free(it);
            it = next;
        }
    }
}
```

```
size_t sim_list_size(SimList *list) {
    if (list) {
        size_t n = 0;
        for (SimList *it = list; it; it = it->next) n++;
        return n;
    } else return 0;
}
```

```
SimList *sim_list_add(SimList *self, void *data) {
    SimList *new_node = malloc(sizeof(SimList));
    if (new_node) {
        new_node->data = data;
        if (self) new_node->next = self;
        else new_node->next = NULL;
    } else fprintf(stderr, "sim_list_add error!\n");
    return new_node;
}
```

```
SimList *sim_list_delete(SimList *self, SimList *node) {
    if (!node) return self;
```

```
    SimList *prev = NULL;
    for (SimList *it = self; it; it = it->next) {
        if (it == node) break;
        if (it->next == node) {
            prev = it;
            break;
        }
    }
    if (prev) prev->next = node->next;
     else self = node->next;
    free(node);
    return self;
}
```

作为 SimList 类的用法示例，将三个整型变量存储于 SimList 对象：

```
int a = 4, b = 5, c = 6;
SimList *list = NULL;
list = sim_list_add(list, &a);
list = sim_list_add(list, &b);
list = sim_list_add(list, &c);
```

以下代码遍历 list，输出套接字，然后释放 list：

```
for (SimList *it = list; it; it = it->next) {
    int *i = it->data;
    printf("socket %d\n", *i);
}
sim_list_free(list);
```

若上述 a、b、c 皆为局部变量，需要将它们的值存储于 list，只需像下面这样做：

```
int a = 4, b = 5, c = 6;
SimList *list = NULL;
int *p;
p = malloc(sizeof(int));
*p = a;
list = sim_list_add(list, p);
p = malloc(sizeof(int));
*p = b;
list = sim_list_add(list, p);
```

```
p = malloc(sizeof(int));
*p = c;
list = sim_list_add(list, p);
```

要释放用于存储 a、b、c 值的内存，只需在对 list 析构前，遍历 list，逐一释放 list->data：

```
for (SimList *it = list; it; it = it->next) {
    free(it->data);
}
sim_list_free(list);
```

不得不承认，用 SimList 存储套接字，有些笨，但终归可以摆脱固定长度的数组的桎梏了。实际上，我们可以为 SimList 实现一个更方便的 add 的方法，只不过该方法是一个宏：

```
/* sim-list.h ++ */
#define SIM_LIST_ADD(self, value, type) do { \
    type *p = malloc(sizeof(type)); \
    *p = value; \
    self = sim_list_add(self, p); \
} while (0)
```

基于 SIM_LIST_ADD 宏，可将上述将局部变量 a、b、c 的值存储于 list 的过程简化为

```
int a = 4, b = 5, c = 6;
SimList *list = NULL;

SIM_LIST_ADD(list, a, int);
SIM_LIST_ADD(list, b, int);
SIM_LIST_ADD(list, c, int);
```

一定要注意，SIM_LIST_ADD 的第一个参数必须是 SimList * 类型的变量。此外，无论是 sim_list_add 还是 SIM_LIST_ADD，新建的链表单元总是位于链表之首。这个 add 方法，它更准确的名字应该是 prepend。不过，我喜欢 add，这个名字短。

9.3 修改 SimServer 类

SimServer 对象原本只能记录一个客户端套接字，现在我们试图让它具备并发能力，这意味着它需要存储多个客户端套接字，具体数量未知。为此，我们可以用 SimList 对象作为 SimServer 对象存储多个客户端套接字的容器：

```
/* sim-network.h ++ */
#include <sim-list.h>
```

```c
/* sim-network.c ++ */
#include <fcntl.h>
#include <sys/select.h>
```

```c
/* sim-network.c [改] */
struct sim_server {
    int listener;
    SimList *clients;
    fd_set read_fds;
    fd_set write_fds;
    const char *error;
};
```

SimServer 对象也许从未想过有一天自己会如此富有，以至于对自己的构造和析构函数都要更加小心翼翼了。

```c
/* sim-network.c [改] */
static int socket_nonblock(int x);
```

```c
SimServer *sim_server(const char *host, const char *port) {
    int fd = first_valid_address(host, port, bind);
    if (fd == -1) return NULL;
    if (listen(fd, 10) == -1) return NULL;
    socket_nonblock(fd); /* 将 fd 设为非阻塞状态 */
```

```c
    SimServer *server = malloc(sizeof(SimServer));
    if (!server) {
        fprintf(stderr, "sim_server error!\n");
        return NULL;
    }
    server->listener = fd;
    server->clients = NULL;
    server->error = NULL;
    return server;
}
```

```c
static int socket_nonblock(int x) {
    return fcntl(x, F_SETFL, fcntl(x, F_GETFL) | O_NONBLOCK);
}
```

```c
/* sim-network.c [改] */
void sim_server_free(SimServer *server) {
    if (server) {
        close(server->listener);
        if (server->clients) {
            for (SimList *it = server->clients; it; it = it->next) {
                if (it->data) free(it->data);
            }
            sim_list_free(server->clients);
        }
        free(server);
    }
}
```

需要注意的是，sim_server 函数中，将用于监听的套接字设成了非阻塞状态，这是实现服务端对 I/O 多路复用机制的支持所必须的。

9.4 修改 sim_server_run_once

将基于 select 的 I/O 多路复用机制纳入 sim_server_run_once 函数：

```c
/* sim-network.c [改] */
void sim_server_run_once(SimServer *self) {
    int fd_max;
    /* 尽量让 select 成功运行 */
    while (1) {
        FD_ZERO(&self->read_fds);
        FD_ZERO(&self->write_fds);
        FD_SET(self->listener, &self->read_fds);
        fd_max = self->listener;
        for (SimList *it = self->clients; it; it = it->next) {
            int client = *(int *)(it->data);
            FD_SET(client, &self->read_fds);
            FD_SET(client, &self->write_fds);
            if (fd_max < client) fd_max = client;
        }
```

```c
        if (select(fd_max + 1, &self->read_fds,
                   &self->write_fds, NULL, NULL) == -1) {
            if (errno == EINTR) continue;
```

```
            else {
                self->error = "sim_server_run_once error!";
                break;
            }
        } else break;
    }
```

```
    /* 接纳新的连接 */
    if (FD_ISSET(self->listener, &self->read_fds)) {
         /* 尽量让 accept 运行成功 */
        while (1) {
            int fd = accept(self->listener, NULL, NULL);
            if (fd == -1) {
                if (errno == EINTR) continue;
                else {
                    self->error = "sim_server_run_once error!";
                    break;
                }
            } else {
                /* 将 fd 设为非阻塞状态 */
                socket_nonblock(fd);
                /* 将 fd 加入 self->clients */
                SIM_LIST_ADD(self->clients, fd, int);
                /* 恢复 self 无错状态 */
                if (self->error) self->error = NULL;
                break;
            }
        }
    }
}
```

上述代码中，两处 while (1) 循环只是为了尽量保证 select 和 accept 不会受系统中断信号的干扰而出错，详见 **6.5** 节。在通常情况下，这两处 while (1) 的循环体只运行一次便可退出，不会无限循环。

正常情况下，每运行一次 sim_server_run_once，SimServer 对象的 clients 成员便可纳入一个新的客户端套接字。在下一次运行 sim_server_run_once 时，该套接字会被 select 提交给系统审批。若审批通过，该套接字会被保留在 read_fds 和 write_fds 中。

9.5 修改 sim_server_receive

sim_server_receive 依然基于 **6.5** 节的 recv_robustly 从客户端套接字接收数据，只是现在需要遍历 SimServer 对象的 clients 中的每个客户端套接字，若该套接字在 SimServer 的 read_fds 中，则通过它接收数据。此外，由于可能有多个客户端套接字皆可读，从每个套接字读取的数据保存在一个 SimStr 对象里，故而我们需要用一个 SimList 对象存储这些 SimStr 对象。

```
/* sim-network.h [改] */
SimList *sim_server_receive(SimServer *self);
```

```
/* sim-network.c [改] */
SimList *sim_server_receive(SimServer *self) {
    SimList *msgs = NULL;
    SimList *it = self->clients;
    while (it) {
        int client = *(int *)(it->data);
        if (FD_ISSET(client, &self->read_fds)
            && FD_ISSET(client, &self->write_fds)) {
            SimStr *msg = recv_robustly(client);
            if (msg) {
                msgs = sim_list_add(msgs, msg);
            } else {
                /* 客户端套接字无法读取，可能对端已经关闭了连接 */
                /* 我们需要从 self->clients 移除 it，并关闭 client */
                SimList *next = it->next;
                free(it->data);
                self->clients = sim_list_delete(self->clients, it);
                it = next;
                close(client);
                continue;
            }
        }
        it = it->next;
    }
    return msgs;
}
```

9.6 修改 sim_server_send

sim_server_send 需要遍历 SimServer 对象的 clients 中的每个客户端套接字，若该套接字在 SimServer 的 write_fds 中，则通过它向客户端发送数据。

```
/* sim-network.c [改] */
void sim_server_send(SimServer *self, SimStr *msg) {
    SimList *it = self->clients;
    while (it) {
        int client = *(int *)(it->data);
        if (FD_ISSET(client, &self->read_fds)
            && FD_ISSET(client, &self->write_fds)) {
            if (send_robustly(client, msg) == -1) {
                /* 客户端套接字无法写入数据，可能对端已经关闭了连接 */
                /* 我们需要从 self->clients 移除 it，并关闭 client */
                SimList *next = it->next;
                free(it->data);
                self->clients = sim_list_delete(self->clients, it);
                it = next;
                close(client);
                continue;
            }
        }
        it = it->next;
    }
}
```

9.7 threebody 并发

现在，可以重写 threebody.c，欺骗 ywj 和 other-ywj，让她们都以为自己得到了 threebody 的眷顾，前者很快得到了 threebody 的回复，后者以为 threebody 颇有耐心，容许她的迟疑……

```
/* threebody.c */
#include "sim-network.h"
int main(void) {
    SimServer *threebody = sim_server("localhost", "8080");
    if (!threebody) {
        fprintf(stderr, "sim_server failed!\n");
        exit(-1);
    }
```

```c
    /* 服务端程序运转 */
    while (1) {
        sim_server_run_once(threebody);
        if (sim_server_safe(threebody)) {
            /* 从客户端接收信息 */
            SimList *msgs = sim_server_receive(threebody);
            if (msgs) {
                for (SimList *it = msgs; it; it = it->next) {
                    SimStr *msg = it->data;
                    printf("%s\n", sim_str_raw(msg));
                    sim_str_free(msg);
                }
                sim_list_free(msgs);
            }
            /* 向客户端发送信息 */
            SimStr *msg = sim_str("threebody: Hi");
            sim_server_send(threebody, msg);
            sim_str_free(msg);
        }
    }
    sim_server_free(threebody);
    return 0;
}
```

9.8 总结

服务端使用单向链表管理接入的客户端，是有一些弊端。例如获取链表长度，要遍历所有结点。删除链表中的某个结点，也需要遍历链表，找到该结点的前一个结点，方能完成删除过程。不过，select 最多只能支持 1024 个客户端并发访问，单向链表的性能并非不可接受，且胜在简单。下面是一些你可以不劳而获的成果：

```
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/sim-str.h
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/sim-str.c
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/sim-list.h
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/sim-list.c
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/sim-network.h
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/sim-network.c
https://liyanrui.github.io/2025/c-network-programming/wrapper-again/threebody.c
```

第 10 章　世界是你的，也是我的

10.1　前言

进程，也许是绝大多数人既熟悉又陌生的事物。

若一个人很自信的说，万物皆备于我……他便是一个进程无疑了。在操作系统中运行的任何一个进程，它都会认为，整个系统资源，它可以随意使用，例如，内存可以想要多少，就能拥有多少……完全可以说，孟老夫子在 2000 多年前已经提前拥有共产主义意识了。

矛盾之处在于，任何一个程序，它的设计者都不会让它浪费资源，而程序在操作系统中运行起来，亦即变成进程时，它却觉得自己拥有整个了世界。如果大多数人都没有这样的觉悟，我对人类能否实现共产主义表示悲观。

10.2　电子穿过双缝

经常翻阅量子力学科普书的人应该都知道，量子力学最有名的实验，是电子双缝实验。一组电子仿佛同时穿过两个缝隙，在缝隙后面的屏幕上形成干涉图像，一旦观测它们，便会导致它们立刻选择某条路径，于是干涉图像消失。

Unix 或 Linux 操作系统也为进程提供了颇为相像的机制，即 `fork` 函数：

```
#include <unistd.h>
pid_t fork(void);
```

`fork` 能够为进程制造一个类似双缝装置的程序逻辑，因为它不像普通的 C 函数那样，每次调用，只能返回一次，`fork` 可以同时返回两次。然后，进程会像电子穿过双缝那样穿过这两次返回所对应的程序逻辑。例如

```
/* foo.c */
#include <unistd.h>
#include <stdio.h>
int main(void) {
    pid_t a = fork();
    if (a == 0) {
        printf("I am %d.\n", getpid());
    } else {
        printf("I am %d.\n", getpid());
    }
    return 0;
}
```

`getpid` 函数用于获取当前进程的 ID，它与 `fork` 函数皆为系统调用，且二者的返回值类型皆为 `pid_t`，这是整型类型的别名，专用于表达进程 ID，当然也有 0 和 `-1` 这种含的特殊意义的值。

编译上述的 foo.c 并执行所得程序：

```
$ gcc foo.c -o foo
$ ./foo
I am 33989.
I am 33990.
```

亦即上述代码中的 `if ... else ...` 的两个条件分支都被执行了。

倘若你的思维之前一直停留在经典模式，但愿此刻你不会开始怀疑这个世界是否真实了。费曼说，没人能理解量子力学。我们不妨同意这个观点，不过，我们应该能理解同时通过 `fork`「双缝」的进程，这个机制毕竟是上个世纪下半叶的人类创造的。

现在，将上述 foo.c 略微修改一处，只需关注 `main` 函数：

```
int main(void) {
    pid_t a = fork();
    if (a == 0) {
        printf("I am %d.\n", getpid());
    } else {
        printf("I am %d.\n", a);
    }
    return 0;
}
```

`else` 分支里打印的不在是 `getpid()` 的返回值，而是 `fork` 的两次返回的值之一，因为这两次返回是同时的，因此我无法使用「第一次返回的值或第二次返回的值」之类的说法。

重新编译 foo.c 并运行所得程序，则程序里的两次 `printf` 打印的内容是相同的，例如

```
I am 41590.
I am 41590.
```

这说明，对于 `else` 分支里的那个进程而言，它持有 `if` 分支里的进程的 ID。

对 foo.c 再略作修改：

```
int main(void) {
    printf("I am %d.\n", getpid());
    pid_t a = fork();
    if (a == 0) {
        printf("(%d, %d)\n", a, getpid());
    } else {
        printf("(%d, %d)\n", a, getpid());
    }
    return 0;
}
```

再度编译 foo.c，然后运行所得程序，结果如下：

```
I am 41851.
(41852, 41851)
(0, 41852)
```

现在能看出一些端倪了么？main 函数所在的进程，ID 为 41851，在调用 fork 之后，在 else 分支里的进程恰好是 main 函数所在的进程，而 if 分支里的进程是一个新的进程。

现在，可以下一个结论：在 fork 的返回值为 0 的情况里，会出现一个新的进程，而原有的进程——main 函数所在的进程继续在 fork 返回值为非 0 的情况中存在，并且它能够持有那个新进程的 ID。若不理解这个结论，需要你再认真回顾一下上述对 foo.c 的两次修改及程序运行结果。原有的进程，习惯上称为父进程，而新的进程称为子进程。

实际上，有的时候，fork 会返回 -1，表示它执行时出错。这种情况，可能性虽然很小，但理论上是存在的，因此健壮的多进程并发程序，需严谨对待。例如

```c
int a = fork();
if (a == 0) { /* 子进程 */
    ... ... ...
} else { /* 父进程 */
    if (a == -1) {
        fprintf(stderr, "fork error!\n");
        exit(-1);
    } else {
        ... ... ...
    }
}
```

考考你，你知道下面这段代码（假如 fork 的执行都是成功的）的结果是什么吗？

```c
int main(void) {
    fork();
    fork();
    printf("I am %d.\n", getpid());
    return 0;
}
```

建议动手试验，若结果不符合你的答案，你需要再认真理解关于 fork 同时返回两次这一事实。

10.3 麦田守望者

物理学家可能至今依然不明白一个电子如何同时穿过双缝，而我们却已经很清楚一个进程如何穿过 fork 返回的两个条件分支，父进程将「自身」复制为一个新的子进程，二者同时穿过各自的条件分支，并相互干涉，只是这种干涉，进程毫无所知，它们只是以为自己在独立地运行着。若

不理解操作系统的进程调度原理，人类也以为通过 fork 产生的条件分支的两个进程不会干涉。事实上，并发的进程可能存在干涉，甚至能创造出薛定谔的猫。

在操作系统看来，父进程及其 fork 出来的子进程，有很多人会按照一些过时的教科书所说的，操作系统会将父进程的资源完整复制一份交给子进程。倘若如此，那父进程和子进程在资源方面是完全独立的，也就不发生任何干涉，但实际上现代操作系统采用的是 Copy-on-Write 机制。COW 机制就像两个人同时阅读同一本书，其中有一人想在某页上做笔记，操作系统会为他单独复制这一页供他使用。

父进程和子进程的各自独立运行实际上往往也是假象，因为操作系统在调度它们的时候，可能会让它们运行在单个 CPU 核心上。在这种情况下，操作系统会运行一会父进程，然后将其挂起，再运行一会子进程，然后再将子进程挂起，恢复父进程的运行，如此循环交替，直至它们终止。

还有一个令人伤脑筋的事实是，若子进程结束，父进程尚在运行，则对于操作系统而言，子进程会处于即死又生的状态，所以你可以联想到薛定谔的猫了。这样的子进程，叫僵尸进程。名字有些吓人，但实际上它已经死了，只是还占用着资源，即尸位素餐。制作僵尸进程很容易，例如

```
#include <unistd.h>
int main(void) {
    if (fork() == 0) {  /* 创建子进程 */
        return 0;       /* 子进程立即退出，成为僵尸 */
    }
    while (1) {         /* 父进程仍在一直运行 */
        sleep(1);
    }
    return 0;
}
```

假设上述源码编译成的程序为 foo，运行 foo 之后，可通过以下命令查看是 foo 的子进程是否为僵尸进程：

```
$ ps aux | grep foo
```

在我的机器上，上述命令给出的结果为

```
...    9232  ...   S+   ... ./foo
...    9233  ...   Z+   ... [foo] <defunct>
```

其中含有 Z+ 和 defunct 的进程，便是僵尸进程。

要避免子进程成为僵尸进程，需要在父进程中使用 wait 函数，让父进程先不要做别的事情，静静等待子进程的结束。wait 也是一个系统调用，用法并不难。倘若你不想追究细节，只需在父进程中添加

```
wait(NULL);
```

例如

```c
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    if (fork() == 0) {
        return 0;
    }
    wait(NULL); /* 守望 */
    while (1) {
            sleep(1);
    }
    return 0;
}
```

将上述源码再次编译为程序 foo 并运行，再使用 `ps | grep foo` 命令查看进程状态，便看不到僵尸态的子进程了，因为它被父进程回收了。

不妨将 `fork` 简单理解为 `malloc`，将 `wait` 简单理解为 `free`。构建新的进程，像是构建一块有生命的内存空间。大多数情况下，父进程不 `wait` 子进程也无妨，除非它是网络服务端之类的程序，需要保持长期运行。对于网络服务端而言，若子进程开启过多且不回收，系统资源可能很快会被消耗殆尽，从而导致父进程崩溃。

父进程等着子进程先结束，这件事会让人觉得不太舒适，试问世上可有白发人想送黑发人么？其实，可以浪漫一些。我的父亲虽然不在了，这事的确发生了，可他依然在守望着我。一代又一代，构成绵延不绝的守望链，直至家族绝后。

有的时候，父进程先于子进程结束，原因也是它没有守望子进程。例如

```c
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    if (fork() == 0) {
        sleep(3); /* 将子进程阻塞 3 秒 */
        return 0; /* 子进程退出 */
    }
    return 0; /* 父进程退出 */
}
```

这更符合现实的情况，即黑发人送白发人，只是黑发人会变成孤儿，以天为父，以地为母。失去父进程的子进程会被操作系统中的 `init` 进程领养。`init` 进程是操作系统启动后运行的第 1 个进程，亦称 1 号进程，如天如地。

应该没人想当孤儿。父母们，责任在肩，还是努力多给子女一些守望吧。

10.4 守望的艺术

wait 的声明如下：

```
#include <sys/wait.h>
pid_t wait(int *status);
```

wait 执行成功，它会返回结束的子进程 ID，并将子进程的退出状态保存在 status 指向的对象，否则返回 -1。有一组宏，可以破译各种可能的子进程退出状态。

- 若子进程正常退出，WIFEXITED(status) 的结果为非 0；
- WEXITSTATUS(status) 返回子进程的退出状态——子进程返回的值；
- 若子进程因其他进程发送给它的信号而引发终止，WIFSIGNALED(status) 返回真值；
- WTERMSIG(status) 返回引起子进程退出的信号。

现在还不需要了解操作系统为进程提供的信号处理机制，故而上述的 4 个宏，我们暂且只需要关心前两个即可，以下示例程序演示了它们的基本用法：

```
/* foo.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main(void) {
    int a = fork();
    if (a == 0) {  /* 创建子进程 */
        printf("子进程: 我是 %d! \n", getpid());
        sleep(7);
        return 0;
    }
```

```
    else { /* 父进程 */
        if (a == -1) {
            fprintf(stderr, "fork error!\n");
            exit(-1);
        }
        int status;
        pid_t child_id = wait(&status);
        if (child_id == -1) {
            fprintf(stderr, "wait error!\n");
            exit(-1);
        }
```

```
        if (WIFEXITED(status)) {
            printf("父进程: %d 正常退出了, "
                   "其退出状态为 %d\n",
                   child_id, WEXITSTATUS(status));
        } else {
            printf("父进程: %d 的退出存在异常! \n", child_id);
        }
    }
    return 0;
}
```

上述代码的运行结果如下：

```
子进程: 我是 17119, 我退出了!
父进程: 17119 正常退出了, 其退出状态为 0。
```

需要注意，上述代码中的父进程要等待子进程大概 7 秒，方能输出内容，而子进程的输出则是在程序甫运行时便出现了，它给出了子进程的 ID。若在父进程等待期间，快速在另一个终端用 kill 命令杀死子进程，例如

```
$ kill -9 17119
```

则父进程会立刻输出：

```
父进程: 17175 的退出存在异常!
```

kill -9 命令是 kill 进程通过 ID 向对应的进程发送 SIGKILL 信号，然后导致操作系统会强行杀死收到该信号的进程。有些信号，进程可以捕捉并作出响应，但它们对 SIGKILL 却无能为力，这个信号是进程无法违抗的死亡通知。

10.5 非阻塞等待

wait 是阻塞式的，它会让父进程挂起，无法执行后续指令，直到子进程退出。若只有父进程只有一个子进程，为了回收该子进程，父进程阻塞等待，这种做法实际上是没有太大意义的，因为在实际效用上，它与单进程的程序并无区别。

使用比 wait 更为基础的 waitpid 函数，能够实现父进程非阻塞等待子进程退出。waitpid 的声明如下：

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
```

该函数比 wait 多出两个参数，pid 和 options，前者用于指定等待某个子进程，若为 -1，表示等待所有子进程，后者用于设定等待方式，若为 WNOHANG，表示非阻塞等待。

父进程用 waitpid 回收指定的子进程，若子进程并未退出，则 waitpid 会返回 0，而不像 wait 那样将父进程挂起。若 waitpid 返回 -1，表示执行失败。

waitpid 的非阻塞特性，意味着我们需要在一个循环中不断使用 waitpid 尝试回收子进程，直至 waitpid 返回子进程的 ID，而不是 0 或 -1。例如

```c
/* foo.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int a = fork();
    if (a == 0) {  /* 创建子进程 */
        printf("子进程: 我是 %d! \n", getpid());
        sleep(5);
        return 0;
    } else {
        if (a == -1) {
            fprintf(stderr, "fork error!\n");
            exit(-1);
        }
        int once = 1;
        while (1) {
            int status;
            pid_t child_id = waitpid(-1, &status, WNOHANG);
            if (child_id > 0) {
                printf("父进程: %d 退出了! \n", child_id);
                break;
            } else if (child_id == 0) {
                if (once) printf("父进程: 等待子进程退出。\n");
            } else {
                fprintf(stderr, "waitpid error!\n");
                exit(-1);
            }
            once = 0;
            sleep(1); /* 降低 cpu 占用 */
        }
    }
    return 0;
}
```

上述代码，我的试验结果如下：

```
父进程：等待子进程退出。
子进程：我是 8191!
父进程：8191 退出了!
```

10.6 信号处理

以轮询的方式回收已结束的子进程，虽然能让父进程非阻塞运行，但是在非阻塞等待期间，父进程要执行的操作只能在轮循过程中精心部署，例如上一节示例程序中，为了避免父进程重复说「等待子进程退出」，我使用了变量 once 控制父进程在非阻塞等待期的行为不会被轮询过程重复，虽然有效，但使得程序逻辑更为复杂。实际上，子进程在退出时，会向父进程发送一个信号 SIGCHLD。若父进程主动忽略该信号，或对该信号作出一些响应，则退出的子进程不会变成僵尸进程。

让父进程忽略子进程的退出信号，非常简单，只需在 main 函数的开始，或者所有的调用 fork 函数的语句之前插入以下语句：

```
signal(SIGCHLD, SIG_IGN)
```

SIG_IGN 是 SIGNAL IGNORE 的缩写。很多操作系统级别的信号，皆可凭此法忽略。不过，多数时候，我们需要回收子进程占用的一些资源，需要主动对子进程的退出信号作出响应。

最简单的响应机制是将一个用于处理信号的函数作为参数传递给 signal。例如

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
```

```
void handler(int signal) {
    int pid = wait(NULL);
    printf("父进程：%d 退出了！\n", pid);
}
```

```
int main(void) {
    signal(SIGCHLD, handler);
    int a = fork();
    if (a == 0) {  /* 创建子进程 */
        printf("子进程：我是 %d! \n", getpid());
        sleep(5);
        return 0;
    }
```

```
    /* 让父进程阻塞，模拟长时间运行 */
    while (1) sleep(1);
    return 0;
}
```

上面示例，在 fork 出子进程后，父进程进入主动阻塞模式，相当于它在执行很多其他任务，同时等待子进程退出信号。我的试验结果如下：

```
子进程：我是 19221!
父进程：19221 退出了!
```

试验结果出现后，可使用 Ctrl + C 关闭父进程。这个组合键可驱动操作系统向终端前台正在运行的进程发送 SIGINT 信号，若进程未对该信号作出响应，操作系统会终止它。

10.7 更安全的信号处理

历史原因，即使符合 POSIX 标准的操作系统所实现的 signal 也不尽相同，此外，当父进程存在较多子进程时，signal 可能会存在丢失信号的情况，即当它正在响应某个信号的同时，又有不断有新的信号发送过来，而 signal 不能对信号进行排队，故而新的信号可能会丢失。在生产环境中，更推荐使用 sigaction 函数，当然它的用法比 signal 略复杂。

使用 sigaction 函数，首先需要准备一个结构体，例如

```
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART|SA_NOCLDSTOP;
```

struct sigaction 结构体实际上有 5 个成员变量，上面只设定了其中 3 个。sa.handler 的值是上一节定义的信号处理函数。sigemptyset(&sa.sa_mask) 可将信号掩码清空，表示在信号处理期间不阻塞任何信号。倘若需要在信号处理期间阻塞某些信号，可以通过 sigaddset 函数增加信号掩码，不过，目前我们用不到这些高级知识。sa.sa_flags 是一组标志，用于精确控制 sigaction 的行为，其中 SA_RESTART 的作用是自动重启被信号中断的系统调用，SA_NOCLDSTOP 的作用是忽略子进程暂停或恢复时产生的 SIGCHLD 信号。

对于处理 SIGCHLD 信号而言，上述对 sa 的设定已经足够，若不甚理解，完全可不求甚解，暂且照抄，如同学习模拟电路的路数，贵在实践，而非理论。下面只需简单调用 sigaction 函数，即可完成对 SIGCHLD 的安全处理：

```
int a = sigaction(SIGCHLD, &sa, NULL);
if (a == -1) {
        fprintf(stderr, "sigaction error!\n");
        exit(-1);
}
```

sigaction 第三个参数，也是一个结构体，类型与 sa 相同，用于返回进程对 SIGCHLD 信号的原有配置，若不关心，设为 NULL 即可。若 sigaction 执行成功，返回 0，否则返回 -1。

基于上述知识，对上一节的示例进行修改：

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
```

```
void handler(int signal) {
    int pid = wait(NULL);
    printf("父进程: %d 退出了！\n", pid);
}
```

```
int main(void) {
    /* SIGCHLD 信号处理 */
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART|SA_NOCLDSTOP;
    int a = sigaction(SIGCHLD, &sa, NULL);
    if (a == -1) {
        fprintf(stderr, "sigaction error!\n");
        exit(-1);
    }
```

```
    /* 开启子进程 */
    int a = fork();
    if (a == 0) {  /* 创建子进程 */
        printf("子进程: 我是 %d! \n", getpid());
        sleep(5);
        return 0;
    }
    /* 让父进程阻塞，模拟长时间运行 */
    while (1) sleep(1);
    return 0;
}
```

10.8 坍缩

现在，请将历史回退到第 **5** 章，我们曾经的田园时代！基于那段历史中的字符串类和网络类，再结合多进程机制，在套接字函数皆为阻塞的情景中为 threebody 赋予并发能力。

```c
/* threebody.c */
#include <signal.h>
#include <sys/wait.h>
#include <unistd.h>
#include "sim-network.h"
void handler(int signal) { wait(NULL); }
```

```c
int main(void) {
    /* SIGCHLD 信号处理 */
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART|SA_NOCLDSTOP;
    int a = sigaction(SIGCHLD, &sa, NULL);
    if (a == -1) {
        fprintf(stderr, "sigaction error!\n");
        exit(-1);
    }
    /* 服务端 */
    SimServer *threebody = sim_server("localhost", "8080");
    while (1) {
        sim_server_run_once(threebody);
        if (fork() == 0) {
             /* 子进程不需要监听套接字 */
            sim_server_close_listener(threebody);
            /* 从客户端接收数据 */
            SimStr *msg_from = sim_server_receive(threebody);
            if (sim_str_safe(msg_from)) {
                printf("%s\n", sim_str_raw(msg_from));
            }
            sim_str_free(msg_from);
```

```c
            /* 向客户端发送信息 */
            SimStr *msg_to = sim_str("threebody: Hi!");
            sim_server_send(threebody, msg_to);
            sim_str_free(msg_to);
            sim_server_close_client(threebody);
            exit(0);
        }
```

```
        /* 父进程不需要客户端套接字 */
        sim_server_close_client(threebody);
    }
    sim_server_free(threebody);
    return 0;
}
```

上述所有的代码，都应该很熟悉的，因为我已经用尽了 200

```
/* sim-network.h ++ */
void sim_server_close_listener(SimServer *self);
```

```
/* sim-network.c ++ */
void sim_server_close_listener(SimServer *self) {
    if(self) {
        close(self->listener);
        self->listener = -1;
    } else {
        fprintf(stderr, "sim_server_close error: NULL pointer!\n");
    }
}
```

置于 close_client 方法，实际上已经实现了，即 close 方法。现在我们需要对该方法进行更名，将 sim_server_close 更名为 sim_server_close_client：

```
/* sim-network.h [改] */
void sim_server_close_close(SimServer *self);
```

```
/* sim-network.c [改] */
void sim_server_close_client(SimServer *self) { close(self->client); }
```

基于修改后的网络类，编译上述 threebody.c 并运行所得 threebody：

```
$ gcc -I. sim-str.c sim-network.c threebody.c -o threebody
$ ./threebody
```

然后打开两个终端，分别用于运行第 **6** 章的 other-ywj 和 ywj。先在一个终端里运行 other-ywj，它与 threebody 建立连接后，会主动延迟 15 秒，然后才开始与 threebody 通信。随即在另一个终端迅速运行 ywj。

上述试验所得结果与第 **8** 章的试验结果相似。ywj 无需等到 other-ywj 得到 threebody 的回复后放能与 threebody 通信。基于多进程机制为服务端实现的并发，在拥现代普遍拥有多颗 CPU 核心的计算机上，性能远胜基于 I/O 多路复用的并发。即使计算机只有单颗 CPU 核心，多进程

实现的并发，在代码的简洁性上也远胜 I/O 多路复用实现的并发。在处理与「同时」有关的问题上，量子力学胜过广义相对论，早已屡经实验验证过的。

10.9 总结

进程以复制自身的方式同时穿过 `fork` 构造的双缝，演化出父进程和子进程的叠加态。

在父进程中，我们可以使用 `wait` 或 `waitpid` 终结这个分身术，迫使这个系统坍缩，从而塑造了子进程的历史。若父进程能够响应 `SIGCHLD` 信号，则系统坍缩的方式几乎天衣无缝，将系统的不确定性保留到了最后一刻，具有电子双缝延迟选择实验一样的魅力，即电子同时通过双缝之后再去观测它，依然会导致它发生坍缩。

父进程始终存在，它不断开启子进程，又不断回收子进程，最终它拥有了许多子进程的历史。也许我们看不到的那一个又一个电子，它们可能承载着自宇宙大爆炸以来它们经过的全部历史。

人类以为是他们的观测导致了电子叠加态的坍缩，他们可能过于多情了。观测自己的分身，创造一段历史，也许原本便是电子与生俱来的能力。之所以电子双缝会发生干涉效应，仅仅是因为电子觉得它还没必要让自己的分身变成历史。

第 11 章 弦动

11.1 前言

我用了爱因斯坦的广义相对论类比同步 I/O 多路复用机制，用了量子力学类比多进程机制。我该用什么类比计算机中的线程呢？不妨用超弦理论，它们至少都是「线」状的。

每个进程都包含着一个主线程。在 C 程序中，主线程是每个进程中的 `main` 函数。在主线程中，可创建一些子线程，亦即主线程也具备分身术，这个过程可类比为，孙悟空从头上拔下一些毫毛，在口中嚼碎，喷出去，变出诸多化身。这些化身的法力弱，胜在数量庞大。至于多进程机制的类比，在《西游记》里也是有的，即孙悟空和六耳猕猴的关系。

线程是程序运行时操作系统能够予以调度的最小单元。如同超弦理论中，弦以某种模式振动而表现为某种基本粒子，例如电子，光子，夸克……我们不妨认为，线程的运行决定了进程的功能。当多个线程并行运行时，它像一根弦的振动效应。这种类比自然是不严谨的，但是在单个进程中通过多个线程实现的并行运算，它象征着同步 I/O 多路复用机制与多进程机制的融合，犹如超弦理论那般自信地认为自己统一了广义相对论和量子力学，从而跻身为万物理论。

这些类比并不严谨，但它们可以作为一路走过来的我们亲手做的路标。有了它们，我们能够很快找到回去的路，这意味着能够更安心地探索未知空间，更何况这些物理学概念本身就非常有趣。当你发现，精准把握主线程和子线程的调度原理——像理解弦的振动效应，也许便获得了一种奇异的能力，通过它便能够聆听计算机中每个程序弹奏出的乐章。

11.2 叮咚

在 Unix 或 Linux 系统中，可使用 `pthread_create` 函数创建子线程，该函数的形式为

```
#include <pthread.h>
int pthread_create(pthread_t *restrict thread,
                   const pthread_attr_t *restrict attr,
                   void *(*start_routine)(void *),
                   void *restrict arg);
```

`pthread_create` 的返回值为 0 表示成功，否则表示失败。该函数本该为新创建的子线程分配一个结构不透明的数据类型的实例表征线程对象，但返回值专用于表示执行过程是否成功，故而只能在参数列表中以结构体指针的方式传出，即第一个参数为该函数返回的子线程对象。第二个参数是线程属性，初学时无需关心，设为 `NULL` 即可。第三个参数是线程的某一例程——某个具体实现，而第四个参数是传递于该实例的参数。

以下程序创建了一个只会「咚」一声的子线程，主线程负责「叮」：

```
/* foo.c */
#include <pthread.h>
#include <stdio.h>
```

```c
void *foo(void *arg) {
    printf("咚！\n");
    return NULL;
}
```

```c
int main(void) {
    printf("叮！\n");
    pthread_t thread;
    /* 为了代码简单，以下函数未处理可能出现的错误 */
    pthread_create(&thread, NULL, foo, NULL);
    pthread_join(thread, NULL);
    return 0;
}
```

与多进程机制中使用 wait 回收子进程相似，创建的子线程，也需要回收，所用函数为 pthread_join，其第一个参数为线程对象，第二个参数是线程例程的返回值，它是个双重指针——我想，你应该已经熟悉了这种在 C 语言编程中常用的手段，即以双重指针的形式传出指针。由于上述线程例程 foo 的返回值为 NULL，意味着 phthread_join 无需关心其返回值，故而将其第 2 个参数设为 NULL。pthread_join 返回 0 表示成功，否则表示失败。

Linux 内核层面未对线程提供直接支持，而是通过外部库 pthread（POSIX Threads）予以支持，故而编译上述代码，需要连接该库：

```
$ gcc foo.c -o foo -lpthread
```

不过，gcc 提供了比 -lpthread 更全面的选项 -pthread，大多数情况下，我们用后者。另外，在你的 Linux 系统里，gcc 会默认连接 pthread 库，可以不必明确给出 pthread 库的连接选项，不过我是在写教程，只能不厌其烦。

运行所得程序 foo，结果可能并不会让你觉得惊奇：

```
$ ./foo
叮！
咚！
```

11.3 历史

我更喜欢将线程例程的返回值视为历史，就像弹奏的琴音会对应到乐谱上的某个符号——这个符号表征着琴音的某个频率。下面的程序构造了一个有历史的线程：

```c
/* foo.c */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

```c
void *foo(void *arg) {
    printf("咚! \n");

    int *a = malloc(sizeof(int));
    *a = 42;
    return a;
}
```

```c
int main(void) {
    printf("叮! \n");
    pthread_t foo_t;
    if (pthread_create(&foo_t, NULL, foo, NULL) == 0) {
        int *a = NULL;
        if (pthread_join(foo_t, (void **)&a) == 0) {
            printf("history: %d\n", *a);
            free(a);
        }
    }
    return 0;
}
```

这次在创建和回收子线程的过程中，有了一些防御性处理。可以看到，在主线程中回收子线程时，可以获得子线程的返回值。程序的输出为：

```
叮!
咚!
history: 42
```

子线程也能修改父线程的某个变量，相当于父进程的某段历史被子进程改写。例如

```c
#include <pthread.h>
#include <stdio.h>
```

```c
void *foo(void *arg) {
    int *a = arg;
    printf("咚! \n");
    *a += 1;
    return NULL;
}
```

```c
int main(void) {
    printf("叮! \n");
```

```
    pthread_t foo_t;
    int n = 42;
    if (pthread_create(&foo_t, NULL, foo, &n) == 0) {
        if (pthread_join(foo_t, NULL) == 0) {
            printf("history: %d\n", n);
        }
    }
    return 0;
}
```

程序的输出为：

```
叮!
咚!
history: 43
```

11.4 开始混乱

你可能会觉得，上文的几个程序，它们即使直接调用 foo 函数，输出的结果也是一样，何必用 pthread_create 呢？我希望你有这个疑问，否则接下来你不会享受不到感受惊奇的一刻了。看以下程序：

```
#include <pthread.h>
#include <stdio.h>
void *foo(void *arg) {
    printf("叮! \n");
    return NULL;
}
void *bar(void *arg) {
    printf("咚! \n");
    return NULL;
}
int main(void) {
    pthread_t foo_t, bar_t;
    pthread_create(&foo_t, NULL, foo, NULL);
    pthread_create(&bar_t, NULL, bar, NULL);
    pthread_join(bar_t, NULL);
    pthread_join(foo_t, NULL);
    return 0;
}
```

上述程序，编译后，若多次运行，你可能经常会得到

```
叮!
咚!
```

偶尔会得到

```
咚!
叮!
```

这意味着什么？你已无法确定程序的输出是什么了！

请记住这句话，所有的不确定性，它更深刻的内涵可能是，宇宙的运行原理依赖的是某种并发机制，而不确定性不过是我们对该机制的最为直接的感受。你扔出了一枚硬币，它出现正面的概率是 1/2，此事亦可理解为，是因你存在的概率为 1/2 而导致了这一结果。

你习惯了世界是确定的，应该会觉得我所说的，接近癫狂，但是也许真的存在不确定性的转移呢？我认为，同时通过双缝的电子，它之所以会在观测中坍缩，原因是电子将自身的不确定性转移到了观测设备上了。

无论你是否同意我的比喻，至少你需要承认，现在的进程，即使它是单进程，但是在多线程的作用下，它已经呈现出不确定性了。在超弦理论中，基本粒子，是一根弦在一个蜷缩在时空中的高维空间中的振动形式。依照这个比喻，我们可以认为，上述程序是主线程在一个「三维空间」中的振动形式，程序的不确定性因此而出现。超弦理论通过在数学上构造了一个蜷缩在时空中的六维空间作为弦的振动空间，从而兼容了量子力学的本质——不确定性。

以下程序，可以让进程的不确定性更为清晰：

```c
/* foo.c */
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
```

```c
int a = 0;
void *foo(void *arg) {
    for (size_t i = 0; i < 10; i++) {
        a++;
        sleep(0.1);
        a--;
    }
    return NULL;
}
```

```c
void create_two_threads(void *(*f)(void *)) {
    pthread_t foo_t, bar_t;
    pthread_create(&foo_t, NULL, f, NULL);
    pthread_create(&bar_t, NULL, f, NULL);
```

```
    pthread_join(bar_t, NULL);
    pthread_join(foo_t, NULL);
}
```

```
int main(void) {
    create_two_threads(foo);
    printf("a = %d\n", a);
    return 0;
}
```

当线程 foo_t 和 bar_t 同时执行例程时，全局变量 a 的值会被它们修改。这种修改，在逻辑上，似乎并没有改变 a 的值，因为修改过程是让 a 增 1，线程主动阻塞 0.1 秒以模拟线程做了一些其他工作，然后再让 a 减 1，a 的最终结果应该还是 0。但是，倘若多次执行上述程序，例如执行 100 次：

```
$ for ((i = 1; i <= 100; i++)); do ./foo; done
```

你会发现，虽然多数情况下，程序 foo 输出的 a 值为 0，但是偶尔会出现 a 值为 1 或 -1 的情况，我不能确定是否还存在其他情况，但无论如何，倘若有其他程序依赖这个程序的输出的 a 值为 0，那么很有可能会导致一场灾难，例如用于发射宇宙飞船的程序，但是对人类产生更大影响的是，历史的混乱可能会引发人类的战争。

为什么会如此呢？自然是因为线程 foo_t 和 bar_t 是同时运行的，然而 a 是个全局变量，二者同时对 a 的值进行修改，若是 foo_t 在某一步修改 a 值时，它面对的是 bar_t 所修改的结果，可能会出现错乱，反之亦然。

11.5 服务端并发

主线程创建的多个子线程可以同时运行，这意味着我们可以像基于多进程机制实现 threebody 对多个客户端并发访问的支持。请将历史再度回退到第 5 章。我们可以基于田园时代的的字符串类和网络类，结合多线程机制，在套接字函数皆为阻塞的情景中为 threebody 赋予与多进程并发相似的能力。

```
/* threebody.c */
#include <unistd.h>
#include <pthread.h>
#include "sim-network.h"
```

```
void *handler(void *arg) {
    /* 类型转换 */
    SimServer *threebody_copy = arg;
```

```
    /* 从客户端接收数据 */
    SimStr *msg_from = sim_server_receive(threebody_copy);
    if (sim_str_safe(msg_from)) {
        printf("%s\n", sim_str_raw(msg_from));
    }
    sim_str_free(msg_from);

    /* 向客户端发送信息 */
    SimStr *msg_to = sim_str("threebody: Hi!");
    sim_server_send(threebody_copy, msg_to);
    sim_str_free(msg_to);

    /* 关闭客户端，释放 threebody 副本 */
    sim_server_close(threebody_copy);
    free(threebody_copy);
    return NULL;
}
```

```
int main(void) {
    SimServer *threebody = sim_server("localhost", "8080");
    while (1) {
        sim_server_run_once(threebody);
        /* 创建新的线程 */
        pthread_t t;
        SimServer *threebody_copy = sim_server_copy(threebody);
        pthread_create(&t, NULL, handler, threebody_copy);
        pthread_detach(t);
    }
    sim_server_free(threebody);
    return 0;
}
```

上述代码为了防止多个线程因共享 SimServer 对象 threebody 而带来的客户端套接字错乱，为每个新建的线程使用 sim_server_copy 构造了 threebody 的副本，该副本包含着新线程与之通信的客户端套接字。倘若不如此，那么很有可能会出现一个线程所用的客户端套接字会被主线程的 sim_server_sun 在接受新的客户端连接时修改而失效。

sim_server_copy 的声明和定义如下：

```
/* sim-network.h ++ */
/* 构造的副本，可用 free 释放 */
SimServer *sim_server_copy(SimServer *server);
```

```c
/* sim-network.c ++ */
SimServer *sim_server_copy(SimServer *server) {
    if (!server) return NULL;
    SimServer *copy = malloc(sizeof(SimServer));
    if (!copy) {
        fprintf(stderr, "sim_server_copy error!\n");
        return NULL;
    }
    *copy = *server;
    return copy;
}
```

还需要注意的是，上述的 threebody.c 中在创建新线程 t 后，又将 t 传给 pthread_detach 处理，该函数的作用是，让 t 脱离主线程而独立运行，可避免在主线程中使用 pthread_join 回收子线程。若不用此法，在主线程中使用 pthread_join 回收子线程，会导致主线程阻塞，亦即导致服务端的并发能力丧失。

编译 threebody.c，运行所得程序 threebody：

```
$ gcc -I. sim-str.c sim-network.c threebody.c -o threebody
$ ./threebody
```

然后打开两个终端，分别用于运行第 **6** 章的 other-ywj 和 ywj。先在一个终端里运行 other-ywj，它与 threebody 建立连接后，会主动延迟 15 秒，然后才开始与 threebody 通信。随即在另一个终端迅速运行 ywj。

上述试验所得结果与第 **10** 章的服务端多进程并发试验结果相似，ywj 无需等到 other-ywj 得到 threebody 的回复后放能与 threebody 通信。理论上，线程的切换速度远胜进程，故而服务端基于多线程的并发，性能通常优于多进程的并发。

线程与网络编程的联系，言尽于此。关于线程本身，却还有一些有趣且值得探究的方向。

11.6 互斥

多个线程的并行，若它们之间共享数据，便会导致混乱。我们无法消除这些混乱，但是却可以对其施加约束，这是操作系统赋予我们的能力。

我们将可能会导致多线程程序出现数据不确定性的区域称为临界区。为了保证程序运行结果的确定性，需要在临界区对线程加以约束，以避免它们在该区域出现冲突。这种约束，用计算机学科的术语表达，就是同步，即所有的线程到了临界区，都不要着急，都要乖乖等待操作系统的调度，在得到许可后方能访问临界区。站在人生的十字路口，我该何去何从？很简单，红灯停，绿灯行。

前言中希望用超弦理论比喻多线程机制，上文多少有些偏离该主旨，不过超弦理论也实在过于抽象，它也必须附着在人类对琴弦的认知上。多线程机制的临界区约束，犹如弹古筝、琵琶或吉他时，需要摁压琴弦，否则无法弹出富有变化的旋律。我不懂器乐，甚至对任何称作音乐的知识，几乎一无所知，所以我也不知道这样说是否正确。若有偏颇，望你不吝赐教。与摁压琴弦类似的现

象，在超弦理论中同样存在，即所有的开弦必须附着在某种结构上，诸如 D 膜。一条开弦，可以附着在多个 D 膜上……不要问我何谓 D 膜，我也不知道，犹如我不懂音乐。

Unix 或 Linux 系统中对临界区的施加的约束称为互斥（Mutex），不过软件工程中通常称之为「锁」，用于标定某块代码，不容多线程同时访问。互斥是系统资源，以 pthread_mutex_t 类型表示。互斥最简单的用法如下：

```
/* 全局互斥（或全局锁） */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
void *foo(void *arg) {
    for (size_t i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutex); /* 加锁 */
        a++;
        sleep(0.1);
        a--;
        pthread_mutex_unlock(&mutex); /* 解锁 */
    }
    return NULL;
}
```

上述代码片段，定义了一个以全局变量形式存在的互斥 mutex，在函数 foo 中，用 pthread_mutex_lock/unlock 对临界区域进行标定，从而约束了同时访问临界区的多个线程，增强了进程运行的确定性。基于上述代码修改上一节中的示例程序，便可保证程序的输出的 a 值不会出现非 0 的情况。

互斥的基本原理，想必你已基本理解了，你有没有发现，它很像之前所学习的网络编程中的阻塞？阻塞虽然简单，但性能不高，会导致很多线程在等待中浪费了太多时间。

11.7 内耗

看下面的程序：

```
/* foo.c */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define MAX_PRODUCED 5
int data;
int produced = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```c
void *producer(void *arg) {
    sleep(10); /* 模拟一些运算过程 */
    for (int i = 0; i < MAX_PRODUCED; i++) {
        pthread_mutex_lock(&mutex);
        /* 在区间 [0, 99] 中随机生成 1 个数字 */
        data = rand() % 100;
        produced++; /* 记录生成数据的次数 */
        printf("生产者生产数据: %d\n", data);
        pthread_mutex_unlock(&mutex);
        sleep(1); /* 模拟一些运算过程 */
    }
    return NULL;
}
```

```c
void *consumer(void *arg) {
    while (1) {
        sleep(1); /* 模拟一些运算过程 */
        printf("消费者等待……\n");
        pthread_mutex_lock(&mutex);
        if (produced > 0) { /* 有数据产出 */
            printf("消费者消费数据: %d\n", data);
            produced--; /* 消费 1 条数据 */
        } else {
            pthread_mutex_unlock(&mutex);
            continue;
        }
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

```c
int main() {
    pthread_t producer_thread, consumer_thread;
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);
    return 0;
}
```

这个程序实现了一个非常简单的生产者/消费者模型。producer 负责生产，consumer 负责消费。程序中使用了全局变量 data 表示被生产和消费的对象。

多线程例程 producer 和 consumer 共用了同一个互斥 mutex，这意味着虽然二者同时运行的，但是当二者之一对互斥进行锁定时，另一个例程若需要使用该互斥，则必须等待前者解除互斥锁定。如此便形成了一种奇特的现象，这两个例程发生了纠缠，就像是一个开弦在蜷缩在时空中的高维空间中发生了量子态范畴的自我纠缠……我也不知道我在说什么。我猜，你若是想理解我这句话，必须始终记住，例程 producer 和 consumer 不过是主线程在这个高维空间中的两种不同的振动形态，它们发生的纠缠，实际上还是主线程在该空间中的自我纠缠。

如果你编译上述程序并运行，得到的结果可能是

```
消费者等待……
消费者等待……
消费者等待……
消费者等待……
消费者等待……
消费者等待……
生产者生产数据：83
消费者等待……
消费者消费数据：83
生产者生产数据：86
消费者等待……
消费者消费数据：86
生产者生产数据：77
消费者等待……
```

也许你敏锐地注意到了，当生产者尚未产生数据时，消费者不得不在空耗中等待，每次等待都要锁定互斥，解锁互斥……这个过程是颇为浪费 CPU，即使我故意用了 sleep(1) 主动进行阻塞，以免你看到太多次消费者等待……。

还记得上次我们是如何解决网络编程中在非阻塞情况下服务端过多消耗 CPU 这一问题的吗？我们是利用了操作系统提供的同步 I/O 多路复用机制，而且我也多次强调，这种机制堪比计算机软件世界里的爱因斯坦的广义相对论。事实上，超弦理论……不，操作系统也为多线程机制中也提供了类似的功能。

11.8 超距作用

在使用互斥加以约束的临界区中，可以通过条件变量实现一个例程在运行时触发其他例程运行，同时该例程的运行也可由其他例程触发。

我们通过改写上一节中的两个线程例程，来理解条件变量的基本用法。首先，定义一个全局的条件变量：

```
pthread_cond_t data_ready = PTHREAD_COND_INITIALIZER;
```

data_ready 便是条件变量。作为初学者，不必纠结 PTHREAD_COND_INITIALIZER 是什么含义以及有无其他值可初始化条件变量。

然后，在 producer 中，若数据准备好，就发出一个信号：

```
void *producer(void *arg) {
    sleep(10); /* 模拟一些运算过程 */
    for (int i = 0; i < MAX_PRODUCED; i++) {
        pthread_mutex_lock(&mutex);
        /* 在区间 [0, 99] 中随机生成 1 个数字 */
        data = rand() % 100;
        printf("生产者生产数据: %d\n", data);
        pthread_cond_signal(&data_ready); /* 发出信号 */
        pthread_mutex_unlock(&mutex);
        sleep(1); /* 模拟一些运算过程 */
    }
    return NULL;
}
```

上述代码中的 pthread_cond_signal 可以通知某个正在等待 data_ready 这个条件变量的线程例程，数据准备好了！在我们的示例中，等待 data_ready 的是 consumer：

```
void *consumer(void *arg) {
    while (1) {
        printf("消费者等待……\n");
        pthread_mutex_lock(&mutex);
        pthread_cond_wait(&data_ready, &mutex);
        printf("消费者消费数据: %d\n", data);
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

在上述示例中，pthread_cond_wait 解锁 mutex，然后会主动阻塞，以等待其他例程发送的信号。收到信号后，它会锁定 mutex，然后返回，于是其后的语句得以继续执行。

用上述修改替换上一节示例中的两个例程，运行结果如下：

```
消费者等待……
生产者生产数据: 83
消费者消费数据: 83
消费者等待……
生产者生产数据: 86
消费者消费数据: 86
```

```
消费者等待……
生产者生产数据：77
消费者消费数据：77
消费者等待……
生产者生产数据：15
消费者消费数据：15
消费者等待……
生产者生产数据：93
消费者消费数据：93
消费者等待……
```

上述结果与上一节不同之处在于，在 producer 耗费 10 秒钟的时间准备数据时，consumer 被阻塞在 pthread_cond_wait 语句处，且不占用 CPU，直至 producer 通过 pthread_cond_signal 唤醒它。

使用 pthread_cond_wait 时需要注意，处于等待的线程例程可能会被虚假唤醒，而此时可能例程所需要的数据并未备好，从而导致进程依然会出现不确定性。为了保险起见，需对例程增加人为防御。例如：

```c
int produced = 0;
void *producer(void *arg) {
    sleep(10); /* 模拟一些运算过程 */
    for (int i = 0; i < MAX_PRODUCED; i++) {
        pthread_mutex_lock(&mutex);
        data = rand() % 100;
        produced++;
        printf("生产者生产数据：%d\n", data);
        pthread_cond_signal(&data_ready); /* 发出信号 */
        pthread_mutex_unlock(&mutex);
        sleep(1); /* 模拟一些运算过程 */
    }
    return NULL;
}
```

```c
void *consumer(void *arg) {
    while (1) {
        printf("消费者等待……\n");
        pthread_mutex_lock(&mutex);
        while (produced == 0) {
            pthread_cond_wait(&data_ready, &mutex);
        }
        printf("消费者消费数据：%d\n", data);
```

```
        produced--;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

我们用计数器 produced 为 pthread_cond_wait 增加了一层保险机制，即使 pthread_cond_wait 被误触，例程依然可以根据 produced 的值断定是否继续等待。

如果有多个线程例程都在等待同一个条件变量，pthread_cond_signal 只能通知其中一个，若全部通知可使用 pthread_cond_broadcast，二者用法相同。一个线程能够通知所有正在等待的线程，这让你想到了什么？前者像不像一个服务端，后者像不像连接到服务端的客户端？由于互斥的存在，被唤醒的多个线程，也只能逐一访问临界区。服务端/客户端模型，在理念上，完全可以称之为生产者/消费者模型。

这很像同步 I/O 多路复用机制，对吧？所以我说，多线程机制实际上能够统一同步 I/O 多路复用机制和多进程并发机制，犹如超弦理论能够统一广义相对论和量子力学。

11.9 总结

我不懂超弦理论，但是如果研究超弦理论的人，能尝试从计算机多线程机制中获得一些比喻，也许有助于他们能够写出更好的超弦理论科普书。不过，曾经存在着五种超弦理论，后来爱德华·威滕提出了膜理论，将五种超弦理论统一了起来，手法是除了引入膜结构，也为五种超弦理论所认为的十维宇宙模型增加了一个新的维度，论证了五种超弦理论只是这个新维度退化后的产物。也许你会问，那么计算机软件世界里，存在与 M 理论对应的机制吗？

第 12 章 天地一指

12.1 前言

在超弦理论致力于统一广义相对论与量子力学过程中，令超弦理论学家尴尬的是，他们创造了五种不同的超弦理论，这不是统一，而是更严重的分裂。好在爱德华·威滕出手，在超弦理论的十维空间设定上增加了一个新的维度，并提出了膜的概念，认为以振动的方式形成粒子的弦是 1-维膜，亦即弦是膜的特例。

实际上，操作系统对线程的调度，也有多种策略，足够凑出 5 种了。我将每种策略对应每个超弦理论，这就意味着，若追求一种大一统的线程调度理论，也需要在线程上增加一个维度，同时构造出与膜在哲学上有共性的结构。

这个结构是什么呢？协程！它的历史据说比进程和线程都要悠久，只是长期未受学术界和工程界的重视而沉睡数十年，直至最近这些年，因对回调地狱这一顽疾有神奇疗效而大行其道。现今，若是何种编程语言未提供与构造协程相关的语法，应该会招来众多批评。

C 语言没有协程的概念，不过，我们可以尝试给出一些近似的模拟，也许能让你更为深刻地理解，何为协程。

12.2 新的维度

实际上这个新的维度，我们在第 11 章基于多线程、互斥以及条件语句所实现的生产者/消费者程序中已经感受到了，只不过是以线程「通信」的方式。若不用线程，基于多进程以及进程间通信（例如网络通信）机制，也能实现生产者/消费者程序。

现在，我们考虑的是，普通的两个 C 函数，它能否实现生产者/消费者模型呢？若答案是能，这意味着普通的函数不仅可并发运行且能彼此节制，那么无论是多线程机制，还是多进程机制，在理论上它们就毫无神秘之处可言了；它们之所以神秘，是因操作系统实现的调度和通信机制对我们而言是透明的，而现在我们要打破这种透明。

在 C 语言中，两个普通的函数 foo 和 bar，它们之间若存在关系——在物理学里，你可称其为引力，只能是 foo 调用 bar，或 bar 调用 foo。不过，若再引入一个函数 foobar，我们希望在其中能够建立 foo 和 bar 的关系，前提是无需二者存在调用关系，就像在生产者/消费者模型中，一个只考虑生产，一个只考虑消费，但二者却存在联系。

为了更好地表达我们的意图，我需要先写一个普通的 foo 函数，它用于生产数据：

```
#define TOTAL 7
int data = 0;
void foo(void) {
    for (int i = 0; i < TOTAL; i++) {
        data = i + 1;
        printf("生产者：生成数据 %d\n", data);
    }
}
```

再写一个普通的 bar 函数，用于消费数据：

```
void bar(void) {
    printf("消费者：消费数据 %d\n", data);
}
```

现在，我们期望的是，在 foo 每产出 1 条数据，bar 随即消费它，但二者不存在调用关系。实现这一目的的第一步是，需要让 foo 在每产出 1 条数据后，便暂停，但是 C 语言没有给普通函数提供这样的暂停支持，操作系统也只能暂停线程或进程，但是仔细想一下，我们用 return 能够让 foo 在产出 1 条数据后就返回：

```
void foo(void) {
    for (int i = 0; i < TOTAL; i++) {
        data = i + 1;
        printf("生产者：生成数据 %d\n", data);
        return;
    }
}
```

现在若在 foobar 函数里运行 foo 函数和 bar 函数：

```
void foobar(void) {
    foo();
    bar();
}
```

可以完成数据的第一次产出和消费。

若在 foobar 中再次运行 foo 时，foo 能从上一次它的返回之处继续运行，便可产出下 1 条数据了，不过前提是需要知道上一次循环变量的值。这个前提，可借助 static 变量实现：

```
void foo(void) {
    static int i;
    for (i = 0; i < TOTAL; i++) {
        data = i + 1;
        printf("生产者：生成数据 %d\n", data);
        return;
        /* 我们希望再次运行 foo 时，从此处运行 */
    }
}
```

在函数内的 static 变量 i，它拥有全局变量的特性，在 foo 函数再次运行时，i 依然是上一次 foo 函数返回时的状态。通过 static 变量，能够让 foo 函数拥有记忆……或者历史。

实现再次运行 foo 时从它上次的返回处继续运行，需借助一个状态机和长期被视为恶魔的 goto 语句：

```
void foo(void) {
    static int i;
    static int state = 0;
    switch (state) {
        case 0: goto BEGIN;
        case 1: goto NEXT;
    }
    BEGIN:
    for (i = 0; i < TOTAL; i++) {
         data = i + 1;
         printf("生产者: 生成数据 %d\n", data);
         state = 1;
         return;
         NEXT:
    }
    state = 0; /* 状态机重置 */
}
```

switch/case 语句实现了一个很简单的状态机，其状态变量也是 foo 的记忆。可能长期以来，大家对 goto 的痛恨，已经让你淡忘了 goto 的用法，现在可趁机温习。goto 可将程序的运行流导向到代码中用标签标定的位置。

现在，在 foobar 里两次调用 foo 和 bar：

```
void foobar(void) {
    foo();
    bar();
    foo();
    bar();
}
```

foobar 的输出应当是

```
生产者: 生成数据 1
消费者: 消费数据 1
生产者: 生成数据 2
消费者: 消费数据 2
```

现在，将 foobar 写为

```
void foobar(void) {
    for (int i = 0; i < TOTAL; i++) {
        foo();
        bar();
    }
}
```

便可将 foo 所有产出的数据逐一交于 bar 消费掉。

但愿你会因此觉得惊奇。这种惊奇不仅仅是我们在 foo 和 bar 无需借助调用关系的前提下实现了二者的协作，它更是一切竟如此简单。可惜的是，物理学界的超弦和 M 理论专家，他们似乎理解不了这种简单，反而制造出一些精妙但复杂的机制，犹如尽浓妆艳抹之能事，打扮一位清水芙蓉般的少女。

12.3 梳子

如果你的确因 goto 的存在而不安，在 C 语言中的确是有方法完全基于你信任的 switch/case 语句而消除 goto 语句以及它所需要的所有标签，只是我又担心，这种方法又会让你对 switch/case 觉得不安。这种方法就是，switch/case 语句能像梳子一样刺入松软的代码块中：

```
void foo(void) {
    static int i;
    static int state = 0;
    switch (state) {
    case 0: for (i = 0; i < TOTAL; i++) {
                data = i + 1;
                printf("生产者: 生成数据 %d\n", data);
                state = 1;
                return;
    case 1: ;
            } /* 循环结束 */
    } /* 状态机结束 */
    state = 0;
}
```

上述代码所用的方法，就是历史上有名的「达夫设备（Duff device）」，它合乎 C 语法，原因是 case 语句与配合 goto 的标签是等价的，它们的作用只是用于标定程序流的位置，但对程序流毫不影响。改变程序流的是 switch 或 goto。

不妨勇敢一些，在新的 foo 函数中，代码左侧的 switch / case 语句是卡拉比-丘空间，右侧的 for 循环结构是我们的时空。在超弦理论里，这个卡拉比-丘空间，还是像超弦理论那样畏畏缩缩地蜷缩于时空之中，而在新的 foo 函数里，它完全融入了时空。

12.4 牛顿

物理学的本质是什么？是创造一套尽量简单的协议，这份协议像是与天地订下的契约。既然我们已经发现了一种可以让函数之间平等相处的方法，而不是传统的谁调用谁，谁被谁调用这种类似物理学中引力机制的方法，我们也可以将其协议化，这就是计算机软件世界的「物理」学。

我们创造的第 1 条协议是

```
#define cr_begin static int state = 0; \
                 switch (state) { \
                 case 0:
```

上述代码定义了一个宏 cr_begin，若像下面这样调用它：

```
cr_begin;
```

可展开为

```
static int state = 0; switch (state) { case 0:;
```

case 0: 后面虽然多处了;，但是无妨，它是空代码。

类似地，创造第 2 条协议：

```
#define cr_yield do {state = 1; \
                     return; \
                     case 1: ; } while (0)
```

do {... } while (0) 是 C 语言宏定义时常用的技法，它用一个只能运行一次的 do ... while 循环结构将一些代码包含起来，使得宏的展开结果不与周边代码产生错误联系。不过，将 case 1: 这样的语句直接放在一个循环结构中，也许你心里难免会有些恐慌。不要怕，case 1: 只是个标签，它无论放在何处，也只是标定当前程序流的位置而已，何况其后只是一个空语句。

第 3 条协议是：

```
#define cr_end do {state = 0;} while (0); }
```

基于上述 3 条协议，便可重写 foo 函数：

```
void foo(void) {
    static int i;
    cr_begin;
    for (i = 0; i < TOTAL; i++) {
            data = i + 1;
            printf("生产者：生成数据 %d\n", data);
            cr_yield;
    }
```

```
    cr_end;
}
```

foo 函数的实现看上去已颇为舒爽，不过存在隐患。例如将 foo 改写为

```
void foo(void) {
    static int i;
    cr_begin;
    for (i = 0; i < TOTAL; i++) {
            data = i + 1;
            printf("生产者: 生成数据 %d\n", data);
            cr_yield;
            printf("生产者: 已产出数据共 %d 次\n", i);
            cr_yield;
    }
    cr_end;
}
```

上述代码等价于

```
void foo(void) {
    static int i;
    static int state = 0;
    switch (state) {
    case 0: for (i = 0; i < TOTAL; i++) {
                data = i + 1;
                printf("生产者: 生成数据 %d\n", data);
                state = 1;
                return;
    case 1:     ;
                printf("生产者: 已产出数据共 %d 次\n", i);
                state = 1;
                return;
    case 1: ;
            } /* 循环结束 */
    } /* 状态机结束 */
    state = 0;
}
```

假设上述的 foo 函数已运行了两次，它在第 3 次运行时，switch 会将程序流切换到第一个 case 1 处，但实际上应该切换到第 2 个 case 1，亦即当 cr_yield 多次出现时，状态变量 state 需要有变化，它不能只是 1。为了解决这个问题，需要重新定义 cr_yield:

```
#define cr_yield(n) do {state = n; \
                        return; \
                        case n: ; } while (0)
```

新的 cr_yield 需要 1 个参数，用作状态变量 state 的值。

修改上述 foo 函数：

```
void foo(void) {
    static int i;
    cr_begin;
    for (i = 0; i < TOTAL; i++) {
            data = i + 1;
            printf("生产者: 生成数据 %d\n", data);
            cr_yield(1);
            printf("生产者: 已产出数据共 %d 次\n", i);
            cr_yield(2);
    }
    cr_end;
}
```

宏展开后的代码等价于以下代码：

```
void foo(void) {
    static int i;
    static int state = 0;
    switch (state) {
    case 0: for (i = 0; i < TOTAL; i++) {
                data = i + 1;
                printf("生产者: 生成数据 %d\n", data);
                state = 1;
                return;
    case 1: ;
                printf("生产者: 已产出数据共 %d 次\n", i);
                state = 2;
                return;
    case 2: ;
        } /* 循环结束 */
    } /* 状态机结束 */
    state = 0;
}
```

不过，我们有更好的解决方案。C 语言标准中有一个宏 __LINE__，调用它时，展开结果是当前代码的行号，我们可以用它作为状态变量的值，便可保证多次调用 cr_yield 时，不会出现重复状态。重新定义 cr_yield：

```
#define cr_yield do {state = __LINE__; \
                     return; \
                     case __LINE__: ; } while (0)
```

现在 cr_yield 又变成无参数宏了。只是要记住，不要在同一行内多次调用 cr_yield，永远如此，不然状态变量 state 的值又要出现重复了。

12.5 返回值

上一节实现的 cr_yield 只适合无返回值的协程。实际上大多数函数需要有返回值，故而还需改进 cr_yield，使之能支持更具一般性的函数。

C 语言从 C99 标准开始提供了可变参数宏的语法。我们可以基于可变参数宏定义 cy_yield：

```
#define cr_yield(...) do {state = __LINE__; \
                          return __VA_ARGS__; \
                          case __LINE__: ; } while (0)
```

对于无返回值的函数，只需像下面这样使用 cr_yield：

```
cr_yield();
```

对于有返回值的函数，假设返回值为 foo，只需

```
cr_yield(foo);
```

12.6 拉链

我想出来一个例子，它更能表达协程的用途。这个例子很像生活中的拉链。

假设有一个数组 numbers，其中存储了一些整型数，有正数，有负数。另外有两个函数，foo 和 bar，它们都要遍历 numbers。foo 需要处理 numbers 里的正数，例如将其增 1。我们希望当 foo 对 numbers 里的每个正数增 1 的时候，bar 能随即将该正数减 1。

我们先用协程实现 foo 函数：

```
void foo(int *numbers, int n) {
    static int i;
    cr_begin;
    for (i = 0; i < n; i++) {
        int a = numbers[i];
        if (a > 0) printf("(%d, ", a + 1);
```

```
        cr_yield;
    }
    cr_end;
}
```

bar 函数与 foo 类似：

```
void bar(int *numbers, int n) {
    static int i;
    cr_begin;
    for (i = 0; i < n; i++) {
        int a = numbers[i];
        if (a > 0) printf("%d)\n", a - 1);
        cr_yield();
    }
    cr_end;
}
```

foo 和 bar 就像拉链两边的链带，或者像两条铁轨，或者像 DNA 的双链……下面的 foobar 函数则像拉锁头、火车、读取 DNA 的蛋白质……

```
void foobar(void) {
    int numbers[] = {1, 3, -2, -3, 5};
    int n = sizeof(numbers) / sizeof(int);
    for (int i = 0; i < n; i++) {
        foo(numbers, n);
        bar(numbers, n);
    }
}
```

foobar 的输出应当是

```
(2, 0)
(4, 2)
(6, 4)
```

协程能够将两个独立的过程交相运行。在操作系统中，不同的线程或不同的进程实际上也是用着相似的原理运行的。对于上述的 foo 和 bar 而言，foobar 是它们的操作系统。

12.7 可重入性

我们的协程物理学，对于简单的情况已经是足够用的了，但是在多线程机制里，它会很容易错乱起来。例如，如果有两个线程皆以上一节的 foo 函数作为例程，由于循环变量 i 和状态变量

state 皆为 static，这意味着它们是两个线程共享的变量，当一个线程修改它，它便可能会导致另一个线程的行为异常，通过互斥加以保护也无用，因为这两个变量本不应该被多个线程共享。还有一处，foo 函数产出的数据，也是保存在全局变量 data 里的，它也不该被多个线程共享。

像 foo 这样的函数，在多线程环境中，堪称以一女事二夫——仅作比喻，不歧视性别和伦理，通常称它们为不具备可重入性。为了程序的安全，让函数具备可重入性通常是必要的。

若一个函数，它原本依赖静态变量或全局变量，但是为了可重入性，必须放弃这种依赖，最终它必须以闭包（Closure）的形式存在。倘若你之前经常听到闭包而不知其意，那么至少在 C 语言的语境里，我已很清晰地给你刻画出了它该有的样子，倘若你依然未能想象出来，不妨看下面重新定义的 foo 函数：

```
void foo(int *state, int *i, int *data) {
    switch (*state) {
    case 0: for (*i = 0; *i < TOTAL; (*i)++) {
                *data = *i + 1;
                printf("生产者：生成数据 %d\n", *data);
                *state = 1;
                return;
    case 1: ;
            }
    }
    *state = 0;
}
```

我们只需将 foo 原本依赖的静态变量和全局变量，统统变成指针形式的参数。至于这些指针指向的变量，它们可以是某个函数的局部变量，例如，

```
void foobar(void) {
    int state = 0, i = 0; data = 0;
    foo(&state, &i, &data);
}
```

于是，foo 函数便是可重入的函数，而且我们也可以称它是一个闭包。若对闭包给出一个明确的定义，就是具备捕捉外部变量能力的函数。就像一个茶杯，它虽然在宇宙之中，但是它不也装着整个宇宙么？在 C 程序里，将指针作为函数参数，是一种非常普遍的现象，只是大家很少去关心这样的函数是闭包，原因是 C 语言并不觉得有什么必要开设这个概念。

同理，我们也能将消费数据的函数 bar 改成闭包，让它无需再依赖全局变量 data：

```
void bar(int *data) {
    printf("消费者：消费数据 %d\n", *data);
}
```

对于变成了闭包的 foo，之前我们设计的协程物理学，也需要相应地予以改进，但这并非难事。你可以作为一道练习题去完成它。

若我此刻我说，在超弦理论中，以振动的方式形成各种基本粒子的开弦，必须附着在一种叫作 D 膜的结构上，而在 M 理论中，基本粒子是由一种 2 维的 M 膜的振动形成的，而这种膜不需要附着在其它结构上。可能你不懂这句话的意思，请放心，我也不懂我在说什么。我只是大致明白了，在多线程机制中，若想用多个线程做一些有用处的事情，这些线程之间必须用互斥和条件变量予以约束，而在协程中，这种约束原本是通过全局变量或函数内的静态变量实现的，但是当我们将协程升级为闭包，这种约束也就不存在了。闭包可以不依附其它结构而自由存在。

12.8 总结

天地一协程，万物一闭包。

C 语言没有协程，也没有闭包，却也能无中生有。

我们好像已经走得太远了，甚至失去了踪迹。

协程，与网络编程有什么关系呢？

第 13 章 返朴

13.1 前言

一路行来，甚为艰难，所幸见识和遐想颇多。进程、线程及至协程，我并无计划将这些技术融入 Sim 项目。这些技术都很有用处，可它们过于聪明了。驾驭它们，如同驯服野马，这与 Sim 项目的本意是矛盾的。Sim 项目只能在你企图驯服这些野马时，为你提供马鞍和缰绳。之前的章节实现的 Sim 项目，已有驯服的野马在奔跑了。我们应该删除这些聪明之处，不然此时的聪明，在未来可能是自作聪明的绊脚石。

本文以第 **9** 章的 Sim 项目的实现为底本，清理一些代码，并增加一些方法和宏，让 Sim 项目趋向于提供简洁的机制，而非既定的策略。

13.2 简化

首先，修改 `SimServer` 对象的 `receive` 和 `send` 方法，令其面向给定的客户端套接字：

```
/* sim-network.h [改] */
SimStr *sim_server_receive(SimServer *self, int client);
void sim_server_send(SimServer *self, int client, SimStr *data);
```

```
/* sim-network.c [改] */
SimStr *sim_server_receive(SimServer *self, int client) {
    if (client < 0) return NULL;
    if (FD_ISSET(client, &self->read_fds)) {
        SimStr *msg = recv_robustly(client);
        if (msg) return msg;
        else {
            /* 客户端套接字无法读取，将其标记为无效 */
            sim_server_invalid_client(self, client);
            return NULL;
        }
    }
}
void sim_server_send(SimServer *self, int client, SimStr *data) {
    if (FD_ISSET(client, &self->write_fds)) {
        if (send_robustly(client, data) == -1) {
            sim_server_invalid_client(self, client);
        }
    }
}
```

上述代码中使用的 SimServer 的 invalid_client 方法尚未定义，现在你只需假设它存在。

在之前的 receive 和 send 方法的实现中，遇到无效的客户端套接字，我们是当场将从 SimServer 对象的 clients 链表中将其删除，这个过程不该交由一个仅负责数据接收或发送的函数完成，如同一个快递员给我送货，他不能因为没能联系到我，便自作聪明，认为我这个人并不存在。网络通信中，收发数据的函数，将无效的套接字暂且记录下来，最终交由 SimServer 对象在适当的时机予以处理，更为妥当。

通过上述修改，你是否发现，我们之前实在是太过于追求聪明而创造了许多非自然的功能？

还需要注意，第 9 章实现的 receive 和 send 方法，判断客户端套接字是否有效时，用的是严格的条件，即要求客户端套接字能同时满足可读和可写，即

```
if (FD_ISSET(client, &read_fds) && FD_ISSET(client, &write_fds)) {
    /* 数据接收或发送过程 */
    ... ... ... ... ... ...
}
```

这个条件，当初在 8.7 节中引入，现在我们应放弃它。服务端从一个客户端套接字读数据，只能要求该套接字在 read_fds 之内，反之向该客户端套接字写入数据，只能要求它在 write_fds 之内。

13.3 无效的客户端

SimServer 对象在收发数据的过程中会遇到一些无效的客户端套接字，上一节的处理方式是将其标记为无效。我们可以在 SimServer 类里新设一个单向链表对象 invalid_clients，出现于该链表中的客户端套接字即为无效。

```
/* sim-network.c [改] */
struct sim_server {
    int listener;
    SimList *clients;
    SimList *invalid_clients;
    fd_set read_fds;
    fd_set write_fds;
    const char *error;
};
```

```
/* sim-network.c [改] */
SimServer *sim_server(const char *host, const char *port) {
    /* [上文] server->clients = NULL; */
    server->invalid_clients = NULL;
    /* [下文] server->error = NULL; */
}
```

在上述修改中，我们在第 4 章创造的示意标记的基础上，又创造了一种新的示意标记，即在所修改的代码中，根据上述 [上文] 和 [下文] 标记中的代码找到相应的两个位置，在它们中间插入代码。我觉得，假如你也不想让我将函数的原有代码都复制过来，然后在其中悄悄插入一行代码，那么你应该能理解我在说什么。

SimServer 的析构函数也要修改，如下：

```
/* sim-network.c [改] */
static void clients_free(SimList *clients);
void sim_server_free(SimServer *server) {
    if (server) {
        close(server->listener);
        clients_free(self->clients);
        clients_free(self->invalid_clients);
        free(server);
    }
}
```

```
static void clients_free(SimList *clients) {
    if (clients) {
        for (SimList *it = server->clients; it; it = it->next) {
            if (it->data) free(it->data);
        }
        sim_list_free(clients);
    }
}
```

sim_server_invalid_client 的声明和实现如下：

```
/* sim-network.h ++ */
void sim_server_invalid_client(SimServer *self, int client);
```

```
/* sim-network.c ++ */
void sim_server_invalid_client(SimServer *self, int client) {
    bool existed = false;
    for (SimList *it = self->invalid_clients; it; it = it->next) {
        int fd = *(int *)it->data;
        if (fd == client) {
            existed = true;
            break;
        }
    }
```

```
    if (!existed) {
        SIM_LIST_ADD(self->invalid_clients, client, int);
    }
}
```

在每一次运行 SimServer 的 run 方法时，首先需要基于 invalid_clients，将 clients 中的无效客户端套接字删除，这个过程需要一个专用的函数：

```
/* sim-network.h ++ */
static SimList *find_node(SimList *list, int fd);
static SimList *clean_clients(SimList *clients, SimList *invalid_clients) {
    for (SimList *it = invalid_clients; it; it = it->next) {
        int client = *(int *)it->data;
        SimList *t = find_node(clients, client);
        if (t) clients = sim_list_delete(clients, t);
        close(client);
    }
    return clients;
}
```

其中，find_node 可从链表中寻找含有指定套接字的结点：

```
/* sim-network.h ++ */
static SimList *find_node(SimList *list, int fd) {
    SimList *t = NULL;
    for (SimList *it = list; it; it = it->next) {
        int a = *(int *)it->data;
        if (a == fd) {
            t = it;
            break;
        }
    }
    return t;
}
```

上述一系列操作，体现了单向列表在查找和删除某个结点时过于耗费时间，不过依然可以容忍。

现在，可修改 sim_server_run_once 了，在其开头对无效的套接字予以清除：

```
/* sim-network.c [改] */
void sim_server_run_once(SimServer *self) {
    /* 清理无效的套接字 */
    SimList *invalid_clients = self->invalid_clients;
    self->clients = clean_clients(self->clients, invalid_clients);
```

```
    /* 释放 invalid_clients */
    for (SimList *it = invalid_clients; it; it = it->next) {
        free(it->data);
    }
    sim_list_free(invalid_clients);
    self->invalid_clients = NULL;
    /* [下文]
    int fd_max;
    while (1) {
        FD_ZERO(&self->read_fds); */
}
```

13.4 公之于众

由于 SimServer 对象的 recieve 和 send 方法需要给定客户端套接字方能使用，我们必须将 SimServer 对象的 clients 公开给用户：

```
/* sim-network.h ++ */
SimList *sim_server_clients(SimServer *self);
```

```
/* sim-network.c ++ */
SimList *sim_server_clients(SimServer *self) {
    if (self) return self->clients;
    else return NULL;
}
```

用户需要记住，在遍历客户端链表过程中，对于任一结点 it，需要使用类似以下语句，提取客户端套接字：

```
int client = *(int *)it->data;
```

也许会有无数软件工程爱好者会告诫我，这样暴露了太多细节，程序会非常不安全。我承认，事实的确如此，但我还是愿意相信 Sim 库的用户都希望自己所写的代码是安全的，毕竟他们在生活中的大多数时间是不会闯红灯的。

13.5 新的 threebody

为了新的 SimServer 类是否可用以及展示其用法，需重写 threebody 程序：

```
/* threebody.c */
#include "sim-network.h"
```

```c
int main(void) {
    SimServer *threebody = sim_server("localhost", "8080");
    if (!threebody) {
        fprintf(stderr, "sim_server failed!\backslashn");
        exit(-1);
    }
    /* 服务端程序运转 */
    SimStr *hi = sim_str("threebody: hi!");
    while (1) {
        sim_server_run_once(threebody);
        if (!sim_server_safe(threebody)) continue;

        SimList *clients = sim_server_clients(threebody);
        printf("%lu clients!\backslashn", sim_list_size(clients));
        for (SimList *it = clients; it; it = it->next) {
            int client = *(int *)it->data;
            /* 从客户端接收信息 */
            SimStr *msg = sim_server_receive(threebody, client);
            if (msg) {
                printf("%s\backslashn", sim_str_raw(msg));
                sim_str_free(msg);
            } else continue;
            /* 向客户端发送信息 */
            sim_server_send(threebody, client, hi);
        }
    }
    sim_str_free(hi);
    sim_server_free(threebody);
    return 0;
}
```

编译 threebody.c：

```
$ gcc -I. sim-str.c sim-list.c sim-network.c threebody.c -o threebody
```

运行 threebody：

```
$ ./threebody
```

然后运行第 **6** 章的 ywj 和 other-ywj，除了 threebody 能够正常与她们应答之外，也能看到 threebody 输出的当前客户端连接数量，无论运行多少次 ywj，这个数量绝大多数时间是 1，说明我们为 `SimServer` 对象增加的清除无效客户端套接字的功能是有效的。

13.6 宏

我们需要为 Sim 项目新建一个头文件 sim-macros.h，用于定义一些常用的宏，这些宏有一些是只在 Sim 项目内部使用的，也有一些供 Sim 库的用户使用，例如协程宏。

我们先为 `sim_str_safe`，`sim_client_safe`，`sim_server_safe` 以及未来一些新的类，定义一个通用的安全检测宏：

```
/* sim-macros.h */
#ifndef SIM_MACROS_H
#define SIM_MACROS_H

/* 只在 Sim 项目内部使用 */
#define SIM_OBJECT_SAFE(object) do { \
        if (self) { \
         /* 不太致命的错误 */ \
         if (self->error) { \
             fprintf(stderr, "%s error: %s\backslashn", \
                 __func__, self->error); \
             return false; \
         } \
     } else { \
         /* 致命错误 */ \
         fprintf(stderr, "%s error: NULL pointer!", __func__); \
         return false; \
     } \
     return true; \
} while (0)

#endif
```

注意，上述代码所用的 `__func__` 是 C99 标准引入的标识符，它是一个字符串常量，表示当前代码所属函数的名字。例如，若在 `sim_str_safe` 中使用 `SIM_OBJECT_SAFE`，则 `__func__` 便是 `sim_str_safe`。

现在修改 Sim 项目中现有的全部 `safe` 方法：

```
/* sim-str.c ++ */
#include "sim-macros.h"
```

```
/* sim-str.c [改] */
bool sim_str_safe(SimStr *self) {
    SIM_OBJECT_SAFE(self);
}
```

```c
/* sim-network.c ++ */
#include "sim-macros.h"
```

```c
/* sim-network.c [改] */
bool sim_client_safe(SimStr *self) {
    SIM_OBJECT_SAFE(self);
}

bool sim_server_safe(SimServer *self) {
    SIM_OBJECT_SAFE(self);
}
```

这个宏，现在已经让我们可以少写大约 30 行非常无趣的代码了。要善用宏！

13.7 协程

我们将基于 duff 设备的协程宏也放在 sim-macros.h 中。在 Sim 项目中，我们不使用协程，在基于 Sim 库编写网络应用程序时，我们以及其他用户，可以使用协程。

第 **12** 章指出，若让协程具备可重入性，它只能以闭包的形式出现，而所谓闭包，就是一个函数，如果它的参数是指针，在其内部能够通过这个指针修改外部变量的值。在 C 语言编程中，这原本是一种极为常见函数形式，理论家们将其命名为闭包，反而非常难以理解了。

闭包形式的协程，其参数通常至少需要 3 个指针，例如

```c
void foo(int *state, int *i, void *data) {
    /* 闭包形式的协程 */
    ... ... ... ... ...
}
```

倘若 `foo` 函数还需要其他参数，因参数过多，便会导致该函数的意义难以被他人理解。我们可以将与协程有关的参数聚拢为一个结构体对象：

```c
/* sim-macros.h ++ */
typedef struct sim_cr SimCr;
```

`struct sim_cr` 可以根据自己的需要予以定义，例如针对上述协程 `foo` 的参数结构体可定义为

```c
struct sim_cr {
    int state;
    int i;
    void *data;
    bool done;
};
```

其中 state 用于控制协程状态，done 用于表示协程是否终止。在使用上述结构体时，只需构造一个参数结构体对象，然后将该对象的地址传给 foo：

```
/* 定义协程 foo */
void foo(SimCr *cr) {
    /* 闭包形式的协程 */
    ... ... ... ... ...
    /* 协程结束 */
    cr->done = true;
}
```

```
/* 调用协程 */
SimCr cr = {0, 0, NULL, false};
foo(&cr);
```

定义结构体 sim_cr 时，结构体一定要有 state 和 done，切记，切记，切记！至于其他成员，要根据具体需求而定，例如

```
struct sim_cr {
    int state;
    SimList *it;
    SimStr *str;
    bool done;
};
```

基于 SimCr 类型，便可定义闭包形式的协程宏：

```
/* sim-macros.h ++ */
#define sim_cr_begin(cr) switch (cr->state) { case 0:

#define sim_cr_yield(cr,...) do {cr->state = __LINE__; \
                                 return __VA_ARGS__; \
                                 case __LINE__: ; } while (0)

#define sim_cr_end(cr) } cr->done = true;
```

13.8 叮咚或咚叮

对于阻塞的套接字，我们曾试验过基于进程和线程的并发，即为服务端接受的每个客户端分配一个进程或线程与之通信。现在我们已经基本理解了协程，不妨大胆构想，在基于非阻塞的套接字和同步 I/O 多路复用机制实现的服务端中，若为每个客户端分配一个协程，结果会如何？

我们再次以 threebody 为例，试试这个想法。

```c
/* threebody.c */
#include "sim-macros.h"
#include "sim-network.h"
```

我们将协程必须的参数定义为

```c
/* threebody.c ++ */
struct sim_cr {
    int state;
    bool done;
    SimStr *hi;
    char *ding;
    char *dong;
};
```

然后定义一个可与客户端通信的协程：

```c
/* threebody.c ++ */
static void handle_client(SimCr *cr, SimServer *server, int client) {
    sim_cr_begin(cr);
    while (!cr->ding || !cr->dong) {
        if (!cr->ding) {
            SimStr *msg = sim_server_receive(server, client);
            if (msg) {
                cr->ding = "叮";
                printf("%s\backslashn", sim_str_raw(msg));
                sim_str_free(msg);
            }
        }
        if (!cr->dong) {
            sim_server_send(server, client, cr->hi);
            if (sim_server_safe(server)) cr->dong = "咚";
        }
        sim_cr_yield(cr);
    }
    sim_cr_end(cr);
}
```

在 main 函数的一开始，我们定义一个指针数组 coroutines，并将其所有元素初始化为 NULL。在服务端运转过程中，每次若出现新的客户端套接字，便在 coroutines 的对应槽位为其分配协程参数，然后运行协程 handle_client。若某个协程结束，便将其槽位清空。

```c
/* threebody.c ++ */
int main(void) {
    SimCr **coroutines = malloc(1024 * sizeof(SimCr *));
    SimServer *threebody = sim_server("localhost", "8080");
    if (!threebody) {
        fprintf(stderr, "sim_server failed!\backslashn");
        exit(-1);
    }
    /* 服务端程序运转 */
    SimStr *hi = sim_str("threebody: hi!");
    SimCr cr_init = {0, false, hi, NULL, NULL};
    while (1) {
        sim_server_run_once(threebody);
        if (sim_server_safe(threebody)) {
             /* 协程调度 */
            SimList *clients = sim_server_clients(threebody);
            for (SimList *it = clients; it; it = it->next) {
                int client = *(int *)it->data;
                SimCr *cr = coroutines[client];
                if (cr) {
                    if (cr->done) {
                        /* 释放协程参数 */
                        free(cr);
                        coroutines[client] = NULL;
                    } else {
                        /* 运行协程 */
                        handle_client(cr, threebody, client);
                    }
                } else {
                    /* 分配协程参数 */
                    cr = malloc(sizeof(SimCr));
                    *cr = cr_init;
                    coroutines[client] = cr;
                    /* 运行协程 */
                    handle_client(cr, threebody, client);
                }
            }
        }
    }
    sim_str_free(hi);
```

```
    for (int i = 0; i < 1024; i++) {
        SimCr *cr = coroutines[i];
        if (cr) free(cr);
    }
    free(coroutines);
    sim_server_free(threebody);
    return 0;
}
```

也许上述 threebody.c 是自古以来最为复杂的一个，它与之前的版本最显著的区别是，像 other-ywj 这样迟疑的客户端，即使她尚未给 threebody 发送数据，threebody 会先回复她，然后继续等她发送数据，于是协程参数里的 `dong` 先有值，而 `ding` 则后有值，亦即「咚叮」。像 ywj 这样毫不迟疑的客户端，则是「叮咚」。

若着实不理解这个协程版本的 threebody，也无妨，毕竟它过于聪明了……不过，它也非常愚蠢，用了有着 1024 个槽位的协程参数表。之所以是 1024，因为 `select` 实现的同步 I/O 多路复用机制最多能允许服务端接受 1024 个客户端的连接。若对其改进，可用链表或动态数组。

13.9 总结

作为一个创造者，主要任务不仅是消除作品中的愚蠢之处，同样也要消除其聪明之处。大器免成，大巧若拙，是设计者能致以用户的最大敬意。每个人都有其过人之处。若你觉得自己是个聪明的人，不过是因为你向来过于关心自己。

下面是最新的 Sim 项目的一切：

```
https://liyanrui.github.io/2025/c-network-programming/uncarved/sim-str.h
https://liyanrui.github.io/2025/c-network-programming/uncarved/sim-str.c
https://liyanrui.github.io/2025/c-network-programming/uncarved/sim-list.h
https://liyanrui.github.io/2025/c-network-programming/uncarved/sim-list.c
https://liyanrui.github.io/2025/c-network-programming/uncarved/sim-network.h
https://liyanrui.github.io/2025/c-network-programming/uncarved/sim-network.c
https://liyanrui.github.io/2025/c-network-programming/uncarved/threebody.c
```

跋

有很多人，认为 C 语言非常落后了，没有这个，没有那个。他们说，C++, Java, C#, Haskell, go, rust 以及各式各样的脚本语言，它们既有这个，又有那个……其中任何一门语言都比 C 语言更现代，更值得学习。

他们的看法实际上是对的，但也是错的。在语言上，他们是对的。在编程上，他们是错的。

C 语言没有面向对象编程，面向接口编程以及函数式编程的语法，也没有异常机制、类型自动推导以及内存自动回收等特性。许多年前，我喜欢与 C++ 语言爱好者争辩，我说 C 语言也能写面向对象程序，有时会以 GObject 库作为论据，而我却一直未能真正学会这个库的用法。这种争辩着实没有任何必要。无论我多么喜欢 C 语言，坦诚而言，C 语言在设计上是落后的。用 C 语言写程序，像是用先秦散文的形式写文章，语言的表现力不足以支撑大规模商业软件开发。

在编程上，C 却比当世任何一门语言都先进。这个角度是最近这两年我才想通的。C 的先进性，通读过 CSAPP 这本书（即《深入理解计算机系统》）的人应该都能有所觉察。C 语言可嵌入汇编语言，它有指针、有线程、有进程，有进程间通信，有套接字函数，有 Win32 API，有 Unix 或 Linux 数百个系统调用，有通用计算机，也有小型嵌入式设备。

C 的先进性，一言蔽之，虚心实腹。

你可能孜孜不倦于探究面向对象领域的各种设计模式，函数式编程领域的范畴论，动态语言五彩缤纷的开发框架，在语言层面上，这些开发范式的主要工作是，提高代码的复用性。软件开发的复杂度，凭借代码复用可以得到降低吗？不完全如此。许多复杂性只是从应用软件开发层面转移到编译器或解释器的开发工作中了，而且语言本身也变得日益复杂……试问，你学得会 C++ 或 Rust 吗？还有 Haskell 呢？

复杂性，从不会消失，它只会转移。

C 语言应对软件开发复杂度的策略一贯是分而治之，将复杂度分割为可控的单元，然后借助线程和进程实现这些单元，再以共享内存、管道或网络套接字等通信机制将这些单元连接为系统。这种应对软件复杂度的策略从 Unix 诞生的时候便已经确立了。

C++ 语言有值得称道的标准模板库，Java 语言有值得称道的内存自动回收机制，go 语言有值得称道的并发特性，rust 语言有值得称道的内存安全性……那么，C 语言为什么不能有值得称道的操作系统里的一切呢？当你用着语法更好的语言，努力描述着计算机世界的存在和风光时，你需要记着，C 语言早已与这个世界浑然一体了。

索引

f

fork **85**

g

gitpid **85**

k

kill **91**

s

signal **93**

w

wait **90**

waitpid **91**